

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>CycleAssess</title>
  <link
href="https://fonts.googleapis.com/css2?family=Sora:wght@300;400;500;600;700&family=Lora:ital,wght@0,400;0,600;1,400&display=swap" rel="stylesheet">
  <link href="assets/assess.css" rel="stylesheet">
  <!-- SylCycle cloud: Firebase + shared auth -->
  <script
src="https://www.gstatic.com/firebasejs/10.14.1/firebase-app-compat.js"></script>
  <script
src="https://www.gstatic.com/firebasejs/10.14.1/firebase-auth-compat.js"></script>
  <script
src="https://www.gstatic.com/firebasejs/10.14.1/firebase-firestore-compat.js"></scr
ipt>
  <script src="sylcycle-auth.js"></script>
  <script
src="https://cdnjs.cloudflare.com/ajax/libs/exceljs/4.3.0/exceljs.min.js"></script>
</head>
<body class="assess-booting">

<!-- == HEADER == -->
<header class="app-header no-print">
  <div style="display:flex;align-items:center;gap:9px">
    
    <div class="app-logo">Cycle<em>Assess</em></div>
  </div>
  <div style="display:flex;align-items:center;gap:8px">
    <a class="home-link" href="index.html"><span>Cycle Suite</span></a>
    <button class="settings-btn" onclick="openSettings()" title="Settings"
aria-label="Settings"><svg viewBox="0 0 24 24" aria-hidden="true" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round"
stroke-linejoin="round"><path d="M12 15.5A3.5 3.5 0 1 0 12 8a3.5 3.5 0 0 0
7.5z"/><path d="M19.4 15a1.7 1.7 0 0 0 .34 1.87l.06.06a2 2 0 1 1-2.83
2.83l-.06-.06A1.7 1.7 0 0 0 15 19.4a1.7 1.7 0 0 0-1 .6 1.7 1.7 0 0 0-.4 1.1V21a2 2
0 1 1-4 0v-.09A1.7 1.7 0 0 0 8.6 19.4a1.7 1.7 0 0 0-1.87.34l-.06.06a2 2 0 1
1-2.83-2.83l.06-.06A1.7 1.7 0 0 0 4.6 15a1.7 1.7 0 0 0-.6-1 1.7 1.7 0 0
0-1.1-.4H3a2 2 0 1 1 0-4h.09A1.7 1.7 0 0 0 4.6 8.6a1.7 1.7 0 0
0-.34-1.87l-.06-.06a2 2 0 1 1 2.83-2.83l.06.06A1.7 1.7 0 0 0 9 4.6a1.7 1.7 0 0
0 1-.6 1.7 1.7 0 0 0 .4-1.1V3a2 2 0 1 1 4 0v.09A1.7 1.7 0 0 0 15.4 4.6a1.7 1.7 0 0
0 1.87-.34l.06-.06a2 2 0 1 1 2.83 2.83l-.06.06A1.7 1.7 0 0 0 19.4 9c.27.4.7.68
1.2.78.18.04.37.06.56.06H21a2 2 0 1 1 0 4h-.09A1.7 1.7 0 0 0 19.4
15z"/></svg></button>
  <div
style="font-size:11px;font-weight:600;color:var(--ink-soft);letter-spacing:0.01em;t
ext-align:right;line-height:1.4">Built by<br><span

```

```

style="color:var(--accent);font-weight:700">Syl Magnin</span></div>
</div>
</header>

```

```

<!-- == BOTTOM NAV == -->
<nav class="bottom-nav hidden" id="bottom-nav">
  <button class="nav-btn active" id="nav-record" onclick="goTab('record')">
    <span class="nav-icon">📅</span>Record
  </button>
  <button class="nav-btn" id="nav-assess" onclick="goTab('assess')">
    <span class="nav-icon">📊</span>Assess
  </button>
  <button class="nav-btn" id="nav-students" onclick="goTab('students')">
    <span class="nav-icon">👥</span>Students
  </button>
  <button class="nav-btn" id="nav-heatmap" onclick="goTab('heatmap')">
    <span class="nav-icon">📈</span>Insights
  </button>
</nav>

```

```

<!-- == SCREEN: Setup == -->
<div class="screen active" id="screen-setup">
  <div class="eyebrow">Welcome</div>
  <h1 class="screen-title">Set up CycleAssess</h1>
  <p class="screen-sub">Domain and learning outcomes are identified automatically
from each photo. No rubric to configure.</p>
  <div class="card">
    <div class="form-group">
      <label class="form-label">Cycle</label>
      <select id="setup-cycle" onchange="onSetupCycleChange()"></select>
    </div>
    <div class="form-group">
      <label class="form-label">Year level</label>
      <select id="setup-year"></select>
      <div class="form-hint">Saved as your teaching context. You can change it
later in Settings if you teach or test another cycle.</div>
    </div>
  </div>
  <div class="card">
    <div class="form-group">
      <label class="form-label">Default context <span
style="font-weight:400;color:var(--ink-soft)">(optional)</span></label>
      <textarea id="setup-context" placeholder="e.g. School follows the French
curriculum at an international school. Add any class or reporting expectations
here."></textarea>
      <div class="form-hint">Applied to every assessment. You can add specific
context per session.</div>
    </div>
  </div>
  <div class="card">

```

```

    <div class="form-group">
      <label class="form-label">Anthropic API key</label>
      <input type="password" id="api-key" placeholder="sk-ant-api03-..."
autocomplete="off">
      <div class="form-hint">Used only on this device. Stored locally, not
shared.</div>
    </div>
  </div>
  <div class="btn-row" style="margin-top:4px">
    <button class="btn btn-primary" onclick="completeSetup()">Get started
-></button>
  </div>
</div>

<!-- == SCREEN: Assess == -->
<div class="screen" id="screen-record">

  <!-- Step 1: input -->
  <div id="batch-step-input">
    <div class="card record-class-card">
      <div class="section-label">Class</div>
      <select id="batch-class-select" style="font-size:13.5px;padding:9px 12px"
onchange="onBatchClassChange()">
        <option value="">– All classes –</option>
      </select>
    </div>
    <div class="card record-student-card">
      <div class="section-label">Student <span
style="font-weight:400;color:var(--ink-soft);text-transform:none;letter-spacing:0">
– add a new student to your roster</span></div>
      <select id="batch-student-select" onchange="onBatchStudentSelect()"
style="margin-bottom:8px;font-size:13.5px;padding:9px 12px">
        <option value="">– Select existing student –</option>
      </select>
      <div class="autocomplete-wrap">
        <input type="text" id="batch-student-name" placeholder="Or type a new
name..." autocomplete="off" style="font-size:13.5px">
      </div>
    </div>
    <div class="card record-activity-card">
      <div class="section-label">Activity input</div>
      <div class="input-type-row">
        <button class="input-type-btn active" id="itype-text"
onclick="setInputType('text')"><span class="it-icon">✎</span>Describe it</button>
        <button class="input-type-btn" id="itype-photo"
onclick="setInputType('photo')"><span class="it-icon">📷</span>Photo</button>
        <button class="input-type-btn" id="itype-pdf"
onclick="setInputType('pdf')"><span class="it-icon">📄</span>PDF</button>
      </div>

```

```

<!-- Text description -->
<div id="binput-text">
  <textarea id="batch-desc" placeholder="Describe the activity e.g. Drama
warm-up games focusing on turn-taking and expression. We did a maths worksheet on
calculating area using the formula."
style="min-height:90px;font-size:13.5px;line-height:1.65"></textarea>
</div>

<!-- Photo upload -->
<div id="binput-photo" style="display:none">
  <div class="upload-zone" style="padding:20px">
    <input type="file" accept="image/*" onchange="handleBatchPhoto(event)">
    <div class="upload-icon" style="font-size:24px">📷</div>
    <div class="upload-main" style="font-size:13px">Upload or photograph
the worksheet/activity</div>
  </div>
  <img id="batch-photo-preview" class="preview-img"
style="display:none;max-height:180px" alt="">
</div>

<!-- PDF upload -->
<div id="binput-pdf" style="display:none">
  <div class="upload-zone" style="padding:20px">
    <input type="file" accept="application/pdf"
onchange="handleBatchPDF(event)">
    <div class="upload-icon" style="font-size:24px">📄</div>
    <div class="upload-main" style="font-size:13px">Upload PDF worksheet or
document</div>
  </div>
  <div id="batch-pdf-name"
style="font-size:12px;color:var(--met);margin-top:8px;font-weight:500"></div>
</div>

<!-- Domain + outcomes (same picker as assess mode) -->
<div class="card record-domain-card">
  <div class="section-label">Domain</div>
  <select id="batch-domain-select" style="font-size:13.5px;padding:9px 12px"
onchange="onBatchDomainChange()">
    <option value="">Identify automatically</option>
    <option value="Mathematics">Mathematics</option>
    <option value="French">French</option>
    <option value="English Literacy">English Literacy</option>
    <option value="Science and Technology">Science and Technology</option>
    <option value="History and Geography">History and Geography</option>
    <option value="Civics and Moral Education">Civics and Moral
Education</option>
    <option value="Visual Arts">Visual Arts</option>
    <option value="Music">Music</option>
    <option value="PE">PE</option>
  </select>
</div>

```

```

        <option value="Modern Languages">Modern Languages</option>
    </select>
    <div class="outcome-picker" id="batch-outcome-picker">
        <div class="outcome-divider">Learning outcomes</div>
        <div class="outcome-picker-opts" id="batch-outcome-opts"></div>
        <div class="outcome-divider">Or</div>
        <label style="display:flex;align-items:center;gap:9px;padding:8px
11px;border:1.5px solid
var(--border);border-radius:8px;cursor:pointer;background:var(--bg);font-size:13px;
user-select:none">
            <input type="radio" name="batch-outcome-mode" value="ai"
id="batch-outcome-ai" checked onchange="onBatchOutcomeModeChange()"
style="width:auto;accent-color:var(--accent)">
            <span>Identify the most relevant outcomes automatically</span>
        </label>
        <label style="display:flex;align-items:center;gap:9px;padding:8px
11px;border:1.5px solid
var(--border);border-radius:8px;cursor:pointer;background:var(--bg);font-size:13px;
margin-top:6px;user-select:none">
            <input type="radio" name="batch-outcome-mode" value="custom"
id="batch-outcome-custom" onchange="onBatchOutcomeModeChange()"
style="width:auto;accent-color:var(--accent)">
            <span>Write my own outcome</span>
        </label>
        <div id="batch-custom-outcome-wrap" style="display:none;margin-top:6px">
            <input type="text" id="batch-custom-outcome-text" placeholder="Describe
the specific outcome..." style="font-size:13px">
        </div>
    </div>
    </div>

    <div class="btn-row">
        <button class="btn btn-primary" id="batch-identify-btn"
onclick="runBatchIdentify()">Next: set up class grid →</button>
    </div>
</div>

<!-- Step 2: class grid -->
<div id="batch-step-grid" style="display:none">
    <div class="card" style="margin-bottom:12px">
        <div class="section-label">Activity</div>
        <div id="batch-edit-banner"
style="display:none;background:var(--accent-lt);border:1px solid
var(--accent-bd);border-radius:8px;padding:9px
12px;font-size:12.5px;color:var(--accent);margin-bottom:10px"></div>
        <div id="batch-activity-summary"
style="font-size:13px;color:var(--ink-mid);line-height:1.55"></div>
        <div id="batch-domain-chip" class="domain-chip"
style="margin-top:10px;margin-bottom:0"></div>
        <div id="batch-outcomes-display"

```

```

style="margin-top:10px;display:flex;flex-direction:column;gap:6px"></div>
  <div id="batch-domain-warn"
style="display:none;background:#fff8e6;border:1px solid
#f0d080;border-radius:8px;padding:9px
13px;font-size:12.5px;color:#7a5a10;margin-top:10px;align-items:center;gap:10px">
    ⚠ Is this English literacy work? It may have been misidentified.
    <button onclick="batchContext.domain='English
Literacy';document.getElementById('batch-domain-chip').textContent='📖 English
Literacy';document.getElementById('batch-domain-warn').style.display='none'"
style="background:var(--met);color:white;border:none;border-radius:6px;padding:4px
10px;font-size:11.5px;font-weight:600;cursor:pointer;font-family:'Sora',sans-serif;
white-space:nowrap">Switch to English Literacy</button>
  </div>
  <button class="btn btn-ghost" onclick="backToBatchInput()"
style="margin-top:12px;font-size:12px">← Edit activity</button>
</div>

<div class="card">
  <div class="section-label">Class</div>
  <div class="quick-fill">
    <span class="quick-fill-label">Fill all presets:</span>
    <button class="qf-btn met" onclick="quickFillAll('met')">✓ All
Met</button>
    <button class="qf-btn wt" onclick="quickFillAll('wt')">~ All Working
Towards</button>
    <button class="qf-btn nm" onclick="quickFillAll('nm')">X All Not
Met</button>
  </div>
  <div style="font-size:11.5px;color:var(--ink-soft);margin:-2px 0
12px;line-height:1.5">
    Row buttons are presets for <strong>all selected outcomes</strong>. Use
<strong>Split</strong> on a student row to fine-tune individual outcomes before
saving.
  </div>
  <div class="class-grid-header">
    <div class="cg-name-hdr">Student</div>
    <div class="cg-status-hdr met">Met</div>
    <div class="cg-status-hdr wt">W/T</div>
    <div class="cg-status-hdr nm">N/M</div>
    <div style="width:138px"></div>
  </div>
  <div id="class-grid-rows"></div>
  <div id="batch-no-students"
style="display:none;text-align:center;padding:24px;font-size:13px;color:var(--ink-s
oft)">
    No students in roster yet – add one below.
  </div>
  <div
style="display:flex;gap:8px;margin-top:10px;padding-top:10px;border-top:1px solid
var(--border)">

```

```

        <input type="text" id="batch-new-student" placeholder="Add a student by
name..." style="font-size:13px;flex:1"
onkeydown="if(event.key==='Enter')addStudentToBatchGrid()">
        <button class="btn btn-secondary" onclick="addStudentToBatchGrid()"
style="font-size:13px;padding:8px 14px;white-space:nowrap">+ Add</button>
    </div>
</div>

    <div class="btn-row">
        <button class="btn btn-success" id="batch-save-btn"
onclick="saveBatchRecord()">✓ Save activity record</button>
        <button class="btn btn-secondary" onclick="backToBatchInput()">←
Back</button>
    </div>
</div>

</div>
</div>

<div class="screen" id="screen-assess">

    <!-- Session context banner -->
    <div id="session-banner"
style="display:none;background:var(--accent-lt);border:1px solid
var(--accent-bd);border-radius:10px;padding:9px
13px;margin-bottom:16px;font-size:12px;color:var(--accent);align-items:center;justi
fy-content:space-between">
        <span id="session-label-display"></span>
        <button class="btn-icon" onclick="clearSessionContext()"
title="Clear">×</button>
    </div>

    <div class="progress-wrap">
        <div class="progress-top">
            <span class="progress-lbl" id="progress-step">Ready to assess</span>
            <span class="progress-lbl" id="progress-count"></span>
        </div>
        <div class="progress-bar"><div class="progress-fill" id="progress-fill"
style="width:0%"></div></div>
    </div>

    <!-- Input state -->
    <div id="state-input">
        <div class="card">
            <div class="section-label">Class</div>
            <select id="assess-class-select" onchange="onAssessClassChange()"
style="font-size:13.5px;padding:9px 12px">
                <option value="">– All classes –</option>
            </select>
        </div>
    </div>

```

```

<div class="card">
  <div class="section-label">Student</div>
  <select id="student-select" onchange="onStudentSelect()"
style="margin-bottom:8px;font-size:13.5px;padding:9px 12px">
    <option value="">— Select existing student —</option>
  </select>
  <div class="autocomplete-wrap">
    <input type="text" id="student-name" placeholder="Or type a new name..."
oninput="onNameInput()" autocomplete="off">
    <div class="autocomplete-list" id="ac-list"></div>
  </div>
</div>
<div class="card">
  <div class="section-label">Domain</div>
  <select id="domain-select" style="font-size:13.5px;padding:9px 12px"
onchange="onDomainChange()">
    <option value="">Identify automatically</option>
    <option value="Mathematics">Mathematics</option>
    <option value="French">French</option>
    <option value="English Literacy">English Literacy</option>
    <option value="Science and Technology">Science and Technology</option>
    <option value="History and Geography">History and Geography</option>
    <option value="Civics and Moral Education">Civics and Moral
Education</option>
    <option value="Visual Arts">Visual Arts</option>
    <option value="Music">Music</option>
    <option value="PE">PE</option>
    <option value="Modern Languages">Modern Languages</option>
  </select>
  <div class="outcome-picker" id="outcome-picker">
    <div class="outcome-divider">Learning outcomes</div>
    <div class="outcome-picker-opts" id="outcome-opts"></div>
    <div class="outcome-divider">Or</div>
    <label style="display:flex;align-items:center;gap:9px;padding:8px
11px;border:1.5px solid
var(--border);border-radius:8px;cursor:pointer;background:var(--bg);font-size:13px;
user-select:none" id="ai-opt-label">
      <input type="radio" name="outcome-mode" value="ai" id="outcome-ai"
checked onchange="onOutcomeModeChange()"
style="width:auto;accent-color:var(--accent)">
      <span>Let the assessment identify the most relevant outcomes</span>
    </label>
    <label style="display:flex;align-items:center;gap:9px;padding:8px
11px;border:1.5px solid
var(--border);border-radius:8px;cursor:pointer;background:var(--bg);font-size:13px;
margin-top:6px;user-select:none" id="custom-opt-label">
      <input type="radio" name="outcome-mode" value="custom"
id="outcome-custom" onchange="onOutcomeModeChange()"
style="width:auto;accent-color:var(--accent)">
      <span>Write my own outcome</span>
    </label>
  </div>
</div>

```



```

    </label>
    <div class="outcome-custom-input" id="custom-outcome-wrap"
style="display:none">
      <input type="text" id="custom-outcome-text" placeholder="Describe the
specific outcome to assess..." style="margin-top:6px;font-size:13px">
    </div>
  </div>
</div>
<div class="card">
  <div class="section-label">Work sample</div>
  <div class="input-type-row" style="margin-bottom:12px">
    <button class="input-type-btn active" id="assess-itype-photo"
onclick="setAssessInputType('photo')"><span
class="it-icon"><img alt="photo icon" data-bbox="200 265 215 280"/></span>Photos</button>
    <button class="input-type-btn" id="assess-itype-pdf"
onclick="setAssessInputType('pdf')"><span
class="it-icon"><img alt="document icon" data-bbox="200 325 215 340"/></span>Document</button>
  </div>

  <div id="assess-photo-wrap">
    <div class="section-label" style="margin-bottom:9px">Work photos <span
style="font-weight:400;color:var(--ink-soft);text-transform:none;letter-spacing:0">
(up to 4)</span></div>
    <div class="photo-grid" id="photo-grid">
      <div class="photo-slot" id="slot-0">
        <div class="photo-slot-empty">
          <div class="ps-icon"><img alt="photo icon" data-bbox="200 525 215 540"/></div>
          <div class="ps-btns">
            <div class="ps-btn"><img alt="camera icon" data-bbox="200 565 215 580"/> Camera<input type="file" accept="image/*"
capture="environment" onchange="handlePhoto(event,0)"></div>
            <div class="ps-btn"><img alt="library icon" data-bbox="200 605 215 620"/> Library<input type="file" accept="image/*"
onchange="handlePhoto(event,0)"></div>
          </div>
        </div>
      </div>
      <div class="photo-slot" id="slot-1"
style="opacity:0.4;pointer-events:none">
        <div class="photo-slot-empty">
          <div class="ps-icon"><img alt="photo icon" data-bbox="200 765 215 780"/></div>
          <div class="ps-btns">
            <div class="ps-btn"><img alt="camera icon" data-bbox="200 805 215 820"/> Camera<input type="file" accept="image/*"
capture="environment" onchange="handlePhoto(event,1)"></div>
            <div class="ps-btn"><img alt="library icon" data-bbox="200 845 215 860"/> Library<input type="file" accept="image/*"
onchange="handlePhoto(event,1)"></div>
          </div>
        </div>
      </div>
      <div class="photo-slot" id="slot-2"
style="opacity:0.4;pointer-events:none">
        <div class="photo-slot-empty">

```

```

        <div class="ps-icon">📷</div>
        <div class="ps-btns">
            <div class="ps-btn">📷 Camera<input type="file" accept="image/*"
capture="environment" onchange="handlePhoto(event,2)"></div>
            <div class="ps-btn">📖 Library<input type="file" accept="image/*"
onchange="handlePhoto(event,2)"></div>
        </div>
    </div>
    <div class="photo-slot" id="slot-3"
style="opacity:0.4;pointer-events:none">
        <div class="photo-slot-empty">
            <div class="ps-icon">📷</div>
            <div class="ps-btns">
                <div class="ps-btn">📷 Camera<input type="file" accept="image/*"
capture="environment" onchange="handlePhoto(event,3)"></div>
                <div class="ps-btn">📖 Library<input type="file" accept="image/*"
onchange="handlePhoto(event,3)"></div>
            </div>
        </div>
    </div>
    <div class="photo-count" id="photo-count"></div>
</div>

<div id="assess-pdf-wrap" style="display:none">
    <div class="section-label" style="margin-bottom:9px">Document upload</div>
    <div class="upload-zone" style="padding:20px">
        <input type="file" id="assess-pdf-input" accept="application/pdf,.pdf"
onchange="handleAssessPDF(event)">
        <div class="upload-icon" style="font-size:24px">📄</div>
        <div class="upload-main" style="font-size:13px">Upload scanned document
or worksheet PDF</div>
        <div class="upload-hint">Use this for phone scans saved as a PDF.</div>
    </div>
    <div id="assess-pdf-meta"
style="display:none;align-items:center;justify-content:space-between;gap:10px;margi
n-top:10px;padding:10px 12px;border:1px solid
var(--border);border-radius:8px;background:var(--surface)">
        <div id="assess-pdf-name"
style="font-size:12px;color:var(--met);font-weight:600;min-width:0;word-break:break
-word"></div>
        <button class="btn btn-ghost" onclick="removeAssessPDF()"
style="white-space:nowrap">Remove</button>
    </div>
</div>
<div class="card" id="context-card" style="display:none">
    <div class="section-label">Additional context <span
style="font-weight:400;color:var(--ink-soft);text-transform:none;letter-spacing:0">

```

```

(optional)</span></div>
  <input type="text" id="session-context" placeholder="e.g. Written
multiplication, CM2">
</div>
  <div style="margin-bottom:10px">
    <button onclick="toggleContextCard()"
style="background:none;border:none;font-size:12px;color:var(--accent);cursor:pointe
r;font-family:'Sora',sans-serif;padding:0">
      + Add context for this assessment
    </button>
  </div>
  <div class="detail-toggle" id="detail-toggle"
onclick="toggleDetailedComment()">
    <div class="dt-switch"><div class="dt-knob"></div></div>
    <div>
      <div class="dt-label">Detailed comment</div>
      <div class="dt-hint">Comprehensive breakdown with strengths and areas to
focus on</div>
    </div>
  </div>
  <div class="btn-row">
    <button class="btn btn-primary" id="assess-btn" onclick="runAssessment()"
disabled>Assess →</button>
    <button class="btn btn-secondary" id="finish-btn" onclick="finishSession()"
style="display:none">View session →</button>
  </div>
</div>

<!-- Loading state -->
<div id="state-loading" style="display:none">
  <div class="card loading-box">
    <div class="spinner"></div>
    <div class="loading-msg" id="loading-msg">Analysing the work...</div>
  </div>
</div>

<!-- Result state -->
<div id="state-result" style="display:none">
  <div class="card">
    <div class="section-label">Result – <span id="result-name"
style="font-weight:700;color:var(--ink);text-transform:none;letter-spacing:0;font-s
ize:13px"></span></div>
    <div id="warn-illegible" style="display:none;background:#fff8e6;border:1px
solid #f0d080;border-radius:7px;padding:9px
12px;font-size:12px;color:#7a5a10;margin-bottom:12px">⚠ Some handwriting was
difficult to read. Review before confirming.</div>
    <div style="display:flex;align-items:center;gap:8px;margin-bottom:14px">
      <div id="result-domain" class="domain-chip" style="margin-bottom:0"></div>
      <select id="domain-override" onchange="overrideDomain(this.value)"
style="font-size:11.5px;padding:3px 9px;border:1.5px solid

```

```

var(--border);border-radius:20px;background:var(--bg);color:var(--ink-mid);font-fam
ily:'Sora',sans-serif;cursor:pointer;outline:none">
    <option value="">Change domain...</option>
    <option value="Mathematics">Mathematics</option>
    <option value="French">French</option>
    <option value="English Literacy">English Literacy</option>
    <option value="Science and Technology">Science and Technology</option>
    <option value="History and Geography">History and Geography</option>
    <option value="Civics and Moral Education">Civics and Moral
Education</option>
    <option value="Visual Arts">Visual Arts</option>
    <option value="Music">Music</option>
    <option value="PE">PE</option>
    <option value="Modern Languages">Modern Languages</option>
</select>
</div>
<div id="outcome-list" class="outcome-list"></div>
<div>
    <div class="section-label">Comment / next step <span
style="font-weight:400;color:var(--ink-soft);text-transform:none;letter-spacing:0">
(edit or leave blank)</span></div>
    <textarea id="comment-input" class="comment-edit" placeholder="No comment
generated – add one if needed, or leave blank."></textarea>
</div>
</div>
<div class="btn-row">
    <button class="btn btn-success" onclick="confirmResult()">✓ Confirm &
next</button>
    <button class="btn btn-secondary" onclick="reassess()">↺ Re-assess</button>
</div>
</div>

<!-- Session summary state -->
<div id="state-summary" style="display:none">
    <div class="eyebrow">Session complete</div>
    <h2 class="screen-title" style="font-size:22px;margin-bottom:6px">Session
summary</h2>
    <p style="font-size:13px;color:var(--ink-soft);margin-bottom:20px"
id="summary-sub"></p>
    <div id="summary-cards"></div>
    <div class="btn-row no-print">
        <button class="btn btn-primary" onclick="printSessionCards()">🖨 Print
comment cards</button>
        <button class="btn btn-secondary" onclick="newSession()">+ New
session</button>
    </div>
</div>
</div>

```

```

<!-- == SCREEN: Students == -->

</div>
<div class="screen" id="screen-students">
  <div
style="display:flex;align-items:center;justify-content:space-between;margin-bottom:
14px;flex-wrap:wrap;gap:10px">
    <h2 class="screen-title" style="margin-bottom:0">Students</h2>
    <div style="display:flex;gap:7px;flex-wrap:wrap">
      <button class="btn btn-primary" onclick="openAddStudent()"
style="font-size:12px;padding:7px 12px">+ Add student</button>
      <button class="btn btn-secondary" onclick="openBulkAssign()"
style="font-size:12px;padding:7px 12px">👤 Assign classes</button>
      <button class="btn btn-secondary" onclick="exportAllExcel()"
style="font-size:12px;padding:7px 12px">↓ Excel</button>
      <button class="btn btn-secondary" id="history-toggle-btn"
onclick="toggleHistoryPanel()" style="font-size:12px;padding:7px 12px">📅
History</button>
    </div>
  </div>
  <div class="class-tabs" id="class-tabs"></div>
  <div class="roster-search">
    <input type="text" id="roster-search" placeholder="Search students..."
oninput="renderRoster()">
  </div>
  <div id="history-panel" style="display:none;margin-bottom:20px">
    <div
style="display:flex;align-items:center;justify-content:space-between;margin-bottom:
12px">
      <div style="font-size:15px;font-weight:600;font-family:'Lora',serif">Session
History</div>
      <button onclick="toggleHistoryPanel()"
style="background:none;border:none;font-size:13px;color:var(--ink-soft);cursor:poin
ter;font-family:'Sora',sans-serif">× Close</button>
    </div>
    <div id="history-list"></div>
  </div>
  <div id="roster-list"></div>
</div>

<!-- == SCREEN: Bulk Assign == -->
<div class="screen" id="screen-bulk-assign">
  <button class="back-btn no-print" onclick="goTab('students')">Back to
students</button>
  <div class="eyebrow">Class management</div>
  <h2 class="screen-title" style="margin-bottom:6px">Assign classes</h2>
  <p style="font-size:13px;color:var(--ink-soft);margin-bottom:20px">Assign each
student to a class, and manage your class list.</p>

  <div class="card" style="margin-bottom:14px">

```

```

    <div class="section-label">Your classes</div>
    <div id="class-list-manage"
style="display:flex;flex-wrap:wrap;gap:7px;margin-bottom:12px"></div>
    <div style="display:flex;gap:8px">
        <input type="text" id="new-class-input" placeholder="New class name e.g.
CM1F" style="font-size:13px;flex:1"
onkeydown="if(event.key==='Enter')addNewClass()">
        <button class="btn btn-primary" onclick="addNewClass()" style="padding:8px
14px;font-size:13px">Add</button>
    </div>
</div>

<div class="card">
    <div class="section-label">Students</div>
    <div id="bulk-assign-rows"></div>
</div>

<div class="btn-row" style="margin-top:4px">
    <button class="btn btn-success" onclick="saveBulkAssign()">✓ Save
assignments</button>
</div>
</div>

<!-- = SCREEN: Profile = -->
<div class="screen" id="screen-profile">
    <button class="back-btn no-print" onclick="goTab('students')">Back to
students</button>
    <div id="profile-content"></div>
</div>

<!-- = SCREEN: Insights = -->
<div class="screen" id="screen-heatmap">
    <div
style="display:flex;align-items:center;justify-content:space-between;margin-bottom:
6px;flex-wrap:wrap;gap:8px">
        <h2 class="screen-title" style="margin-bottom:0">Class Insights</h2>
        <div class="hm-view-toggle">
            <button class="hm-view-btn" id="hm-view-list" onclick="setHmView('list')">≡
List</button>
            <button class="hm-view-btn active" id="hm-view-grid"
onclick="setHmView('grid')">⌘ Grid</button>
        </div>
    </div>
    <p style="font-size:13px;color:var(--ink-soft);margin-bottom:18px">Outcomes by
student – green Met, orange Working Towards, red Not Met.</p>
    <div class="heatmap-filter">
        <select id="hm-class-filter" class="filter-select" onchange="renderHeatmap()">
            <option value="">All classes</option>
        </select>
        <select id="hm-domain-filter" class="filter-select" onchange="renderHeatmap()">

```

```

        <option value="">All domains</option>
    </select>
    <select id="hm-date-filter" class="filter-select" onchange="renderHeatmap()">
        <option value="">All time</option>
        <option value="30">Last 30 days</option>
        <option value="90">Last 90 days (this term)</option>
        <option value="180">Last 6 months</option>
        <option value="365">This year</option>
    </select>
</div>
<div id="hm-dashboard">
    <div id="hm-cards" class="hm-card-grid"></div>
    <div id="hm-card-detail" class="hm-panel hm-card-detail hidden"></div>
    <div id="hm-coverage-watch" class="hm-panel"></div>
    <div id="hm-domain-summary" class="hm-panel"></div>
    <div id="hm-trend-summary" class="hm-panel"></div>
</div>
<details class="card heatmap-collapsible" id="heatmap-map-panel">
    <summary type="button">
        <div>
            <div class="heatmap-collapsible-title">Heatmap map</div>
            <div class="heatmap-collapsible-sub">Collapsed by default so the dashboard
hits first. Open it when you want the full grid or list view.</div>
        </div>
        <div class="heatmap-collapsible-arrow">v</div>
    </summary>
    <div class="heatmap-collapsible-body">
        <div class="heatmap-summary" id="hm-summary"></div>
        <div id="heatmap-list"></div>
    </div>
</details>
</div>

<!-- == Print area == -->
<div class="print-area" id="print-area"></div>

<!-- == Settings modal == -->
<div class="modal-overlay" id="settings-modal">
    <div class="modal-box">
        <div class="modal-title">Settings</div>
        <div class="form-group">
            <label class="form-label">API key</label>
            <input type="password" id="settings-key" placeholder="sk-ant-api03-...">
        </div>
        <div class="form-group">
            <label class="form-label">Default context</label>
            <textarea id="settings-context" style="min-height:65px"></textarea>
        </div>
        <div class="form-group">
            <label class="form-label">Cycle</label>

```

```

        <select id="settings-cycle" onchange="onSettingsCycleChange()"></select>
    </div>
    <div class="form-group">
        <label class="form-label">Year level</label>
        <select id="settings-year"></select>
        <div class="form-hint">This is the active teaching context for Record,
Students, Insights, History, and exports. Changing it is useful for admin testing
or a new school year.</div>
    </div>
    <div style="margin-top:18px;padding-top:14px;border-top:1px solid
var(--border);display:flex;flex-direction:column;gap:8px">
        <div style="display:flex;gap:8px">
            <button class="btn btn-secondary" onclick="exportData()" style="flex:1">↓
Export data backup</button>
            <label class="btn btn-secondary"
style="flex:1;cursor:pointer;justify-content:center">
                ↓ Import backup
                <input type="file" accept=".json" onchange="importData(event)"
style="display:none">
            </label>
        </div>
        <button class="btn btn-ghost" onclick="clearAssessmentsOnly()"
style="color:var(--wt);border-color:var(--wt-bd);margin-bottom:6px">🗑️ Clear all
assessments, all cycles (keep students)</button>
        <button class="btn btn-ghost" onclick="clearAllData()"
style="color:var(--nm);border-color:var(--nm-bd)">🗑️ Clear all student data, all
cycles</button>
    </div>
    <div class="modal-actions">
        <button class="btn btn-primary" onclick="saveSettings()">Save</button>
        <button class="btn btn-secondary" onclick="closeSettings()">Cancel</button>
    </div>
</div>
</div>

```

```

<!-- == Add student modal == -->
<div class="modal-overlay" id="add-student-modal">
    <div class="modal-box">
        <div class="modal-title">Add a student</div>
        <div class="form-group">
            <label class="form-label">Full name</label>
            <input type="text" id="add-student-name" placeholder="e.g. Liya Martin"
onkeydown="if(event.key==='Enter')saveNewStudent()">
        </div>
        <div class="form-group">
            <label class="form-label">Class <span
style="font-weight:400;color:var(--ink-soft)">(optional)</span></label>
            <select id="add-student-class" style="font-size:13.5px;padding:9px 12px">
                <option value="">— No class —</option>
            </select>
        </div>
    </div>
</div>

```



```

        </select>
    </div>
    <div class="form-group">
        <label class="form-label">Notes <span
style="font-weight:400;color:var(--ink-soft)">(optional)</span></label>
        <input type="text" id="add-student-notes" placeholder="e.g. EAL student, IEP
for reading...">
    </div>
    <div class="modal-actions">
        <button class="btn btn-primary" onclick="saveNewStudent()">Add
student</button>
        <button class="btn btn-secondary" onclick="closeAddStudent()">Cancel</button>
    </div>
</div>
</div>
<!-- == Edit assessment modal == -->
<div class="modal-overlay" id="edit-modal">
    <div class="modal-box">
        <div class="modal-title" id="edit-modal-title"></div>
        <div id="edit-modal-outcomes"></div>
        <div class="form-group" style="margin-top:12px">
            <label class="form-label">Comment / next step</label>
            <textarea id="edit-modal-comment"
style="min-height:80px;font-size:13px;line-height:1.65"></textarea>
        </div>
        <div class="form-group" id="edit-modal-batch-scope"
style="display:none;margin-top:12px;padding:12px;border:1px solid
var(--accent-bd);border-radius:10px;background:var(--accent-lt)">
            <label
style="display:flex;align-items:flex-start;gap:10px;cursor:pointer;user-select:none
">
                <input type="checkbox" id="edit-modal-apply-batch" checked
style="margin-top:2px;accent-color:var(--accent)">
                <span>
                    <span
style="display:block;font-size:12.5px;font-weight:600;color:var(--ink)">Apply
learning outcome changes to this whole batch record</span>
                    <span class="form-hint" id="edit-modal-batch-hint"
style="display:block;margin-top:4px"></span>
                </span>
            </label>
        </div>
        <div class="modal-actions">
            <button class="btn btn-primary" onclick="saveAssessmentEdit()">Save</button>
            <button class="btn btn-secondary" onclick="closeEditModal()">Cancel</button>
        </div>
    </div>
</div>

```

```

<!-- == Toast == -->
<div class="toast" id="app-toast"></div>

<!-- == Confirm dialog == -->
<div class="confirm-overlay" id="confirm-overlay">
  <div class="confirm-box">
    <div class="confirm-msg" id="confirm-msg"></div>
    <div class="confirm-sub" id="confirm-sub"></div>
    <div class="confirm-btns">
      <button class="btn btn-secondary" id="confirm-cancel-btn">Cancel</button>
      <button class="btn btn-primary" id="confirm-ok-btn">OK</button>
    </div>
  </div>
</div>

<!-- == Rename student modal == -->
<div class="modal-overlay" id="rename-modal">
  <div class="modal-box">
    <div class="modal-title">Rename student</div>
    <div class="form-group">
      <label class="form-label">New name</label>
      <input type="text" id="rename-input"
onkeydown="if(event.key==='Enter')saveRename()">
    </div>
    <div class="modal-actions">
      <button class="btn btn-primary" onclick="saveRename()">Save</button>
      <button class="btn btn-secondary" onclick="closeRename()">Cancel</button>
    </div>
  </div>
</div>
<script src="assets/cycle3-domain-library.js"></script>
<script src="assets/cycle-domain-library.js"></script>
<script src="assets/storage.js"></script>
<script>
const CYCLE_LABELS = {
  cycle1: 'Cycle 1',
  cycle2: 'Cycle 2',
  cycle3: 'Cycle 3'
};
const CYCLE_YEAR_LEVELS = {
  cycle1: ['PS', 'MS', 'GS'],
  cycle2: ['CP', 'CE1', 'CE2'],
  cycle3: ['CM1', 'CM2', 'CM1/CM2']
};
const CYCLE_DOMAIN_LIBRARIES = window.CYCLE_DOMAIN_LIBRARIES || { cycle3:
window.CYCLE3_DOMAIN_LIBRARY || {} };
let currentCycle = 'cycle3';
let currentYearLevel = 'CM1';

```

```

function normaliseCycle(value) {
  return CYCLE_LABELS[value] ? value : 'cycle3';
}

function getCycleYears(cycle = currentCycle) {
  return CYCLE_YEAR_LEVELS[normaliseCycle(cycle)] || CYCLE_YEAR_LEVELS.cycle3;
}

function getCycleLabel(cycle = currentCycle) {
  return CYCLE_LABELS[normaliseCycle(cycle)] || CYCLE_LABELS.cycle3;
}

function getAssessCycleContext() {
  return `${getCycleLabel()} (${currentYearLevel})`;
}

function getCurrentDomainLibrary() {
  return CYCLE_DOMAIN_LIBRARIES[currentCycle] || CYCLE_DOMAIN_LIBRARIES.cycle3 ||
{};
}

function cloneDomainEntry(entry) {
  if (Array.isArray(entry)) return entry.slice();
  if (entry && typeof entry === 'object') {
    return Object.fromEntries(
      Object.entries(entry).map(([subdomain, outcomes]) => [subdomain,
Array.isArray(outcomes) ? outcomes.slice() : []])
    );
  }
  return [];
}

function flattenDomainEntry(entry) {
  if (Array.isArray(entry)) return entry.slice();
  if (entry && typeof entry === 'object') return Object.values(entry).flat();
  return [];
}

function buildDomainStructure(library) {
  return Object.fromEntries(Object.entries(library || {}).map(([domain, entry]) =>
[
  domain,
  cloneDomainEntry(entry)
]));
}

let CYCLE3_DOMAIN_STRUCTURE = buildDomainStructure(getCurrentDomainLibrary());

let CYCLE3_OUTCOMES = Object.fromEntries(
  Object.entries(CYCLE3_DOMAIN_STRUCTURE).map(([domain, entry]) => [domain,

```

```

flattenDomainEntry(entry)])
);

let SUBDOMAIN_AWARE_DOMAINS = new Set(
  Object.entries(CYCLE3_DOMAIN_STRUCTURE)
    .filter(([ , entry]) => entry && !Array.isArray(entry) && typeof entry ===
'object')
    .map(([domain]) => domain)
);

function refreshActiveCurriculumLibrary() {
  CYCLE3_DOMAIN_STRUCTURE = buildDomainStructure(getCurrentDomainLibrary());
  CYCLE3_OUTCOMES = Object.fromEntries(
    Object.entries(CYCLE3_DOMAIN_STRUCTURE).map(([domain, entry]) => [domain,
flattenDomainEntry(entry)])
  );
  SUBDOMAIN_AWARE_DOMAINS = new Set(
    Object.entries(CYCLE3_DOMAIN_STRUCTURE)
      .filter(([ , entry]) => entry && !Array.isArray(entry) && typeof entry ===
'object')
      .map(([domain]) => domain)
  );
  APPROVED_DOMAIN_LIST = Object.keys(CYCLE3_DOMAIN_STRUCTURE);
}

function getDomainOutcomeGroups(domain) {
  const entry = CYCLE3_DOMAIN_STRUCTURE[domain];
  if (!entry) return [];
  if (Array.isArray(entry)) return [{ title: '', outcomes: entry }];
  return Object.entries(entry).map(([title, outcomes]) => ({ title, outcomes:
Array.isArray(outcomes) ? outcomes : [] }));
}

function formatOutcomeSelectionValue(subdomain, outcome) {
  return subdomain ? `${subdomain} - ${outcome}` : outcome;
}

function renderOutcomeOptions(domain, optsId, prefix, toggleFnName) {
  const opts = document.getElementById(optsId);
  const groups = getDomainOutcomeGroups(domain);
  opts.innerHTML = groups.map((group, groupIndex) => {
    const optionHtml = group.outcomes.map((outcome, outcomeIndex) => {
      const optionId = `${prefix}-${groupIndex}-${outcomeIndex}`;
      const storedValue = formatOutcomeSelectionValue(group.title, outcome);
      return `
        <div class="outcome-opt" id="${optionId}" data-value="${esc(storedValue)}"
onclick="${toggleFnName}('${optionId}')">
          <div class="outcome-opt-check"></div>
          <span class="outcome-opt-text">${esc(outcome)}</span>
        </div>`;
    });
  });
}

```

```

    }).join('');

    if (!group.title) return optionHtml;
    return `
      <div class="outcome-group">
        <div class="outcome-group-title">${esc(group.title)}</div>
        ${optionHtml}
      </div>`;
  }).join('');
}

```

```

let APPROVED_DOMAIN_LIST = Object.keys(CYCLE3_DOMAIN_STRUCTURE);
const APPROVED_DOMAIN_ALIASES = {
  'mathematiques': 'Mathematics',
  'maths': 'Mathematics',
  'français': 'French',
  'french': 'French',
  'english literacy': 'English Literacy',
  'anglais': 'English Literacy',
  'sciences et technologie': 'Science and Technology',
  'science and technology': 'Science and Technology',
  'histoire géographie': 'History and Geography',
  'history and geography': 'History and Geography',
  'emc': 'Civics and Moral Education',
  'civics and moral education': 'Civics and Moral Education',
  'arts plastiques': 'Visual Arts',
  'visual arts': 'Visual Arts',
  'éducation musicale': 'Music',
  'music': 'Music',
  'eps': 'PE',
  'pe': 'PE',
  'langues vivantes': 'Modern Languages',
  'langues vivantes étrangères ou régionales': 'Modern Languages',
  'langues vivantes étrangères': 'Modern Languages',
  'langue vivante': 'Modern Languages',
  'modern languages': 'Modern Languages'
};

```

```

function normalizeCurriculumText(value = '') {
  return String(value || '')
    .normalize('NFD')
    .replace(/[-']/g, '')
    .toLowerCase()
    .replace(/["'"/]/g, '')
    .replace(/\s*[\s*]/g, ' ')
    .replace(/[\^a-z0-9]+/g, ' ')
    .trim();
}

```

```

function tokenizeCurriculumText(value = '') {

```

```

    return normalizeCurriculumText(value)
        .split(' ')
        .filter(Boolean)
        .filter(token => token.length > 2 || /^\\d+$/.test(token));
}

function getApprovedDomainListText() {
    return APPROVED_DOMAIN_LIST.join(', ');
}

function buildDomainRoutingRules() {
    const rules = [];
    if (APPROVED_DOMAIN_LIST.includes('English Literacy')) {
        rules.push('- English spelling, vocabulary, reading, writing, grammar and language arts ALWAYS belong to "English Literacy".');
    } else if (APPROVED_DOMAIN_LIST.includes('Literacy')) {
        rules.push('- Reading, writing, grammar, spelling, vocabulary and language arts belong to "Literacy".');
    } else if (APPROVED_DOMAIN_LIST.includes('Oral and Written Language')) {
        rules.push('- Oral language, stories, vocabulary, letters, early writing and phonological awareness belong to "Oral and Written Language".');
    }
    if (APPROVED_DOMAIN_LIST.includes('Modern Languages')) {
        rules.push('- "Modern Languages" is ONLY for foreign-language learning.');
```

```

        cycleSelect.innerHTML = Object.entries(CYCLE_LABELS)
            .map(([value, label]) => `<option
value="${escAttr(value)}">${esc(label)}</option>`)
            .join('');
        cycleSelect.value = normaliseCycle(cycle);
    });
    ['settings-year', 'setup-year'].forEach(id => {
        const yearSelect = document.getElementById(id);
        if (!yearSelect) return;
        const years = getCycleYears(cycle);
        yearSelect.innerHTML = years.map(level => `<option
value="${escAttr(level)}">${esc(level)}</option>`).join('');
        yearSelect.value = years.includes(year) ? year : years[0];
    });
}

function applyAssessCycle(cycle = 'cycle3', year = '') {
    currentCycle = normaliseCycle(cycle);
    const years = getCycleYears(currentCycle);
    currentYearLevel = years.includes(year) ? year : years[0];
    refreshActiveCurriculumLibrary();
    populateAssessCycleControls(currentCycle, currentYearLevel);
    populateAllDomainSelects();
}

function onSettingsCycleChange() {
    const cycle = document.getElementById('settings-cycle')?.value || currentCycle;
    applyAssessCycle(cycle, document.getElementById('settings-year')?.value || '');
}

function onSetupCycleChange() {
    const cycle = document.getElementById('setup-cycle')?.value || currentCycle;
    applyAssessCycle(cycle, document.getElementById('setup-year')?.value || '');
}

function normalizeApprovedDomain(value = '') {
    const raw = String(value || '').trim();
    if (!raw) return '';
    const normalized = normalizeCurriculumText(raw);
    if (APPROVED_DOMAIN_ALIASES[normalized]) return
    APPROVED_DOMAIN_ALIASES[normalized];
    return APPROVED_DOMAIN_LIST.find(domain => normalizeCurriculumText(domain) ===
    normalized) || raw;
}

function normalizeStoredDomain(value = '') {
    const domain = normalizeApprovedDomain(value);
    return APPROVED_DOMAIN_LIST.includes(domain) ? domain : String(value ||
    '').trim();
}

```

```

function normaliseYearForCycle(cycle = 'cycle3', year = '') {
  const safeCycle = normaliseCycle(cycle);
  const years = getCycleYears(safeCycle);
  return years.includes(year) ? year : years[0];
}

function getAssessCycleSnapshot() {
  return {
    cycle: normaliseCycle(currentCycle),
    yearLevel: normaliseYearForCycle(currentCycle, currentYearLevel)
  };
}

function normalizeStoredCycleFields(record, fallback = {}) {
  if (!record || typeof record !== 'object') return record;
  const fallbackCycle = normaliseCycle(fallback.cycle || 'cycle3');
  record.cycle = normaliseCycle(record.cycle || record.preferredCycle ||
fallbackCycle);
  record.yearLevel = normaliseYearForCycle(record.cycle, record.yearLevel ||
fallback.yearLevel || '');
  return record;
}

function assessmentMatchesActiveCycle(assessment) {
  const cycle = normaliseCycle(assessment?.cycle || 'cycle3');
  return cycle === currentCycle;
}

function filterAssessmentsForActiveCycle(assessments = []) {
  return (assessments || []).filter(assessmentMatchesActiveCycle);
}

function sessionForActiveCycle(session) {
  if (!session || typeof session !== 'object') return null;
  const fallbackCycle = normaliseCycle(session.cycle || 'cycle3');
  const students = (session.students || []).filter(entry => {
    const entryCycle = normaliseCycle(entry?.cycle || fallbackCycle);
    return entryCycle === currentCycle;
  });
  if (!students.length) return null;
  return { ...session, students };
}

function studentForActiveCycle(student) {
  if (!student || typeof student !== 'object') return student;
  return {
    ...student,
    assessments: filterAssessmentsForActiveCycle(student.assessments || [])
  };
}

```



```

}

function studentsForActiveCycle(students = []) {
  return (students || [])
    .map(studentForActiveCycle)
    .filter(student => (student.assessments || []).length);
}

function normalizeStoredAssessmentData(data) {
  if (!data || typeof data !== 'object') return { students: {} };
  const defaultCycle = normaliseCycle(data.preferredCycle || 'cycle3');
  const defaultYear = normaliseYearForCycle(defaultCycle, data.yearLevel || '');
  data.students = data.students && typeof data.students === 'object' ?
data.students : {};
  Object.values(data.students).forEach(student => {
    if (!student || typeof student !== 'object') return;
    if (Array.isArray(student.assessments)) {
      student.assessments.forEach(assessment => {
        if (assessment && assessment.domain) assessment.domain =
normalizeStoredDomain(assessment.domain);
        normalizeStoredCycleFields(assessment, { cycle: defaultCycle, yearLevel:
defaultYear });
      });
    }
    if (student.domainComments && typeof student.domainComments === 'object') {
      const nextComments = {};
      Object.entries(student.domainComments).forEach(([domain, comment]) => {
        nextComments[normalizeStoredDomain(domain)] = comment;
      });
      student.domainComments = nextComments;
    }
  });
  if (Array.isArray(data.sessions)) {
    data.sessions.forEach(session => {
      normalizeStoredCycleFields(session, { cycle: defaultCycle, yearLevel:
defaultYear });
      if (session && session.domain) session.domain =
normalizeStoredDomain(session.domain);
      if (session && Array.isArray(session.students)) {
        session.students.forEach(entry => {
          normalizeStoredCycleFields(entry, { cycle: session.cycle, yearLevel:
session.yearLevel });
          if (entry && entry.domain) entry.domain =
normalizeStoredDomain(entry.domain);
        });
      }
    });
  }
  return data;
}

```

```

function getAllowedOutcomeEntries(domain) {
  const groups = getDomainOutcomeGroups(domain);
  const entries = [];
  groups.forEach(group => {
    (group.outcomes || []).forEach(outcome => {
      entries.push({
        subdomain: group.title || '',
        outcome,
        stored: formatOutcomeSelectionValue(group.title, outcome)
      });
    });
  });
  return entries;
}

function buildOutcomeLibraryPrompt(domain) {
  const groups = getDomainOutcomeGroups(domain);
  if (!groups.length) return '- No approved outcomes found for this domain.';
  return groups.map(group => {
    const lines = (group.outcomes || []).map(outcome => ` -
    ${outcome}`)
    .join('\n');
    if (group.title) return ` - ${group.title}:\n${lines}`;
    return lines;
  })
  .join('\n');
}

function scoreOutcomeCandidate(rawValue, candidateValue) {
  const rawNorm = normalizeCurriculumText(rawValue);
  const candidateNorm = normalizeCurriculumText(candidateValue);
  if (!rawNorm || !candidateNorm) return 0;
  if (rawNorm === candidateNorm) return 1;
  if (rawNorm.includes(candidateNorm) || candidateNorm.includes(rawNorm)) return
  0.94;

  const rawTokens = new Set(tokenizeCurriculumText(rawValue));
  const candidateTokens = new Set(tokenizeCurriculumText(candidateValue));
  if (!rawTokens.size || !candidateTokens.size) return 0;

  let overlap = 0;
  rawTokens.forEach(token => { if (candidateTokens.has(token)) overlap += 1; });
  if (!overlap) return 0;

  const recall = overlap / candidateTokens.size;
  const precision = overlap / rawTokens.size;
  return (recall * 0.72) + (precision * 0.28);
}

function canonicalizeOutcomeChoice(domain, rawValue) {
  const raw = String(rawValue || '').trim();

```

```

if (!domain || !raw) return null;

const entries = getAllowedOutcomeEntries(domain);
let best = null;
let bestScore = 0;

entries.forEach(entry => {
  const score = Math.max(
    scoreOutcomeCandidate(raw, entry.stored),
    scoreOutcomeCandidate(raw, entry.outcome)
  );
  if (score > bestScore) {
    best = entry;
    bestScore = score;
  }
});

return bestScore >= 0.72 ? best : null;
}

function enforceApprovedOutcomes(domain, outcomes) {
  if (!domain || !Array.isArray(outcomes)) return [];
  const mapped = [];
  const seen = new Set();

  outcomes.forEach(outcomeObj => {
    const raw = outcomeObj?.fr || outcomeObj?.text || outcomeObj?.en || '';
    const match = canonicalizeOutcomeChoice(domain, raw);
    if (!match) return;
    if (seen.has(match.stored)) return;
    seen.add(match.stored);
    mapped.push({
      fr: match.stored,
      en: match.stored,
      status: ['met', 'wt', 'nm'].includes(outcomeObj?.status) ? outcomeObj.status
: 'wt'
    });
  });

  return mapped;
}

async function callAnthropicJSON({ messageContent, maxTokens = 800, timeoutMs =
60000, includePdfBeta = false }) {
  const controller = new AbortController();
  const timeout = setTimeout(() => controller.abort(), timeoutMs);
  try {
    let res;
    try {
      const headers = {

```

```

        'Content-Type': 'application/json',
        'x-api-key': window.__key,
        'anthropic-version': '2023-06-01',
        'anthropic-dangerous-direct-browser-access': 'true'
    });
    if (includePdfBeta) headers['anthropic-beta'] = 'pdfs-2024-09-25';
    res = await fetch('https://api.anthropic.com/v1/messages', {
        method: 'POST',
        signal: controller.signal,
        headers,
        body: JSON.stringify({
            model: 'claude-opus-4-5',
            max_tokens: maxTokens,
            messages: [{ role: 'user', content: messageContent }]
        })
    });
} catch (fe) {
    if (fe.name === 'AbortError') throw new Error('Request timed out. Check your
connection and try again.');
```

throw new Error('Network error – could not reach the API.');

```

}

let data;
try {
    data = await res.json();
} catch (e) {
    throw new Error('Could not read response (status ' + res.status + ').');
```

}

```

    if (!res.ok || data.error) throw new Error(data.error?.message || ('API error '
+ res.status));
    const raw = (data.content || []).map(block => block.text || '').join('');
    try {
        return JSON.parse(raw.replace(/```json|```/g, '').trim());
    } catch (e) {
        throw new Error('Could not parse response. Please try again.');
```

}

```

    } finally {
        clearTimeout(timeout);
    }
}

}


```

```

async function identifyApprovedDomain({ messageContent, timeoutMs = 45000,
includePdfBeta = false, activityDescription = '' }) {
    const descriptionLine = activityDescription ? `Teacher context:
${activityDescription}` : '';
    const prompt = `You are identifying the curriculum domain for a
${getAssessCycleContext()} assessment at an international school following the
French curriculum.
```

Choose EXACTLY one approved domain from this list:

```
${getApprovedDomainListText()}
```

Critical school rule:

```
${buildDomainRoutingRules()}
```

```
${descriptionLine}
```

Return ONLY valid JSON:

```
{
  "domain": "<one exact approved domain string>"
};
const result = await callAnthropicJSON({
  messageContent: [...messageContent, { type: 'text', text: prompt }],
  maxTokens: 120,
  timeoutMs,
  includePdfBeta
});
const domain = normalizeApprovedDomain(result?.domain || '');
if (!APPROVED_DOMAIN_LIST.includes(domain)) {
  throw new Error('AI returned a domain outside the approved list.');
```

```
  }
  return domain;
}
```

```
/* =====
  DB LAYER
===== */
```

```
const DB = {
  KEY: 'ca_v2_data',
  SETTINGS_KEY: 'ca_v2_settings',

  writeLocalJson(key, value) {
    try {
      localStorage.setItem(key, JSON.stringify(value));
      return true;
    } catch(e) {
      return false;
    }
  },

  loadData() {
    if (window.SylCycleStorage) return
    normalizeStoredAssessmentData(SylCycleStorage.readJson(this.KEY, {students:{}}));
    try { return
    normalizeStoredAssessmentData(JSON.parse(localStorage.getItem(this.KEY) ||
    '{"students":{}}')); }
    catch(e) { return {students:{}}; }
  },
  saveData(data) {
    const saved = window.SylCycleStorage
```

```

        ? SylCycleStorage.writeJson(this.KEY, data)
        : this.writeLocalJson(this.KEY, data);
    if (saved) {
        if (window.SylCycle) SylCycle.cloudSave(this.KEY, data);
    } else {
        flash('Storage nearly full -- export a backup soon.', 'warn');
    }
},
loadSettings() {
    if (window.SylCycleStorage) return SylCycleStorage.readJson(this.SETTINGS_KEY,
{}));
    try { return JSON.parse(localStorage.getItem(this.SETTINGS_KEY) || '{}'); }
    catch(e) { return {}; }
},
saveSettings(s) {
    const saved = window.SylCycleStorage
        ? SylCycleStorage.writeJson(this.SETTINGS_KEY, s)
        : this.writeLocalJson(this.SETTINGS_KEY, s);
    if (saved && window.SylCycle) SylCycle.cloudSave(this.SETTINGS_KEY, s);
},

saveAssessment(name, assessment) {
    normalizeStoredCycleFields(assessment, getAssessCycleSnapshot());
    const data = this.loadData();
    if (!data.students[name]) data.students[name] = { name, assessments: [] };
    data.students[name].assessments.unshift(assessment);
    this.saveData(data);
},
getStudent(name) {
    const data = this.loadData();
    return data.students[name] || null;
},
getAllStudents() {
    const data = this.loadData();
    return Object.values(data.students).sort((a,b) =>
a.name.localeCompare(b.name));
},
getStudentNames() {
    return this.getAllStudents().map(s => s.name);
},
deleteAssessment(studentName, assessmentId) {
    const data = this.loadData();
    if (data.students[studentName]) {
        data.students[studentName].assessments =
data.students[studentName].assessments.filter(a => a.id !== assessmentId);
        // Do NOT delete the student – they stay in the roster even with 0
assessments
    }
    this.saveData(data);
},

```

```

updateAssessment(studentName, assessmentId, updates) {
  const data = this.loadData();
  const st = data.students[studentName];
  if (st) {
    const a = st.assessments.find(a => a.id === assessmentId);
    if (a) Object.assign(a, updates);
  }
  this.saveData(data);
},
updateStudentNotes(name, notes) {
  const data = this.loadData();
  if (data.students[name]) data.students[name].notes = notes;
  this.saveData(data);
},
updateStudentDomainComment(name, domain, comment) {
  const data = this.loadData();
  if (!data.students[name] || !domain) return;
  if (!data.students[name].domainComments) data.students[name].domainComments =
{};
  if (comment) data.students[name].domainComments[domain] = comment;
  else delete data.students[name].domainComments[domain];
  this.saveData(data);
},
saveSession(session) {
  normalizeStoredCycleFields(session, getAssessCycleSnapshot());
  if (Array.isArray(session.students)) {
    session.students.forEach(entry => normalizeStoredCycleFields(entry, { cycle:
session.cycle, yearLevel: session.yearLevel }));
  }
  const data = this.loadData();
  if (!data.sessions) data.sessions = [];
  data.sessions.unshift(session);
  // Keep last 100 sessions
  if (data.sessions.length > 100) data.sessions = data.sessions.slice(0, 100);
  this.saveData(data);
},
getSessions() {
  const data = this.loadData();
  return data.sessions || [];
},
deleteSession(sessionId) {
  const data = this.loadData();
  const session = (data.sessions || []).find(s => s.id === sessionId);
  if (!session) return;
  // Remove each assessment from each student's profile
  session.students.forEach(entry => {
    const st = data.students[entry.name];
    if (st) {
      st.assessments = st.assessments.filter(a => a.id !== entry.assessmentId);
      if (st.assessments.length === 0) delete data.students[entry.name];
    }
  });
  this.saveData(data);
}

```

```

    }
  });
  data.sessions = data.sessions.filter(s => s.id !== sessionId);
  this.saveData(data);
},
getClasses() {
  const data = this.loadData();
  return (data.classes || []).sort((a,b) => a.localeCompare(b));
},
saveClass(name) {
  const data = this.loadData();
  if (!data.classes) data.classes = [];
  if (!data.classes.includes(name)) { data.classes.push(name);
this.saveData(data); }
},
deleteClass(name) {
  const data = this.loadData();
  if (data.classes) data.classes = data.classes.filter(c => c !== name);
  // Unassign students from deleted class
  Object.values(data.students).forEach(st => { if (st.class === name) st.class =
''; });
  this.saveData(data);
},
updateStudentClass(studentName, className) {
  const data = this.loadData();
  if (data.students[studentName]) data.students[studentName].class = className;
  this.saveData(data);
},
getStudentsByClass(className) {
  return this.getAllStudents().filter(s => (s.class || '') === className);
}
};

```

```

/* =====
  APP STATE
  ===== */

```

```

let currentPhotos = []; // array of {base64, mimeType}
let currentThumb = null;
let currentPDFData = null; // {base64, name}
let assessInputType = 'photo';
let currentResult = null;
let sessionStudents = []; // current session batch
let editingStudent = null;
let editingAssessmentId = null;
let editingBatchSessionId = null;
let editingBatchAssessmentCount = 0;
let currentProfileStudent = null;
let profileSortMode = 'chronological';
let profileMasteryDomainFilter = 'all';
let profileHistoryOutcomeFilter = '';

```



```

let profileHistoryDomainFilter = '';
let profileMasteryPanelOpen = true;
let profileHistoryPanelOpen = false;
let profileHistoryShowAll = false;
let profileCommentPanelOpen = false;
let profileMapPanelOpen = false;
let batchEditingSessionId = null;
let batchEditReturn = null;
let detailedComment = false;
const STATUS_LABEL = { met: 'Met', wt: 'Working Towards', nm: 'Not Met' };

/* =====
   INIT
===== */

function initApp() {
  const settings = DB.loadSettings();
  applyAssessCycle(settings.preferredCycle || 'cycle3', settings.yearLevel || '');
  if (settings.apiKey) {
    window.__key = settings.apiKey;
    showMainApp();
  } else {
    showScreen('setup');
  }
  document.body.classList.remove('assess-booting');
}

function showMainApp() {
  document.getElementById('bottom-nav').classList.remove('hidden');
  goTab('record');
}

function completeSetup() {
  const key = document.getElementById('api-key').value.trim();
  if (!key) { flash('Please enter your API key.');
```

```

function goTab(tab) {
  showScreen(tab);
  document.querySelectorAll('.nav-btn').forEach(b => b.classList.remove('active'));
  const nb = document.getElementById('nav-' + tab);
  if (nb) nb.classList.add('active');
  if (tab === 'students') renderRoster();
  if (tab === 'record') { populateBatchClassSelect();
populateBatchStudentSelect(); }
  if (tab === 'assess') { populateAssessClassSelect(); populateStudentSelect(); }
  if (tab === 'heatmap') renderHeatmap();
}

function toggleHistoryPanel() {
  const panel = document.getElementById('history-panel');
  const btn = document.getElementById('history-toggle-btn');
  const isOpen = panel.style.display !== 'none';
  panel.style.display = isOpen ? 'none' : 'block';
  btn.style.background = isOpen ? '' : 'var(--accent-lt)';
  btn.style.borderColor = isOpen ? '' : 'var(--accent-bd)';
  btn.style.color = isOpen ? '' : 'var(--accent)';
  if (!isOpen) renderHistory();
}

/* =====
  ASSESS TAB
  ===== */
function toggleContextCard() {
  const card = document.getElementById('context-card');
  card.style.display = card.style.display === 'none' ? 'block' : 'none';
}

function toggleDetailedComment() {
  detailedComment = !detailedComment;
  document.getElementById('detail-toggle').classList.toggle('on', detailedComment);
}

function onDomainChange() {
  const domain = document.getElementById('domain-select').value;
  const picker = document.getElementById('outcome-picker');

  if (!domain) {
    picker.classList.remove('visible');
    return;
  }

  picker.classList.add('visible');
  renderOutcomeOptions(domain, 'outcome-opts', 'opt', 'toggleOutcomeOpt');
}

```

```

// Reset to AI mode
document.getElementById('outcome-ai').checked = true;
onOutcomeModeChange();
}

function toggleOutcomeOpt(optionId) {
  // Switch to manual selection mode when clicking an outcome
  document.getElementById('outcome-ai').checked = false;
  document.getElementById('outcome-custom').checked = false;
  document.getElementById('custom-outcome-wrap').style.display = 'none';
  const el = document.getElementById(optionId);
  if (!el) return;
  el.classList.toggle('selected');
  el.querySelector('.outcome-opt-check').textContent =
el.classList.contains('selected') ? '✓' : '';
}

function onOutcomeModeChange() {
  const mode = document.querySelector('input[name="outcome-mode"]:checked')?.value;
  const customWrap = document.getElementById('custom-outcome-wrap');
  customWrap.style.display = mode === 'custom' ? 'block' : 'none';
  // Deselect all outcome chips if switching to AI or custom
  if (mode === 'ai' || mode === 'custom') {
    document.querySelectorAll('.outcome-opt').forEach(el => {
      el.classList.remove('selected');
      el.querySelector('.outcome-opt-check').textContent = '';
    });
  }
}

function getChosenOutcomes() {
  const domain = document.getElementById('domain-select').value;
  if (!domain) return { domain: '', outcomes: [], mode: 'ai' };

  const mode = document.querySelector('input[name="outcome-mode"]:checked')?.value
|| 'ai';
  if (mode === 'custom') {
    const txt = document.getElementById('custom-outcome-text').value.trim();
    return { domain, outcomes: txt ? [txt] : [], mode: 'custom' };
  }
  // Check for manually selected chips
  const selected = [...document.querySelectorAll('#outcome-opts
.outcome-opt.selected')].map(el => el.dataset.value ||
el.querySelector('.outcome-opt-text')?.textContent || '');
  if (selected.length > 0) return { domain, outcomes: selected, mode: 'selected' };
  return { domain, outcomes: [], mode: 'ai' };
}

function populateAssessClassSelect() {
  const sel = document.getElementById('assess-class-select');

```

```

    if (!sel) return;
    const classes = DB.getClasses();
    const current = sel.value;
    sel.innerHTML = '<option value="">- All classes -</option>' +
        classes.map(c => `<option value="${esc(c)}">${esc(c)}</option>`).join('');
    ${c===current?'selected':''}>${esc(c)}</option>`).join('');
}

function populateBatchClassSelect() {
    const sel = document.getElementById('batch-class-select');
    if (!sel) return;
    const classes = DB.getClasses();
    sel.innerHTML = '<option value="">- All classes -</option>' +
        classes.map(c => `<option value="${esc(c)}">${esc(c)}</option>`).join('');
    if (rosterClassFilter !== 'all' && rosterClassFilter !== '__none__') sel.value =
rosterClassFilter;
    populateBatchStudentSelect();
}

function onAssessClassChange() {
    const cls = document.getElementById('assess-class-select').value;
    if (cls) rosterClassFilter = cls;
    populateStudentSelect();
}

function populateBatchStudentSelect() {
    const sel = document.getElementById('batch-student-select');
    if (!sel) return;
    const cls = document.getElementById('batch-class-select')?.value || '';
    let students = DB.getAllStudents();
    if (cls) students = students.filter(s => s.class === cls);
    sel.innerHTML = '<option value="">- Select existing student -</option>' +
        students.map(s => `<option
value="${esc(s.name)}">${esc(s.name)}</option>`).join('') +
        '<option value="__new__" style="font-style:italic;color:var(--ink-soft)">+ New
student...</option>';
}

function onBatchStudentSelect() {
    const sel = document.getElementById('batch-student-select');
    const input = document.getElementById('batch-student-name');
    if (sel.value === '__new__') {
        input.value = ''; input.focus(); sel.value = '';
    } else if (sel.value) {
        input.value = sel.value;
    } else {
        input.value = '';
    }
}

```

```

function onBatchClassChange() {
  const cls = document.getElementById('batch-class-select').value;
  rosterClassFilter = cls || 'all';
  populateBatchStudentSelect();
}

function populateStudentSelect() {
  const sel = document.getElementById('student-select');
  if (!sel) return;
  const cls = document.getElementById('assess-class-select')?.value || '';
  let names = DB.getStudentNames();
  if (cls) names = DB.getAllStudents().filter(s => s.class === cls).map(s =>
s.name);
  const current = sel.value;
  sel.innerHTML = '<option value="">- Select existing student -</option>' +
    names.map(n => `<option value="${esc(n)}"
${n===current?'selected':''}>${esc(n)}</option>`).join('') +
    '<option value="__new__" style="font-style:italic;color:var(--ink-soft)">+ New
student...</option>';
}

function onStudentSelect() {
  const sel = document.getElementById('student-select');
  const nameInput = document.getElementById('student-name');
  const val = sel.value;
  if (val === '__new__') {
    nameInput.value = '';
    nameInput.focus();
    sel.value = '';
  } else if (val) {
    nameInput.value = val;
    document.getElementById('ac-list').classList.remove('open');
  } else {
    nameInput.value = '';
  }
  checkReady();
}

function onNameInput() {
  // If user types manually, deselect the dropdown
  const sel = document.getElementById('student-select');
  if (sel && sel.value && sel.value !==
document.getElementById('student-name').value) {
    sel.value = '';
  }
  checkReady();
  const q = document.getElementById('student-name').value.trim().toLowerCase();
  const list = document.getElementById('ac-list');
  if (!q) { list.classList.remove('open'); return; }

```

```

    const names = DB.getStudentNames().filter(n => n.toLowerCase().startsWith(q) ||
n.toLowerCase().includes(q));
    if (!names.length) { list.classList.remove('open'); return; }
    list.innerHTML = names.slice(0, 6).map(n => {
        const st = studentForActiveCycle(DB.getStudent(n));
        const lastThumb = st?.assessments[0]?.thumb || '';
        const lastDate = st?.assessments[0]?.date ? formatDate(st.assessments[0].date)
: '';
        const count = st?.assessments.length || 0;
        return `<div class="ac-item" onclick="selectStudent('${n}')">
            ${lastThumb ? `` : `<div
class="ac-thumb"
style="display:flex;align-items:center;justify-content:center;font-size:14px">👤</
div>`}
            <div><div style="font-weight:600">${esc(n)}</div><div
class="ac-meta">${count} assessment${count!==1?'s':''}${lastDate ? ' ' . Last:
'+lastDate : ''}</div></div>
            </div>`;
    }).join('');
    list.classList.add('open');
}

```

```

function selectStudent(name) {
    document.getElementById('student-name').value = name;
    document.getElementById('ac-list').classList.remove('open');
    checkReady();
}

```

```

document.addEventListener('click', e => {
    if (!e.target.closest('.autocomplete-wrap')) {
        document.getElementById('ac-list').classList.remove('open');
    }
});

```

```

function handlePhoto(e, slotIndex) {
    const file = e.target.files[0];
    if (!file) return;
    const reader = new FileReader();
    reader.onload = ev => {
        const img = new Image();
        img.onload = () => {
            const MAX = 1600;
            let { width: w, height: h } = img;
            if (w > MAX || h > MAX) {
                if (w > h) { h = Math.round(h * MAX / w); w = MAX; }
                else { w = Math.round(w * MAX / h); h = MAX; }
            }
            const c1 = document.createElement('canvas');
            c1.width = w; c1.height = h;
            c1.getContext('2d').drawImage(img, 0, 0, w, h);

```

```

    const full = c1.toDataURL('image/jpeg', 0.82);
    currentPhotos[slotIndex] = { base64: full.split(',')[1], mimeType:
'image/jpeg' };

```

```

// Thumbnail from first photo only
if (slotIndex === 0) {
    const TH = 120, TV = 90;
    let tw = TH, th = TV;
    const ratio = w / h;
    if (ratio > TH/TV) { tw = TH; th = Math.round(TH/ratio); }
    else { th = TV; tw = Math.round(TV*ratio); }
    const c2 = document.createElement('canvas');
    c2.width = tw; c2.height = th;
    c2.getContext('2d').drawImage(img, 0, 0, tw, th);
    currentThumb = c2.toDataURL('image/jpeg', 0.4);
}

```

```

// Update slot UI
const slot = document.getElementById('slot-' + slotIndex);
slot.innerHTML = `
    
    <button class="photo-slot-remove"
onclick="removePhoto(event,${slotIndex})">X</button>`;

```

```

// Unlock next slot if not already filled
refreshSlots();
checkReady();
};
img.src = ev.target.result;
};
reader.readAsDataURL(file);
}

```

```

function removePhoto(e, slotIndex) {
    e.stopPropagation();
    currentPhotos[slotIndex] = null;
    if (slotIndex === 0) currentThumb = null;
    resetSlot(slotIndex);
    refreshSlots();
    checkReady();
}

```

```

function resetSlot(i) {
    const slot = document.getElementById('slot-' + i);
    slot.innerHTML = `
        <div class="photo-slot-empty">
            <div class="ps-icon">📷</div>
            <div class="ps-btns">
                <div class="ps-btn">📷 Camera<input type="file" accept="image/*"
capture="environment" onchange="handlePhoto(event,${i})"></div>

```

```

        <div class="ps-btn"><img alt="Library icon" data-bbox="368 91 388 104"/> Library<input type="file" accept="image/*"
onchange="handlePhoto(event,${i})"></div>
    </div>
</div>`;
}

function refreshSlots() {
    const filled = currentPhotos.filter(Boolean).length;
    // Unlock slots sequentially – next empty slot after last filled
    for (let i = 0; i < 4; i++) {
        const slot = document.getElementById('slot-' + i);
        if (!slot) continue;
        if (currentPhotos[i]) {
            slot.style.opacity = '1';
            slot.style.pointerEvents = 'auto';
        } else if (i === 0 || currentPhotos[i-1]) {
            // This is the next available slot
            slot.style.opacity = '1';
            slot.style.pointerEvents = 'auto';
        } else {
            slot.style.opacity = '0.4';
            slot.style.pointerEvents = 'none';
        }
    }
    const count = document.getElementById('photo-count');
    count.textContent = filled === 0 ? '' : filled + ' photo' + (filled !== 1 ? 's' : '') + ' added';
}

function setAssessInputType(type) {
    assessInputType = type;
    ['photo', 'pdf'].forEach(t => {
        const btn = document.getElementById('assess-itype-' + t);
        if (btn) btn.classList.toggle('active', t === type);
    });
    const photoWrap = document.getElementById('assess-photo-wrap');
    const pdfWrap = document.getElementById('assess-pdf-wrap');
    if (photoWrap) photoWrap.style.display = type === 'photo' ? 'block' : 'none';
    if (pdfWrap) pdfWrap.style.display = type === 'pdf' ? 'block' : 'none';
    checkReady();
}

function handleAssessPDF(e) {
    const file = e.target.files[0];
    if (!file) return;
    const reader = new FileReader();
    reader.onload = ev => {
        currentPDFData = { base64: ev.target.result.split(',')[1], name: file.name };
        const meta = document.getElementById('assess-pdf-meta');
        const nameEl = document.getElementById('assess-pdf-name');
    }
}

```



```

    if (nameEl) nameEl.textContent = '✓ ' + file.name;
    if (meta) meta.style.display = 'flex';
    currentThumb = null;
    checkReady();
  };
  reader.readAsDataURL(file);
}

function removeAssessPDF() {
  currentPDFData = null;
  const meta = document.getElementById('assess-pdf-meta');
  const nameEl = document.getElementById('assess-pdf-name');
  const input = document.getElementById('assess-pdf-input');
  if (meta) meta.style.display = 'none';
  if (nameEl) nameEl.textContent = '';
  if (input) input.value = '';
  checkReady();
}

function checkReady() {
  const nameReady = !!document.getElementById('student-name').value.trim();
  const workReady = assessInputType === 'pdf' ? !!currentPDFData :
currentPhotos.some(Boolean);
  document.getElementById('assess-btn').disabled = !(nameReady && workReady);
}

async function runAssessment() {
  const name = document.getElementById('student-name').value.trim();
  const usingPdf = assessInputType === 'pdf';
  const activePhotos = currentPhotos.filter(Boolean);
  if (!name || (usingPdf ? !currentPDFData : !activePhotos.length)) return;
  setState('loading');
  document.getElementById('loading-msg').textContent = usingPdf
    ? ((currentPDFData?.name || 'PDF document') + ' received - ' + (detailedComment
? 'running detailed analysis...' : 'identifying outcomes and analysing the work...'))
    : ((activePhotos.length > 1 ? activePhotos.length + ' photos' : 'Photo') + '
received - ' + (detailedComment ? 'running detailed analysis...' : 'identifying
outcomes and analysing the work...'));

  const settings = DB.loadSettings();
  const defaultCtx = settings.defaultContext || '';
  const sessionCtx = document.getElementById('session-context').value.trim();
  const ctxParts = [defaultCtx, sessionCtx].filter(Boolean);
  const ctxBlock = ctxParts.length ? `
Additional context: ${ctxParts.join(' ')}` : '';

  const { domain: chosenDomain, outcomes: chosenOutcomes, mode: outcomeMode } =
getChosenOutcomes();
  const sourceContent = usingPdf
    ? [{ type: 'document', source: { type: 'base64', media_type: 'application/pdf',

```

```

data: currentPDFData.base64 } }]]
  : activePhotos.map(p => ({ type: 'image', source: { type: 'base64', media_type:
p.mimeType, data: p.base64 } }));
  const includePdfBeta = usingPdf;

  try {
    const resolvedDomain = chosenDomain || await identifyApprovedDomain({
      messageContent: sourceContent,
      timeoutMs: detailedComment ? 60000 : 45000,
      includePdfBeta,
      activityDescription: sessionCtx || defaultCtx || ''
    });

    const domainInstruction = `The domain is fixed as: ${resolvedDomain}. Use this
exact domain. The approved in-app curriculum library is the source of truth.`;

    let outcomeInstruction;
    if (outcomeMode === 'selected' && chosenOutcomes.length > 0) {
      outcomeInstruction = `Assess the student specifically against these
pre-selected approved outcomes:
${chosenOutcomes.map(o => '- ' + o).join('\n')}
Do NOT add any other outcomes.`;
    } else if (outcomeMode === 'custom' && chosenOutcomes.length > 0) {
      outcomeInstruction = `Assess the student against this specific
teacher-entered outcome: "${chosenOutcomes[0]}". Provide the outcome in both the
"fr" and "en" fields exactly as entered. Do NOT add any other outcomes.`;
    } else {
      outcomeInstruction = `Use ONLY approved learning outcomes from the app's
curriculum library for "${resolvedDomain}". Choose 1-3 outcomes that most directly
match the student's work. Prefer 1-2 outcomes. Only choose 3 if the work clearly
and substantially demonstrates three distinct approved outcomes. Copy the wording
EXACTLY from the library. Do NOT invent, paraphrase, merge, translate or adapt
outcomes. If the fit is imperfect, choose the closest approved outcome rather than
making one up.

Approved library for ${resolvedDomain}:
${buildOutcomeLibraryPrompt(resolvedDomain)}`;
    }

    const commentInstruction = detailedComment
      ? `5. Write a comprehensive comment addressed directly to the student (their
name is "${name}"). Start with their name. Cover:
- Strengths demonstrated in this work (be specific, reference what the student
actually did)
- Areas to focus on or improve (with concrete, actionable guidance)
- A suggested next step
Write in light Australian English. Do not use em dashes. The comment should read
as a professional teacher assessment written to the student, not a checklist.`
      : `5. Only generate a comment if there is something genuinely worth flagging.
If outcomes tell the full story, leave comment as empty string.`;

```

```

const commentSchema = detailedComment
  ? '"comment": "<comprehensive comment addressed to the student by name:
strengths, areas to focus on, next step – in light Australian English, no em
dashes>",'
  : '"comment": "<specific next step in light Australian English, no em dashes
– OR empty string>",' ;

```

```

const prompt = `You are an experienced teacher at an international school
following the French curriculum for ${getAssessCycleContext()}. All comments and
English translations must be in English.

```

```

${usingPdf ? "A PDF document of the student's work is attached. It may contain one
or more scanned pages. Treat the whole PDF as one piece of work." :
(activePhotos.length > 1 ? activePhotos.length + " photographs of the student's
work are attached – treat them as a single piece of work spanning multiple pages."
: "A photograph of the student's work is attached.")}${ctxBlock}

```

Your task:

```

1. ${domainInstruction}
2. ${outcomeInstruction}
3. Return between 1 and 3 outcomes only. Prefer 1-2 unless 3 are clearly justified
by the evidence.
4. For each outcome assess whether the student has: "met" it, is "wt" (working
towards) it, or has "nm" (not met) it.
${commentInstruction}

```

Respond with ONLY a valid JSON object – no markdown, no backticks, no preamble:

```

{
  "domain": "${resolvedDomain}",
  "outcomes": [
    { "fr": "<exact approved outcome wording>", "en": "<English copy of the same
approved outcome>", "status": "<met|wt|nm>" }
  ],
  ${commentSchema}
  "illegible": <true if handwriting was too hard to read reliably, otherwise false>
} `;

```

```

const result = await callAnthropicJSON({
  messageContent: [...sourceContent, { type: 'text', text: prompt }],
  maxTokens: detailedComment ? 2000 : 1000,
  timeoutMs: detailedComment ? 90000 : 60000,
  includePdfBeta
});

```

```

let approvedOutcomes;
if (outcomeMode === 'custom' && chosenOutcomes.length > 0) {
  const customStatus = Array.isArray(result.outcomes) && result.outcomes[0] &&
['met', 'wt', 'nm'].includes(result.outcomes[0].status) ? result.outcomes[0].status :
'wt';

```

```

        approvedOutcomes = [{ fr: chosenOutcomes[0], en: chosenOutcomes[0], status:
customStatus }];
    } else {
        approvedOutcomes = enforceApprovedOutcomes(resolvedDomain, result.outcomes ||
[]);
        if (outcomeMode === 'ai') approvedOutcomes = approvedOutcomes.slice(0, 3);
        if (outcomeMode === 'selected' && chosenOutcomes.length > 0 &&
!approvedOutcomes.length) {
            approvedOutcomes = chosenOutcomes.map((outcome, index) => ({
                fr: outcome,
                en: outcome,
                status: Array.isArray(result.outcomes) && result.outcomes[index] &&
['met', 'wt', 'nm'].includes(result.outcomes[index].status) ?
result.outcomes[index].status : 'wt'
            }));
        }
    }

    if (!approvedOutcomes.length) {
        throw new Error('AI did not return outcomes from the approved curriculum
library.');
```

```

    }

    currentResult = {
        domain: resolvedDomain,
        outcomes: approvedOutcomes,
        comment: result.comment || ''
    };

    document.getElementById('result-name').textContent = name;
    document.getElementById('warn-illegible').style.display = result.illegible ?
'block' : 'none';
    renderResultUI(currentResult);
    setState('result');
} catch(err) {
    setState('input');
    flash('Assessment failed: ' + err.message);
}

function overrideDomain(val) {
    if (!val || !currentResult) return;
    currentResult.domain = val;
    document.getElementById('result-domain').textContent = '📄 ' + val;
    document.getElementById('domain-override').value = '';
}

function renderResultUI(r) {
    document.getElementById('result-domain').textContent = '📄 ' + r.domain;
    const list = document.getElementById('outcome-list');
```

```

list.innerHTML = '';
r.outcomes.forEach((o, i) => {
  const row = document.createElement('div');
  row.className = 'outcome-row ' + o.status;
  row.innerHTML = `
    <div class="status-dot"></div>
    <div class="outcome-text">
      <select class="status-select" onchange="updateStatus(${i}, this.value)">
        <option value="met" ${o.status==='met'? 'selected':''}>Met</option>
        <option value="wt" ${o.status==='wt'? 'selected':''}>Working
Towards</option>
        <option value="nm" ${o.status==='nm'? 'selected':''}>Not Met</option>
      </select>
      <div class="outcome-fr">${esc(getOutcomeDisplayPrimary(o))}</div>
      ${getOutcomeDisplaySecondary(o) ? `<div
class="outcome-en">${esc(getOutcomeDisplaySecondary(o))}</div>` : ''}
    </div>`;
  list.appendChild(row);
});
const commentEl = document.getElementById('comment-input');
commentEl.value = r.comment || '';
// Auto-expand for detailed comments
if (detailedComment && r.comment) {
  commentEl.style.minHeight = '180px';
} else {
  commentEl.style.minHeight = '';
}
commentEl.style.height = 'auto';
commentEl.style.height = commentEl.scrollHeight + 'px';
}

function updateStatus(i, val) {
  if (!currentResult) return;
  currentResult.outcomes[i].status = val;
  const rows = document.querySelectorAll('.outcome-row');
  if (rows[i]) rows[i].className = 'outcome-row ' + val;
}

function reassess() { setState('loading'); setTimeout(runAssessment, 80); }

function confirmResult() {
  const name = document.getElementById('student-name').value.trim();
  const comment = document.getElementById('comment-input').value.trim();
  currentResult.comment = comment;
  const cycleSnapshot = getAssessCycleSnapshot();

  const assessment = {
    id: Date.now().toString(36) + Math.random().toString(36).slice(2,5),
    date: new Date().toISOString().split('T')[0],
    ...cycleSnapshot,
  }

```

```

    domain: currentResult.domain,
    outcomes: currentResult.outcomes,
    comment,
    thumb: assessInputType === 'pdf' ? '' : (currentThumb || '')
  };

  // Assign class if one is active in filter
  const data2 = DB.loadData();
  if (data2.students[name] && rosterClassFilter !== 'all' && rosterClassFilter !==
  '___none___' && !data2.students[name].class) {
    data2.students[name].class = rosterClassFilter;
    DB.saveData(data2);
  }
  DB.saveAssessment(name, assessment);
  sessionStudents.push({ name, ...assessment });
  populateStudentSelect();

  const n = sessionStudents.length;
  document.getElementById('progress-step').textContent = `Session: ${n} done`;
  document.getElementById('progress-count').textContent = '';
  document.getElementById('progress-fill').style.width = Math.min(n*5, 95) + '%';
  document.getElementById('finish-btn').style.display = 'inline-flex';

  resetAssessForm();
  setState('input');
}

function resetAssessForm() {
  document.getElementById('student-name').value = '';
  const sel = document.getElementById('student-select'); if (sel) sel.value = '';
  currentPhotos = [];
  for (let i = 0; i < 4; i++) resetSlot(i);
  refreshSlots();
  document.getElementById('photo-count').textContent = '';
  removeAssessPDF();
  setAssessInputType('photo');
  document.getElementById('session-context').value = '';
  document.getElementById('context-card').style.display = 'none';
  document.getElementById('domain-select').value = '';
  document.getElementById('outcome-picker').classList.remove('visible');
  document.getElementById('assess-btn').disabled = true;
  currentPhotos = []; currentThumb = null; currentPDFData = null; currentResult =
  null;
}

function setState(s) {
  ['input', 'loading', 'result', 'summary'].forEach(id =>
    document.getElementById('state-'+id).style.display = id===s ? 'block' : 'none'
  );
}

```

```

function finishSession() {
  if (!sessionStudents.length) { flash('No students assessed this session.');
```

```

return; }
  // Persist session to history
  DB.saveSession({
    id: Date.now().toString(36),
    date: new Date().toISOString().split('T')[0],
    ...getAssessCycleSnapshot(),
    students: sessionStudents.map(s => ({ name: s.name, domain: s.domain,
assessmentId: s.id, cycle: s.cycle, yearLevel: s.yearLevel })))
  });
  document.getElementById('summary-sub').textContent = sessionStudents.length + '
student' + (sessionStudents.length !== 1 ? 's': '') + ' assessed this session';
  const cards = document.getElementById('summary-cards');
  cards.innerHTML = sessionStudents.map(st => {
    const outs = st.outcomes.map(o => `
      <div class="hist-outcome">
        <div class="hist-dot"
style="background:${o.status==='met'? 'var(--met)': o.status==='wt'? 'var(--wt)': 'var(
--nm)'}; margin-top: 4px"></div>
        <div>
          <div class="hist-status"
style="color:${o.status==='met'? 'var(--met)': o.status==='wt'? 'var(--wt)': 'var(--nm)
'}">${STATUS_LABEL[o.status]}</div>
          <div class="hist-outcome-fr">${esc(getOutcomeDisplayPrimary(o))}</div>
        </div>
      </div>`
    ).join('');
    const cmt = st.comment ? `<div class="hist-comment">
${esc(st.comment)}</div>` : '';
    return `<div class="sum-student">
      <div class="sum-name"><span>${esc(st.name)}</span><span
style="font-size: 11px; color: var(--accent); font-weight: 500">${esc(st.domain)}</span>
</div>
      ${outs}${cmt}
    </div>`;
  }).join('');
  setState('summary');
}

function newSession() {
  sessionStudents = [];
  document.getElementById('progress-step').textContent = 'Ready to assess';
  document.getElementById('progress-fill').style.width = '0%';
  document.getElementById('finish-btn').style.display = 'none';
  setState('input');
}

function printSessionCards() {
  const today = new Date().toLocaleDateString('en-GB',

```

```

{day:'numeric',month:'long',year:'numeric'});
const cards = sessionStudents.map(st => {
  const outs = st.outcomes.map(o => `
    <div class="pc-outcome">
      <div class="pc-dot ${o.status}"></div>
      <div class="pc-status ${o.status}">${STATUS_LABEL[o.status]}</div>
    </div><div
class="pc-fr">${esc(getOutcomeDisplayPrimary(o))}</div>${getOutcomeDisplaySecondary
(o) ? `<div class="pc-en">${esc(getOutcomeDisplaySecondary(o))}</div>` : ''}</div>
    </div>`).join('');
  const cmt = st.comment ? `<div class="pc-comment"><strong>Next step:</strong>
${esc(st.comment)}</div>` : '';
  return `<div class="pc"><div class="pc-name">${esc(st.name)}</div><div
class="pc-domain">${esc(st.domain)}</div>${outs}${cmt}</div>`;
}).join('');
document.getElementById('print-area').innerHTML = `<div
class="print-head"><h1>Session – ${today}</h1><p>${sessionStudents.length} students
assessed</p></div>${cards}`;
window.print();
}

```

```

/* =====
STUDENTS TAB
===== */

```

```

async function deleteStudent(name) {
  if (!await appConfirm('Delete student?', `Remove ${name} and all their assessment
records. This cannot be undone.`, 'Delete', 'btn-ghost')) return;
  const data = DB.loadData();
  delete data.students[name];
  DB.saveData(data);
  renderRoster();
}

```

```

async function deleteSessionRecord(sessionId) {
  const sessions = DB.getSessions();
  const s = sessions.find(x => x.id === sessionId);
  if (!s) return;
  const label = s.activity ? `${s.activity}` : `session from
${formatDate(s.date)}`;
  const count = s.students.length;
  if (!await appConfirm('Delete session?', `This will remove ${count} assessment
record${count!==1?'s':''} from student profiles. Cannot be undone.`, 'Delete all',
'btn-ghost')) return;
  DB.deleteSession(sessionId);
  renderHistory();
  renderRoster();
}

```



```

function renderHistory() {
  const list = document.getElementById('history-list');
  const sessions = DB.getSessions().map(sessionForActiveCycle).filter(Boolean);

  // Also reconstruct sessions from assessment data (catches records saved before
  session logging)
  const allStudents = DB.getAllStudents();
  const reconstructed = {}; // key: date+activity -> {date, activity, type,
  students:[]}
  allStudents.forEach(st => {
    filterAssessmentsForActiveCycle(st.assessments || []).forEach(a => {
      // Check if this assessment already has a session log
      const hasLog = sessions.some(s => s.students.some(e => e.assessmentId ===
a.id));
      if (hasLog) return;
      const key = a.date + '|' + (a.activity || a.domain || '');
      if (!reconstructed[key]) reconstructed[key] = { date: a.date, activity:
a.activity || '', type: a.type || 'assess', students: [], _reconstructed: true,
cycle: a.cycle, yearLevel: a.yearLevel };
      reconstructed[key].students.push({ name: st.name, domain: a.domain,
assessmentId: a.id, cycle: a.cycle, yearLevel: a.yearLevel });
    });
  });

  // Merge logged sessions + reconstructed, sort by date desc
  const recoList = Object.values(reconstructed);
  const allSessions = [...sessions, ...recoList].sort((a,b) =>
b.date.localeCompare(a.date));

  if (!allSessions.length) {
    list.innerHTML = `<div class="empty-state"><div
class="empty-icon">📅</div><p>No sessions yet.</p></div>`;
    return;
  }

  list.innerHTML = allSessions.map(s => {
    const rows = s.students.map(st => `
      <div class="sh-row" onclick="openProfile('${esc(st.name)}')">
        <div class="sh-name">${esc(st.name)}</div>
        <div class="sh-domain">${esc(st.domain)}</div>
      </div>`.join('');
    const activityLabel = s.activity
    ? `<div
style="font-size:11.5px;color:var(--ink-soft);font-style:italic;margin-top:1px">${e
sc(s.activity)}</div>`
    : '';
    const typeTag = s.type === 'batch'
    ? `<span
style="font-size:9px;font-weight:700;text-transform:uppercase;letter-spacing:0.07em
;background:var(--accent-lt);color:var(--accent);border:1px solid

```

```

var(--accent-bd);padding:1px 6px;border-radius:4px;margin-left:6px">Batch</span>`
: '';
const recoTag = s._reconstructed
? `<span
style="font-size:9px;color:var(--ink-soft);margin-left:6px">reconstructed</span>`
: '';
const deleteBtn = s._reconstructed
? `<button class="btn-icon"
data-ids='${JSON.stringify(s.students.map(x=>x.assessmentId))}'
data-date='${esc(s.date)}' data-activity='${esc(s.activity||'')}'
onclick="deleteReconstructedSession(this)" title="Delete all records in this
session" style="font-size:15px;color:var(--ink-soft)">🗑️</button>`
: `<button class="btn-icon" onclick="deleteSessionRecord('${esc(s.id)}')"
title="Delete this session and all its records"
style="font-size:15px;color:var(--ink-soft)">🗑️</button>`;
const reopenBtn = s.type === 'batch' && !s._reconstructed
? `<button class="btn btn-secondary" onclick="event.stopPropagation();
reopenBatchSession('${esc(s.id)}', { sourceTab: 'students' }); return false;"
style="padding:6px 10px;font-size:11.5px">↺ Open grid</button>`
: '';
return `<div class="session-hist-card">
<div class="sh-header">
<div>
<div class="sh-date">${formatDate(s.date)}${typeTag}${recoTag}</div>
${activityLabel}
</div>
<div>
style="display:flex;align-items:center;gap:8px;flex-wrap:wrap;justify-content:flex-
end">
${reopenBtn}
<div class="sh-count">${s.students.length}
student${s.students.length!==1?'s':''}</div>
${deleteBtn}
</div>
</div>
<div class="sh-rows">${rows}</div>
</div>`;
}).join('');
}

```

```

async function deleteReconstructedSession(btn) {
const assessmentIds = JSON.parse(btn.dataset.ids);
const date = btn.dataset.date;
const activity = btn.dataset.activity;
const count = assessmentIds.length;
const label = activity || `session from ${formatDate(date)}`;
if (!await appConfirm(`Delete session?`, `This will remove ${count} assessment
record${count!==1?'s':''} from student profiles. Cannot be undone.`, 'Delete all',
'btn-ghost')) return;
const data = DB.loadData();

```

```

    Object.values(data.students).forEach(st => {
      st.assessments = st.assessments.filter(a => !assessmentIds.includes(a.id));
    });
    DB.saveData(data);
    renderHistory();
    renderRoster();
  }

```

```

/* == BULK ASSIGN & CLASS MANAGEMENT == */

```

```

function openAddStudent() {
  // Populate class dropdown
  const sel = document.getElementById('add-student-class');
  const classes = DB.getClasses();
  sel.innerHTML = '<option value="">— No class —</option>' +
    classes.map(c => `<option value="${esc(c)}">${esc(c)}</option>`).join('');
  // Pre-select active filter
  if (rosterClassFilter !== 'all' && rosterClassFilter !== '__none__') sel.value =
rosterClassFilter;
  document.getElementById('add-student-name').value = '';
  document.getElementById('add-student-notes').value = '';
  document.getElementById('add-student-modal').classList.add('open');
  setTimeout(() => document.getElementById('add-student-name').focus(), 100);
}

```

```

function closeAddStudent() {
  document.getElementById('add-student-modal').classList.remove('open');
}

```

```

function saveNewStudent() {
  const name = document.getElementById('add-student-name').value.trim();
  const cls = document.getElementById('add-student-class').value;
  const notes = document.getElementById('add-student-notes').value.trim();
  if (!name) { flash('Please enter a name.');
```

 return; }
 if (DB.getStudent(name)) { flash(`\${name} is already in your roster.`, "warn");
return; }
 const data = DB.loadData();
 data.students[name] = { name, class: cls, notes, assessments: [] };
 DB.saveData(data);
 closeAddStudent();
 renderRoster();
 populateStudentSelect();
 populateBatchStudentSelect();
}

```

function openBulkAssign() {
  showScreen('bulk-assign');
  renderBulkAssign();
}

```

```

function renderBulkAssign() {
  const classes = DB.getClasses();
  const students = DB.getAllStudents();

  // Class list with delete
  document.getElementById('class-list-manage').innerHTML = classes.length
    ? classes.map(c => `
      <div
style="display:inline-flex;align-items:center;gap:5px;background:#f0eaff;border:1px
solid #d0c0f0;border-radius:20px;padding:4px 10px">
        <span
style="font-size:12px;font-weight:600;color:#6040c0">${esc(c)}</span>
        <button onclick="deleteClassAndRefresh('${esc(c)}')
style="background:none;border:none;cursor:pointer;font-size:12px;color:#9060d0;padd
ing:0;line-height:1">X</button>
      </div>` ).join('')
    : '<span style="font-size:12.5px;color:var(--ink-soft)">No classes yet – add
one above.</span>';

  // Student rows
  document.getElementById('bulk-assign-rows').innerHTML = students.length
    ? students.map(st => `
      <div class="bulk-row">
        <div class="bulk-name">${esc(st.name)}</div>
        <select class="bulk-select" id="bulk-${esc(st.name)}"
data-student="${esc(st.name)}">
          <option value="">No class</option>
          ${classes.map(c => `<option value="${esc(c)}"
${(st.class||'')===c?'selected':''}>${esc(c)}</option>` ).join('')}
        </select>
      </div>` ).join('')
    : '<p
style="font-size:13px;color:var(--ink-soft);text-align:center;padding:16px">No
students yet.</p>';
}

function addNewClass() {
  const input = document.getElementById('new-class-input');
  const name = input.value.trim();
  if (!name) return;
  DB.saveClass(name);
  input.value = '';
  renderBulkAssign();
  renderRoster();
}

async function deleteClassAndRefresh(name) {
  if (!await appConfirm(`Delete class?`, `Students in this class will become
unassigned. Assessment data is kept.`, 'Delete', 'btn-ghost')) return;

```

```

    DB.deleteClass(name);
    renderBulkAssign();
    renderRoster();
  }

  async function saveBulkAssign() {
    document.querySelectorAll('.bulk-select').forEach(sel => {
      const name = sel.dataset.student;
      DB.updateStudentClass(name, sel.value);
    });
    renderRoster();
    goTab('students');
  }

  function renderRoster() {
    const q = (document.getElementById('roster-search')?.value ||
    '').trim().toLowerCase();
    const allStudents = DB.getAllStudents();

    // Build class tabs
    const classes = DB.getClasses();
    const tabsEl = document.getElementById('class-tabs');
    if (tabsEl) {
      const hasUnassigned = allStudents.some(s => !s.class);
      tabsEl.innerHTML =
        `<button class="class-tab ${rosterClassFilter==='all'? 'active':''}"
        onclick="setClassFilter('all')">All (${allStudents.length})</button>` +
        classes.map(c => {
          const count = allStudents.filter(s => s.class === c).length;
          return `<button class="class-tab ${rosterClassFilter===c?'active':''}"
          onclick="setClassFilter('${esc(c)}')">${esc(c)} (${count})</button>`;
        }).join('') +
        (hasUnassigned ? `<button class="class-tab
        ${rosterClassFilter==='__none__'? 'active':''}"
        onclick="setClassFilter('__none__')">Unassigned</button>` : '');
    }

    // Filter
    let students = allStudents;
    if (rosterClassFilter !== 'all') {
      students = rosterClassFilter === '__none__'
        ? allStudents.filter(s => !s.class)
        : allStudents.filter(s => s.class === rosterClassFilter);
    }
    if (q) students = students.filter(s => s.name.toLowerCase().includes(q));

    const list = document.getElementById('roster-list');
    if (!students.length) {
      list.innerHTML = `<div class="empty-state"><div
      class="empty-icon">👤</div><p>${allStudents.length ? 'No students in this class.'

```

```

: 'No students yet.<br>Assess some work to build your roster.'</p></div>`;
    return;
}

list.innerHTML = students.map(st => {
    const activeStudent = studentForActiveCycle(st);
    const activeAssessments = activeStudent.assessments || [];
    const last = activeAssessments[0];
    const domains = [...new Set(activeAssessments.map(a => a.domain))];
    const thumb = last?.thumb || '';
    const initial = st.name.trim()[0]?.toUpperCase() || '?';
    const thumbEl = thumb
        ? ``
        : `<div class="student-thumb-placeholder">${initial}</div>`;
    const domainTags = domains.slice(0,4).map(d => `<span
class="domain-tag">${esc(d)}</span>`).join('');
    const classBadge = st.class ? `<span
class="class-badge">${esc(st.class)}</span>` : '';
    return `<div class="student-card" onclick="openProfile('${esc(st.name)}')">
        ${thumbEl}
        <div class="student-info">
            <div class="student-name-row">${esc(st.name)} ${classBadge}</div>
            <div class="student-meta">${activeAssessments.length}
assessment${activeAssessments.length!==1?'s':''}${last?.date ? ' · Last: ' +
formatDate(last.date) : ''}</div>
            <div class="domain-tags">${domainTags}</div>
        </div>
        <div style="display:flex;align-items:center;gap:4px">
            <button class="btn-icon"
onclick="event.stopPropagation();deleteStudent('${esc(st.name)}')" title="Delete
student" style="font-size:14px;color:var(--ink-soft)">🗑️</button>
            <div class="student-arrow">></div>
        </div>
    </div>`;
}).join('');
}

function setClassFilter(cls) {
    rosterClassFilter = cls;
    renderRoster();
}

/* =====
    PROFILE SCREEN
===== */

function openProfile(name) {
    currentProfileStudent = name;
    profileMasteryDomainFilter = 'all';
    profileHistoryOutcomeFilter = '';
    profileHistoryDomainFilter = '';

```

```

profileMasteryPanelOpen = true;
profileHistoryPanelOpen = false;
profileHistoryShowAll = false;
profileCommentPanelOpen = false;
profileMapPanelOpen = false;
showScreen('profile');
document.getElementById('nav-students').classList.add('active');
document.getElementById('nav-assess').classList.remove('active');
document.getElementById('nav-heatmap').classList.remove('active');
renderProfile(name);
}

function getAssessmentPerformanceScore(assessment) {
  const values = { met: 2, wt: 1, nm: 0 };
  const outcomes = Array.isArray(assessment?.outcomes) ? assessment.outcomes : [];
  if (!outcomes.length) return -1;
  const scored = outcomes
    .map(o => values[o?.status])
    .filter(v => typeof v === 'number');
  if (!scored.length) return -1;
  return scored.reduce((sum, value) => sum + value, 0) / scored.length;
}

function setProfileSortMode(mode) {
  profileSortMode = mode === 'performance' ? 'performance' : 'chronological';
  if (currentProfileStudent) renderProfile(currentProfileStudent);
}

function setProfileMasteryDomainFilter(domain) {
  profileMasteryDomainFilter = domain || 'all';
  if (currentProfileStudent) renderProfile(currentProfileStudent);
}

function toggleProfileMasteryPanel(forceOpen = null) {
  profileMasteryPanelOpen = forceOpen === null ? !profileMasteryPanelOpen :
  !!forceOpen;
  if (currentProfileStudent) renderProfile(currentProfileStudent);
}

function toggleProfileHistoryPanel(forceOpen = null) {
  profileHistoryPanelOpen = forceOpen === null ? !profileHistoryPanelOpen :
  !!forceOpen;
  if (currentProfileStudent) renderProfile(currentProfileStudent);
  if (profileHistoryPanelOpen) {
    requestAnimationFrame(() => {
      const target = document.getElementById('profile-history-panel');
      if (target) target.scrollIntoView({ behavior: 'smooth', block: 'start' });
    });
  }
}

```

```

function setProfileHistoryShowAll(showAll = true) {
  profileHistoryShowAll = !!showAll;
  profileHistoryPanelOpen = true;
  if (currentProfileStudent) renderProfile(currentProfileStudent);
  requestAnimationFrame(() => {
    const target = document.getElementById('profile-history-panel');
    if (target) target.scrollIntoView({ behavior: 'smooth', block: 'start' });
  });
}

function toggleProfileCommentPanel(forceOpen = null) {
  profileCommentPanelOpen = forceOpen === null ? !profileCommentPanelOpen :
!!forceOpen;
  if (currentProfileStudent) renderProfile(currentProfileStudent);
  if (profileCommentPanelOpen) {
    requestAnimationFrame(() => {
      const target = document.getElementById('profile-comment-panel');
      if (target) target.scrollIntoView({ behavior: 'smooth', block: 'start' });
    });
  }
}

function toggleProfileMapPanel(forceOpen = null) {
  profileMapPanelOpen = forceOpen === null ? !profileMapPanelOpen : !!forceOpen;
  if (currentProfileStudent) renderProfile(currentProfileStudent);
  if (profileMapPanelOpen) {
    requestAnimationFrame(() => {
      const target = document.getElementById('profile-map-panel');
      if (target) target.scrollIntoView({ behavior: 'smooth', block: 'start' });
    });
  }
}

function setProfileHistoryOutcomeFilter(outcome = '', shouldScroll = false, domain
= '') {
  profileHistoryOutcomeFilter = outcome || '';
  profileHistoryDomainFilter = profileHistoryOutcomeFilter ? (domain || '') : '';
  if (profileHistoryOutcomeFilter) {
    profileHistoryPanelOpen = true;
    profileHistoryShowAll = true;
  }
  if (currentProfileStudent) renderProfile(currentProfileStudent);
  if (shouldScroll) {
    requestAnimationFrame(() => {
      const target = document.getElementById('profile-history-panel');
      if (target) target.scrollIntoView({ behavior: 'smooth', block: 'start' });
    });
  }
}

```



```

function setProfileHistoryOutcomeFilterFromNode(node, shouldScroll = false) {
  if (!node) return;
  setProfileHistoryOutcomeFilter(node.dataset.outcome || '', shouldScroll,
node.dataset.domain || '');
}

function formatMiniDateLabel(dateStr) {
  if (!dateStr) return 'Undated';
  const d = new Date(dateStr + 'T00:00:00');
  if (Number.isNaN(d.getTime())) return dateStr;
  return `${d.getDate()}
$[['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']][d.getMo
nth()]]`;
}

function buildStudentProfileDataset(st) {
  const assessments = filterAssessmentsForActiveCycle(st.assessments ||
[]).slice().sort((a,b) => (a.date || '').localeCompare(b.date || '') || (a.id ||
'').localeCompare(b.id || ''));
  const overall = { met: 0, wt: 0, nm: 0, total: 0 };
  const domainMap = {};
  const outcomeMap = {};
  const matrixMap = {};
  const trendRows = [];

  assessments.forEach((a, index) => {
    const trendRow = {
      key: `${a.date || 'undated'}-${index}`,
      label: formatMiniDateLabel(a.date),
      met: 0, wt: 0, nm: 0, total: 0
    };
    (a.outcomes || []).forEach(o => {
      if (!o || !o.fr) return;
      const status = o.status || 'none';
      if (status === 'met' || status === 'wt' || status === 'nm') {
        overall[status] += 1;
        overall.total += 1;
        trendRow[status] += 1;
        trendRow.total += 1;
      }
      const domain = a.domain || 'Unknown';
      if (!domainMap[domain]) domainMap[domain] = { domain, met: 0, wt: 0, nm: 0,
total: 0, outcomeSet: new Set() };
      if (status === 'met' || status === 'wt' || status === 'nm') {
        domainMap[domain][status] += 1;
        domainMap[domain].total += 1;
      }
      domainMap[domain].outcomeSet.add(o.fr);
    });
  });
}

```

```

    if (!outcomeMap[o.fr]) {
      outcomeMap[o.fr] = {
        key: o.fr,
        fr: o.fr,
        en: o.en || '',
        domain,
        met: 0,
        wt: 0,
        nm: 0,
        total: 0,
        lastSeen: a.date || ''
      };
    }
    if (status === 'met' || status === 'wt' || status === 'nm') {
      outcomeMap[o.fr][status] += 1;
      outcomeMap[o.fr].total += 1;
    }
    if ((a.date || '') >= (outcomeMap[o.fr].lastSeen || ''))
      outcomeMap[o.fr].lastSeen = a.date || outcomeMap[o.fr].lastSeen;

    if (!matrixMap[o.fr]) matrixMap[o.fr] = {};
    matrixMap[o.fr][a.id] = status;
  });
  if (trendRow.total > 0) trendRows.push(trendRow);
});

const domainRows = Object.values(domainMap).map(row => ({
  ...row,
  outcomeCount: row.outcomeSet.size,
  supportCount: row.wt + row.nm,
  metRate: row.total ? row.met / row.total : 0
})).sort((a,b) => (b.supportCount - a.supportCount) || (a.domain ||
'').localeCompare(b.domain || ''));

const outcomeRows = Object.values(outcomeMap).map(row => ({
  ...row,
  supportCount: row.wt + row.nm,
  supportRate: row.total ? ((row.wt + row.nm) / row.total) : 0,
  metRate: row.total ? (row.met / row.total) : 0,
  wtRate: row.total ? (row.wt / row.total) : 0,
  nmRate: row.total ? (row.nm / row.total) : 0,
  metPct: row.total ? Math.round((row.met / row.total) * 100) : 0,
  wtPct: row.total ? Math.round((row.wt / row.total) * 100) : 0,
  nmPct: row.total ? Math.round((row.nm / row.total) * 100) : 0
})).sort((a,b) => (b.total - a.total) || (b.supportCount - a.supportCount) ||
(a.metRate - b.metRate) || a.fr.localeCompare(b.fr));

const recentAssessments = assessments.slice(-8).map((a, idx) => ({
  id: a.id,

```

```

    date: a.date || '',
    label: formatMiniDateLabel(a.date),
    sub: (a.activity || a.domain || `Assessment ${idx+1}`).toString().slice(0, 12)
  ));

  const matrixRows = outcomeRows.filter(row => recentAssessments.some(col =>
matrixMap[row.key] && matrixMap[row.key][col.id])).slice(0, 14);

  const metRate = overall.total ? Math.round((overall.met / overall.total) * 100) :
0;
  const supportFlags = overall.wt + overall.nm;

  return {
    overall,
    domainRows,
    outcomeRows,
    trendRows: trendRows.slice(-8),
    recentAssessments,
    matrixRows,
    matrixMap,
    metRate,
    supportFlags
  };
}

function renderStudentProfileDashboard(st) {
  const dataset = buildStudentProfileDataset(st);
  const bodyHtml = (() => {
    if (!dataset.overall.total) {
      return `
```

```

    || a.fr.localeCompare(b.fr);
  });

  const outcomeMasteryHtml = displayOutcomeRows.length ? `
    <div class="sv-outcome-list">
      ${displayOutcomeRows.map(row => `
        <div class="sv-outcome-row is-clickable ${profileHistoryOutcomeFilter ===
row.fr && (!profileHistoryDomainFilter || profileHistoryDomainFilter ===
row.domain) ? 'active' : ''}" data-outcome="${escAttr(row.fr)}"
data-domain="${escAttr(row.domain)}"
onclick="setProfileHistoryOutcomeFilterFromNode(this,true)" title="Show matching
assessment history">
          <div class="sv-outcome-row-head">
            <div class="sv-outcome-row-main">
              <div class="sv-outcome-domain-chip">${esc(row.domain)}</div>
              <div
class="sv-outcome-fr">${esc(getOutcomeDisplayPrimary(row))}</div>
              ${getOutcomeDisplaySecondary(row) ? `<div
class="sv-outcome-en">${esc(getOutcomeDisplaySecondary(row))}</div>` : ''}
              </div>
              <div class="sv-outcome-score">
                <div class="sv-outcome-score-value">${row.metPct}%</div>
                <div class="sv-outcome-score-label">Met rate</div>
                <div class="sv-outcome-score-sub">${row.met} met out of
${row.total}</div>
              </div>
            </div>
            <div class="sv-outcome-track">
              ${row.metRate > 0 ? `<div class="sv-domain-seg met"
style="width:${row.metRate * 100}%"></div>` : ''}
              ${row.wtRate > 0 ? `<div class="sv-domain-seg wt"
style="width:${row.wtRate * 100}%"></div>` : ''}
              ${row.nmRate > 0 ? `<div class="sv-domain-seg nm"
style="width:${row.nmRate * 100}%"></div>` : ''}
            </div>
            <div class="sv-outcome-counts">
              <div class="sv-outcome-count-badges">
                <span class="sv-outcome-badge met">Met ${row.met}</span>
                <span class="sv-outcome-badge wt">W/T ${row.wt}</span>
                <span class="sv-outcome-badge nm">N/M ${row.nm}</span>
              </div>
              <div class="sv-outcome-last">Seen ${row.total} time${row.total !== 1
? 's' : ''}${row.lastSeen ? ` · Last ${esc(formatDate(row.lastSeen))}` : ''}</div>
            </div>
          </div>`
        ).join('')}
      </div>` : `<div class="sv-empty">No outcomes recorded yet for
${esc(profileMasteryDomainFilter)}</div>`;

  return `
    <div

```

```

style="display:flex;align-items:flex-start;justify-content:space-between;gap:10px;flex-wrap:wrap;margin-bottom:10px">
  <div
style="display:flex;flex-direction:column;gap:8px;min-width:240px;flex:1">
    <div class="profile-collapsible-sub" style="margin-top:0">Summary view.
Filter by domain, then sort that slice by chronology or performance. Click a row to
jump straight to the matching assessment history.</div>
    <div style="display:flex;gap:8px;flex-wrap:wrap;align-items:center">
      <select id="profile-mastery-domain-filter"
onchange="setProfileMasteryDomainFilter(this.value)"
style="font-size:12.5px;padding:7px
10px;min-width:220px;max-width:320px;font-family:'Sora',sans-serif;border:1.5px
solid
var(--border);border-radius:8px;background:var(--bg);color:var(--ink);outline:none"
>
        <option value="all" ${profileMasteryDomainFilter === 'all' ?
'selected' : ''}>All domains</option>
        ${masteryDomains.map(domain => `<option value="${escAttr(domain)}"
${profileMasteryDomainFilter === domain ? 'selected' :
''}>${esc(domain)}</option>`).join('')}
      </select>
      <div class="form-hint"
style="margin-top:0">${displayOutcomeRows.length}
outcome${displayOutcomeRows.length !== 1 ? 's' : ''} shown</div>
    </div>
  </div>
  <div style="display:flex;gap:6px;flex-wrap:wrap">
    <button class="hm-view-btn ${profileSortMode === 'chronological' ?
'active' : ''}" onclick="setProfileSortMode('chronological')"
type="button">Chronological</button>
    <button class="hm-view-btn ${profileSortMode === 'performance' ? 'active'
: ''}" onclick="setProfileSortMode('performance')"
type="button">Performance</button>
  </div>
  </div>
  ${outcomeMasteryHtml}`;
})();

return `
  <div class="card profile-collapsible ${profileMasteryPanelOpen ? 'open' : ''}"
id="profile-mastery-panel">
    <button class="profile-collapsible-toggle"
onclick="toggleProfileMasteryPanel()" type="button">
      <div>
        <div class="profile-collapsible-title">Outcome mastery</div>
        <div class="profile-collapsible-sub">Keep this open for the overview.
Collapse it when you just want the audit trail.</div>
      </div>
      <div class="profile-collapsible-arrow">v</div>
    </button>

```

```

    <div class="profile-collapsible-body">
      ${bodyHtml}
    </div>
  </div>`;
}

function renderStudentRecentOutcomeMap(st) {
  const dataset = buildStudentProfileDataset(st);
  const matrixHtml = (dataset.matrixRows.length &&
dataset.recentAssessments.length) ? `
    <div class="sv-matrix-wrap">
      <table class="sv-matrix">
        <thead>
          <tr>
            <th class="sv-outcome-col">Outcome</th>
            ${dataset.recentAssessments.map(col => `
              <th class="sv-assess-col">
                <div class="sv-assess-top">${esc(col.label)}</div>
                <div class="sv-assess-sub">${esc(col.sub)}</div>
              </th>`).join('')}
          </tr>
        </thead>
        <tbody>
          ${dataset.matrixRows.map(row => `
            <tr>
              <td class="sv-outcome-col">
                <div
class="sv-outcome-fr">${esc(getOutcomeDisplayPrimary(row))}</div>
                ${getOutcomeDisplaySecondary(row) ? `<div
class="sv-outcome-en">${esc(getOutcomeDisplaySecondary(row))}</div>` : ''}
                <div class="sv-outcome-meta">${esc(row.domain)} · ${row.met} met ·
${row.wt} wt · ${row.nm} nm</div>
              </td>
              ${dataset.recentAssessments.map(col => {
dataset.matrixMap[row.key][col.id]) || 'none';
                const label = status === 'met' ? 'Met' : status === 'wt' ? 'W/T' :
status === 'nm' ? 'N/M' : '-';
                return `
                  <td class="sv-cell ${status}">
                    <div class="sv-cell-dot"></div>
                    <div class="sv-cell-label">${label}</div>
                  </td>`;
              }).join('')}
            </tr>`).join('')}
          </tbody>
        </table>
      </div>` : `<div class="sv-empty">Not enough repeated outcome data yet to build
an individual map.</div>`;

```

```

    return `
      <div class="card profile-collapsible ${profileMapPanelOpen ? 'open' : ''}"
id="profile-map-panel">
        <button class="profile-collapsible-toggle" onclick="toggleProfileMapPanel()"
type="button">
          <div>
            <div class="profile-collapsible-title">Recent outcome map</div>
            <div class="profile-collapsible-sub">Collapsed by default so it stops
eating half the profile. Open it when you need the deeper pattern check.</div>
          </div>
          <div class="profile-collapsible-arrow">v</div>
        </button>
        <div class="profile-collapsible-body">
          ${matrixHtml}
        </div>
      </div>`;
}

```

```

function switchProfileDomainComment(name) {
  const st = DB.getStudent(name);
  const sel = document.getElementById('profile-domain-comment-select');
  const ta = document.getElementById('profile-domain-comment-text');
  if (!st || !sel || !ta) return;
  const domain = sel.value;
  ta.value = ((st.domainComments || {})[domain] || '').trim();
  ta.style.height = 'auto';
  ta.style.height = ta.scrollHeight + 'px';
}

```

```

function saveProfileDomainComment(name) {
  const sel = document.getElementById('profile-domain-comment-select');
  const ta = document.getElementById('profile-domain-comment-text');
  if (!sel || !ta || !sel.value) return;
  DB.updateStudentDomainComment(name, sel.value, ta.value.trim());
}

```

```

function buildDomainCommentEvidence(st, domain) {
  const assessments = filterAssessmentsForActiveCycle(st.assessments || [])
    .filter(a => (a.domain || '') === domain)
    .slice()
    .sort((a, b) => ((b.date || '').localeCompare(a.date || '')) || ((b.id ||
'').localeCompare(a.id || '')));

  const overall = { met: 0, wt: 0, nm: 0, total: 0 };
  const outcomeMap = {};

  assessments.forEach(a => {
    (a.outcomes || []).forEach(o => {
      if (!o || !o.fr) return;

```

```

const status = o.status || 'wt';
if (status === 'met' || status === 'wt' || status === 'nm') {
  overall[status] += 1;
  overall.total += 1;
}
if (!outcomeMap[o.fr]) {
  outcomeMap[o.fr] = {
    fr: o.fr,
    en: o.en || '',
    met: 0,
    wt: 0,
    nm: 0,
    total: 0,
    lastDate: a.date || ''
  };
}
if (status === 'met' || status === 'wt' || status === 'nm') {
  outcomeMap[o.fr][status] += 1;
  outcomeMap[o.fr].total += 1;
}
if ((a.date || '') > (outcomeMap[o.fr].lastDate || ''))
outcomeMap[o.fr].lastDate = a.date || '';
});
});

const allOutcomes = Object.values(outcomeMap);
const strengths = allOutcomes
  .filter(row => row.met > 0)
  .sort((a, b) => (b.met - a.met) || ((a.wt + a.nm) - (b.wt + b.nm)) || (b.total
- a.total) || a.fr.localeCompare(b.fr))
  .slice(0, 4)
  .map(row => ({
    outcome: row.fr,
    met: row.met,
    wt: row.wt,
    nm: row.nm,
    total: row.total,
    lastSeen: row.lastDate || ''
  })));

const nextSteps = allOutcomes
  .filter(row => (row.wt + row.nm) > 0)
  .sort((a, b) => ((b.wt + b.nm) - (a.wt + a.nm)) || (b.nm - a.nm) || (b.total -
a.total) || a.fr.localeCompare(b.fr))
  .slice(0, 4)
  .map(row => ({
    outcome: row.fr,
    wt: row.wt,
    nm: row.nm,
    met: row.met,

```



```

        total: row.total,
        lastSeen: row.lastDate || ''
    }));

    const recentAssessments = assessments.slice(0, 6).map(a => {
        const metOutcomes = (a.outcomes || []).filter(o => o.status === 'met').map(o =>
o.fr);
        const supportOutcomes = (a.outcomes || []).filter(o => o.status === 'wt' ||
o.status === 'nm').map(o => `${o.fr} (${o.status === 'wt' ? 'WT' : 'NM'})`);
        return {
            date: a.date || '',
            activity: a.activity || '',
            metOutcomes,
            supportOutcomes,
            teacherComment: a.comment || ''
        };
    });

    return {
        student: st.name,
        domain,
        assessmentCount: assessments.length,
        totals: overall,
        strengths,
        nextSteps,
        recentAssessments
    };
}

async function generateDomainCommentFromData(name) {
    const st = studentForActiveCycle(DB.getStudent(name));
    const sel = document.getElementById('profile-domain-comment-select');
    const ta = document.getElementById('profile-domain-comment-text');
    const btn = document.getElementById('profile-domain-comment-generate-btn');
    if (!st || !sel || !ta || !btn) return;
    if (!window.__key) {
        flash('Add your API key in Settings first.', 'warn');
        return;
    }

    const domain = sel.value;
    if (!domain) {
        flash('Choose a domain first.', 'warn');
        return;
    }

    const evidence = buildDomainCommentEvidence(st, domain);
    if (!evidence.assessmentCount) {
        flash('No assessment data found for that domain.', 'warn');
        return;
    }

```

```
}
```

```
const originalBtn = btn.textContent;  
const previousText = ta.value;  
btn.disabled = true;  
btn.textContent = 'Generating...';  
ta.value = 'Generating comment from stored domain data...';  
ta.style.height = 'auto';  
ta.style.height = ta.scrollHeight + 'px';
```

```
const prompt = `You are writing a concise domain comment for a student profile at  
an international school.
```

```
Use ONLY the evidence provided below.  
Do not invent details.  
Write in light Australian English.  
Keep the tone professional, clear and teacher-like.
```

```
Student: ${st.name}  
Domain: ${domain}
```

```
Evidence:  
${JSON.stringify(evidence, null, 2)}
```

```
Requirements:
```

- Write one paragraph only.
- Start with "In \${domain}, \${st.name} ..."
- Mention clear strengths shown in the stored outcomes.
- Mention one or two next areas to develop.
- If the evidence is limited or mixed, say so plainly.
- Keep it between 55 and 110 words.
- Do not mention percentages, scores, grades or marks.
- Do not mention that this was generated from data.
- Do not use em dashes.

```
Return ONLY valid JSON:
```

```
{  
  "comment": "<domain comment>"  
}`;
```

```
try {  
  const result = await callAnthropicJSON({  
    messageContent: [{ type: 'text', text: prompt }],  
    maxTokens: 260,  
    timeoutMs: 45000  
  });  
  const comment = (result?.comment || '').trim();  
  if (!comment) throw new Error('No comment returned.');
```

```
ta.value = comment;  
ta.style.height = 'auto';
```

```

    ta.style.height = ta.scrollHeight + 'px';
    saveProfileDomainComment(name);
    flash(`${domain} comment generated.`, 'success');
} catch (err) {
    ta.value = previousText;
    ta.style.height = 'auto';
    ta.style.height = ta.scrollHeight + 'px';
    flash('Could not generate domain comment: ' + err.message);
} finally {
    btn.disabled = false;
    btn.textContent = originalBtn;
}
}

```

```

function renderProfile(name) {
    currentProfileStudent = name;
    const st = studentForActiveCycle(DB.getStudent(name));
    if (!st) { document.getElementById('profile-content').innerHTML = '<p>Student not found.</p>'; return; }

```

```

    const assessments = (st.assessments || []).slice().sort((a, b) => {
        if (profileSortMode === 'performance') {
            return (getAssessmentPerformanceScore(a) - getAssessmentPerformanceScore(b))
                || ((b.date || '').localeCompare(a.date || ''))
                || ((b.id || '').localeCompare(a.id || ''));
        }
        return ((b.date || '').localeCompare(a.date || '')) || ((b.id ||
'').localeCompare(a.id || ''));
    });
    const domains = [...new Set(assessments.map(a => a.domain))];
    const filteredAssessments = profileHistoryOutcomeFilter
        ? assessments.filter(a => {
            if (profileHistoryDomainFilter && (a.domain || '') !==
profileHistoryDomainFilter) return false;
            return (a.outcomes || []).some(o => o && o.fr ===
profileHistoryOutcomeFilter);
        })
        : assessments;
    const visibleAssessments = profileHistoryOutcomeFilter
        ? filteredAssessments
        : (profileHistoryShowAll ? filteredAssessments : filteredAssessments.slice(0,
5));
    const hiddenHistoryCount = profileHistoryOutcomeFilter ? 0 :
Math.max(filteredAssessments.length - visibleAssessments.length, 0);
    const domainCounts = {};
    assessments.forEach(a => { domainCounts[a.domain] =
(domainCounts[a.domain] || 0) + 1; });

```

```

// Recurring challenges: outcomes appearing as wt or nm 2+ times

```

```

const challengeMap = {};
assessments.forEach(a => a.outcomes.forEach(o => {
  if (o.status === 'wt' || o.status === 'nm') {
    const key = o.fr;
    if (!challengeMap[key]) challengeMap[key] = { fr: o.fr, en: o.en, count: 0 };
    challengeMap[key].count++;
  }
}));
const challenges = Object.values(challengeMap).filter(c => c.count >=
2).sort((a,b) => b.count-a.count);

const domainChips = Object.entries(domainCounts).map(([d,c]) =>
`<span class="domain-tag" style="font-size:11px;padding:3px 10px">${esc(d)}
×${c}</span>`
).join('');

const challengeHtml = challenges.length ? `
<div class="card" style="margin-bottom:14px">
  <div class="section-label">Recurring challenges</div>
  <div style="display:flex;flex-wrap:wrap;gap:4px">
    ${challenges.map(c => `<span class="challenge-chip">⚠
${esc(getOutcomeDisplayPrimary(c))} ×${c.count}</span>`).join('')}
  </div>
</div>` : '';

const domainCommentHtml = domains.length ? `
<div class="card profile-collapsible ${profileCommentPanelOpen ? 'open' : ''}"
id="profile-comment-panel">
  <button class="profile-collapsible-toggle"
onclick="toggleProfileCommentPanel()" type="button">
    <div>
      <div class="profile-collapsible-title">Generate comment</div>
      <div class="profile-collapsible-sub">Build a domain comment from stored
outcomes and teacher notes, then tweak it if needed.</div>
    </div>
    <div class="profile-collapsible-arrow">√</div>
  </button>
  <div class="profile-collapsible-body">
    <div
style="display:flex;gap:8px;flex-wrap:wrap;align-items:center;margin-bottom:8px">
      <select id="profile-domain-comment-select"
onchange="switchProfileDomainComment('${escAttr(name)}')"
style="font-size:13px;padding:8px 10px;flex:1;min-width:220px">
        ${domains.map(d => `<option
value="${escAttr(d)}">${esc(d)}</option>`).join('')}
      </select>
      <button class="btn btn-primary" id="profile-domain-comment-generate-btn"
onclick="generateDomainCommentFromData('${escAttr(name)}')"
style="white-space:nowrap">⚡ Generate from data</button>
    </div>

```

```

    <div class="form-hint" style="margin-bottom:8px">Uses stored outcomes and
comments from the selected domain only.</div>
    <textarea class="notes-field" id="profile-domain-comment-text"
placeholder="Generate a domain comment, then edit if needed."
oninput="this.style.height='auto';this.style.height=this.scrollHeight+'px'"
onblur="saveProfileDomainComment('${escAttr(name)}')">${esc(((st.domainComments||{})
)[domains[0]]) || ''})</textarea>
  </div>
</div>` : '';

```

```

const historyHtml = visibleAssessments.map(a => {
  const batchSession = a.type === 'batch' ? findBatchSessionByAssessmentId(a.id)
: null;
  const thumbEl = a.thumb
  ? ``
  : a.type === 'batch'
  ? `<div class="batch-thumb-ph">🖼️</div>`
  : `<div class="hist-thumb-ph">🖼️</div>`;
  const batchBadge = a.type === 'batch' ? `<span
class="batch-badge">Batch</span>` : '';
  const activityNote = a.type === 'batch' && a.activity ? `<div
style="font-size:11px;color:var(--ink-soft);margin-top:1px;font-style:italic">${esc
(a.activity)}</div>` : '';
  const outcomes = a.outcomes.map(o => {
    const isMatch = profileHistoryOutcomeFilter && o.fr ===
profileHistoryOutcomeFilter && (!profileHistoryDomainFilter || (a.domain || '') ===
profileHistoryDomainFilter);
    return `
      <div class="hist-outcome" ${isMatch ?
'style="background:var(--accent-lt);border:1px solid
var(--accent-bd);border-radius:8px;padding:7px 8px"' : ''}>
        <div class="hist-dot"
style="background:${o.status==='met'?'var(--met)':o.status==='wt'?'var(--wt)':'var(
--nm)'}"></div>
        <div class="hist-status"
style="color:${o.status==='met'?'var(--met)':o.status==='wt'?'var(--wt)':'var(--nm)
'};min-width:88px">${STATUS_LABEL[o.status]}</div>
        <div><div
class="hist-outcome-fr">${esc(getOutcomeDisplayPrimary(o))}${isMatch ? ' <span
style="font-size:10px;color:var(--accent);font-weight:700;text-transform:uppercase;
letter-spacing:0.06em">Match</span>' : ''}</div>${getOutcomeDisplaySecondary(o) ?
`<div class="hist-outcome-en">${esc(getOutcomeDisplaySecondary(o))}</div>` :
''}</div>
      </div>`;
  }).join('');
  const cmt = a.comment ? `<div class="hist-comment">📝 ${esc(a.comment)}</div>`
: '';
  return `<div class="assess-hist-card">
    <div class="hist-header">
      ${thumbEl}

```

```

        <div class="hist-meta">
            <div class="hist-date">${formatDate(a.date)}</div>
            <div
class="hist-domain">${esc(a.domain)}${batchBadge}</div>${activityNote}
            </div>
            ${batchSession ? `<button class="btn btn-secondary"
onclick='event.stopPropagation(); reopenBatchSession(${jsq(batchSession.id)}), {
sourceTab: "profile", studentName: ${jsq(name)} }'); return false;' type="button"
style="padding:6px 10px;font-size:11px;white-space:nowrap">⌂ Open grid</button>` :
''}

            <button class="btn-icon"
onclick="openEditAssessment('${esc(name)}','${a.id}')" title="Edit">✎</button>
            <button class="btn-icon"
onclick="deleteAssessment('${esc(name)}','${a.id}')" title="Delete"
style="font-size:14px">🗑</button>
            </div>
            <div class="hist-outcomes">${outcomes}</div>
            ${cmt}
        </div>`;
    }).join('');

    document.getElementById('profile-content').innerHTML = `
        <div class="profile-header">
            <div style="display:flex;align-items:center;gap:10px;margin-bottom:4px"><div
class="profile-name" style="margin-bottom:0">${esc(name)}</div><button
onclick="openRenameStudent('${esc(name)}')" title="Edit name"
style="background:none;border:none;cursor:pointer;font-size:16px;color:var(--ink-so
ft);padding:2px;line-height:1">✎</button></div>
            <div style="display:flex;align-items:center;gap:8px;margin-bottom:10px">
                <select id="profile-class-select"
onchange="saveProfileClass('${esc(name)}')" style="font-size:12px;padding:4px
9px;border:1.5px solid
var(--border);border-radius:7px;font-family:'Sora',sans-serif;background:var(--bg);
color:var(--ink);outline:none">
                    <option value="">No class assigned</option>
                    ${DB.getClasses().map(c => `<option value="${esc(c)}" ${((st.class||'')==
c?'selected':'')}>${esc(c)}</option>`).join('')}
                </select>
                <div class="profile-meta" style="margin-bottom:0">Last assessed:
${formatDate(assessments[0]?.date)}</div>
            </div>
            <div class="stat-row">
                <div class="stat-box"><div class="sn">${assessments.length}</div><div
class="sl">Assessments</div></div>
                <div class="stat-box"><div class="sn">${domains.length}</div><div
class="sl">Domains</div></div>
                <div class="stat-box"><div class="sn">${challenges.length}</div><div
class="sl">Recurring challenges</div></div>
            </div>
            <div class="domain-tags" style="flex-wrap:wrap">${domainChips}</div>

```

```

        <div class="profile-header-actions">
            ${domains.length ? `<button class="btn btn-primary profile-inline-btn"
onclick="toggleProfileCommentPanel(true)" type="button">⚡ Generate
comment</button>` : ''}
        </div>
    </div>
    ${domainCommentHtml}
    ${renderStudentProfileDashboard(st)}
    <div class="card profile-collapsible ${profileHistoryPanelOpen ? 'open' : ''}"
id="profile-history-panel">
        <button class="profile-collapsible-toggle"
onclick="toggleProfileHistoryPanel()" type="button">
            <div>
                <div class="profile-collapsible-title">Assessment history</div>
                <div class="profile-collapsible-sub">Audit trail and edit zone. Opens
when you jump here from Outcome mastery.</div>
            </div>
            <div class="profile-collapsible-arrow">v</div>
        </button>
        <div class="profile-collapsible-body">
            <div class="profile-history-note">Use Outcome mastery above for the summary
view. Click a mastery row to filter this history down to the matching
evidence.</div>
            <div class="profile-history-filter ${profileHistoryOutcomeFilter ? '' :
'hidden'}" id="profile-history-filter-bar">
                <div class="profile-history-filter-main">
                    Showing evidence for
                    <span
class="profile-history-filter-chip">${esc(profileHistoryOutcomeFilter ||
'')}</span>
                    ${profileHistoryDomainFilter ? `<span
class="profile-history-filter-chip"
style="background:var(--surface);color:var(--accent);border-color:var(--accent-bd)"
>${esc(profileHistoryDomainFilter)}</span>` : ''}
                    <span style="color:var(--ink-soft)">• ${filteredAssessments.length}
matching assessment${filteredAssessments.length !== 1 ? 's' : ''}</span>
                </div>
                <button class="btn btn-secondary"
onclick="setProfileHistoryOutcomeFilter('')" type="button" style="padding:7px
12px;font-size:12px">Clear filter</button>
            </div>
            ${historyHtml || '<div class="card"><div class="sv-empty">No matching
assessment history for this outcome yet.</div></div>'}
            ${((hiddenHistoryCount > 0 && !profileHistoryOutcomeFilter) ? `<div
class="btn-row" style="margin-top:10px"><button class="btn btn-secondary"
onclick="setProfileHistoryShowAll(true)" type="button">Show ${hiddenHistoryCount}
older assessment${hiddenHistoryCount !== 1 ? 's' : ''}</button></div>` : ''}
            ${((profileHistoryShowAll && filteredAssessments.length > 5 &&
!profileHistoryOutcomeFilter) ? `<div class="btn-row"
style="margin-top:10px"><button class="btn btn-secondary"

```

```

onclick="setProfileHistoryShowAll(false)" type="button">Show fewer</button></div>`
: ''}
    </div>
  </div>
  <div class="card" style="margin-bottom:14px">
    <div class="section-label">Teacher notes</div>
    <textarea class="notes-field" id="profile-notes" placeholder="e.g. EAL
student, joined mid-year, has IEP for reading..."
onblur="saveProfileNotes('${esc(name)}')">${esc(st.notes||'')}</textarea>
  </div>
  ${challengeHtml}
  ${renderStudentRecentOutcomeMap(st)}
  <div class="btn-row no-print" style="margin-top:4px">
    <button class="btn btn-secondary" onclick="printProfile('${esc(name)}')">🖨️
Print profile</button>
    <button class="btn btn-secondary"
onclick="exportStudentExcel('${esc(name)}')">📄 Export to Excel</button>
  </div>
`;

const domainCommentTa = document.getElementById('profile-domain-comment-text');
if (domainCommentTa) {
  domainCommentTa.style.height = 'auto';
  domainCommentTa.style.height = domainCommentTa.scrollHeight + 'px';
}
}

function saveProfileNotes(name) {
  const ta = document.getElementById('profile-notes');
  if (ta) DB.updateStudentNotes(name, ta.value.trim());
}

function saveProfileClass(name) {
  const sel = document.getElementById('profile-class-select');
  if (sel) DB.updateStudentClass(name, sel.value);
  renderRoster(); // refresh badges
}

function buildEditOutcomeSelectOptions(domain, currentValue) {
  const entries = getAllowedOutcomeEntries(domain);
  const optionValues = entries.map(entry => entry.stored);
  const options = [];

  if (currentValue && !optionValues.includes(currentValue)) {
    options.push(`<option value="${escAttr(currentValue)}">${esc(currentValue)}
(current)</option>`);
  }
}

```



```

    entries.forEach(entry => {
      options.push(`<option
value="${escAttr(entry.stored)}">${esc(entry.stored)}</option>`);
    });

    return options.join('');
  }

function findBatchSessionByAssessmentId(assessmentId) {
  return DB.getSessions().find(session => session.type === 'batch' &&
(session.students || []).some(entry => entry.assessmentId === assessmentId)) ||
  null;
}

function getBatchSessionAssessment(session, studentName, assessmentId) {
  const data = DB.loadData();
  const student = data.students?.[studentName];
  if (!student) return null;
  return (student.assessments || []).find(item => item.id === assessmentId) ||
  null;
}

function normalizeBatchReopenOutcomes(entries) {
  const firstWithOutcomes = entries.find(entry =>
Array.isArray(entry.assessment?.outcomes) && entry.assessment.outcomes.length);
  const sourceOutcomes = firstWithOutcomes?.assessment?.outcomes || [];
  if (!sourceOutcomes.length) return [{ fr: '', en: '' }];
  return sourceOutcomes.map(o => ({ fr: o?.fr || o?.en || '', en: o?.en || o?.fr ||
'' }));
}

function getRecordCycleLabel(record) {
  const cycle = normaliseCycle(record?.cycle || 'cycle3');
  const year = normaliseYearForCycle(cycle, record?.yearLevel || '');
  return `${getCycleLabel(cycle)} (${year})`;
}

async function ensureCycleForBatchSession(session) {
  const targetCycle = normaliseCycle(session?.cycle || 'cycle3');
  const targetYear = normaliseYearForCycle(targetCycle, session?.yearLevel || '');
  if (targetCycle === currentCycle) return true;
  const ok = await appConfirm(
    'Switch cycle to reopen this batch?',
    `This saved batch belongs to ${getCycleLabel(targetCycle)} (${targetYear}).
Assess is currently set to ${getAssessCycleContext()}. Switching now keeps domains
and outcomes aligned.`);
  `Switch to ${getCycleLabel(targetCycle)}`,
  'btn-primary'
  );
  if (!ok) return false;

```

```

const settings = DB.loadSettings();
applyAssessCycle(targetCycle, targetYear);
DB.saveSettings({
  ...settings,
  preferredCycle: currentCycle,
  yearLevel: currentYearLevel
});
return true;
}

async function reopenBatchSession(sessionId, options = {}) {
  try {
    const session = DB.getSessions().find(entry => entry.id === sessionId);
    if (!session || session.type !== 'batch') {
      flash('Could not reopen that batch record.', 'warn');
      return;
    }
    if (!await ensureCycleForBatchSession(session)) return;

    const entries = (session.students || []).map(entry => ({
      ...entry,
      assessment: getBatchSessionAssessment(session, entry.name,
entry.assessmentId)
    })).filter(entry => entry.assessment);

    if (!entries.length) {
      flash('No saved student records were found for that batch.', 'warn');
      return;
    }

    const firstAssessment = entries[0].assessment;
    const normalizedOutcomes = normalizeBatchReopenOutcomes(entries);

    batchEditingSessionId = session.id;
    batchEditReturn = { sourceTab: options.sourceTab || 'students', studentName:
options.studentName || '' };
    batchContext = {
      domain: firstAssessment?.domain || session.domain || '',
      outcomes: normalizedOutcomes,
      description: session.activity || firstAssessment?.activity || ''
    };

    batchStudentStatuses = {};
    const forcedStudents = entries.map(entry => {
      const st = DB.getStudent(entry.name);
      const savedStatuses = (entry.assessment?.outcomes || []).map(o => o?.status
|| null);
      const paddedStatuses = normalizedOutcomes.map((_, index) =>
savedStatuses[index] || null);
      batchStudentStatuses[entry.name] = {

```

```

        outcomeStatuses: paddedStatuses,
        comment: entry.assessment?.comment || '',
        absent: false,
        assessmentId: entry.assessment?.id || ''
    });
    return { name: entry.name, assessments: st?.assessments || [], class:
st?.class || '', _saved: true };
    });

    goTab('record');
    populateBatchClassSelect();
    populateBatchStudentSelect();
    showBatchStep('grid');

    requestAnimationFrame(() => {
        try {
            showBatchGrid({ preserveStatuses: true, forcedStudents, disableClassFilter:
true });
            document.getElementById('batch-step-input').style.display = 'none';
            document.getElementById('batch-step-grid').style.display = 'block';
            window.scrollTo({ top: 0, behavior: 'smooth' });
            flash(`Opened saved batch grid for ${forcedStudents.length}
students${forcedStudents.length !== 1 ? 's' : ''}.`, 'success');
        } catch (innerErr) {
            console.error('Batch reopen render failed', innerErr);
            flash('Opened the record tab, but the saved grid failed to render.',
'warn');
        }
    });
} catch (err) {
    console.error('Batch reopen failed', err);
    flash('Could not reopen that batch record.', 'warn');
}
}

```

```

function applyBatchOutcomeLabelsToSession(sessionId, domain, outcomeLabels) {
    if (!sessionId || !domain || !Array.isArray(outcomeLabels) ||
!outcomeLabels.length) return 0;
    const data = DB.loadData();
    const session = (data.sessions || []).find(entry => entry.id === sessionId);
    if (!session) return 0;

    let updatedCount = 0;
    (session.students || []).forEach(entry => {
        const student = data.students?.[entry.name];
        if (!student) return;
        const assessment = (student.assessments || []).find(item => item.id ===
entry.assessmentId);
        if (!assessment || assessment.domain !== domain) return;
        assessment.outcomes = (assessment.outcomes || []).map((outcome, index) => {

```

```

        const nextLabel = outcomeLabels[index];
        if (!nextLabel) return outcome;
        return { ...outcome, fr: nextLabel, en: nextLabel };
    });
    updatedCount += 1;
});

DB.saveData(data);
return updatedCount;
}

async function deleteAssessment(name, id) {
    if (!await appConfirm('Delete assessment?', 'This record will be removed from the student profile. Cannot be undone.', 'Delete', 'btn-ghost')) return;
    DB.deleteAssessment(name, id);
    renderProfile(name);
}

function openEditAssessment(name, id) {
    const st = DB.getStudent(name);
    if (!st) return;
    const a = st.assessments.find(x => x.id === id);
    if (!a) return;
    editingStudent = name;
    editingAssessmentId = id;
    const batchSession = a.type === 'batch' ? findBatchSessionByAssessmentId(id) : null;
    editingBatchSessionId = batchSession?.id || null;
    editingBatchAssessmentCount = (batchSession?.students || []).length || 0;
    document.getElementById('edit-modal-title').textContent = name + ' - ' + a.domain;
    document.getElementById('edit-modal-outcomes').innerHTML = a.outcomes.map((o, i) => `
        <div class="form-group" style="padding:12px;border:1px solid
var(--border);border-radius:10px;background:var(--bg)">
            <label class="form-label" style="font-weight:600;margin-bottom:8px">Outcome
${i + 1}</label>
            <select id="em-outcome-${i}" style="font-size:13px;padding:7px
10px;margin-bottom:8px">
                ${buildEditOutcomeSelectOptions(a.domain, o.fr)}
            </select>
            <select id="em-status-${i}" style="font-size:13px;padding:7px 10px">
                <option value="met" ${o.status==='met'? 'selected':''}>Met</option>
                <option value="wt" ${o.status==='wt' ? 'selected':''}>Working
Towards</option>
                <option value="nm" ${o.status==='nm' ? 'selected':''}>Not Met</option>
            </select>
            <div class="form-hint" style="margin-top:6px">Change the outcome if the
assessment was tagged to the wrong learning outcome.</div>
        </div>`).join('');

```

```

a.outcomes.forEach((o, i) => {
  const outcomeSelect = document.getElementById('em-outcome-' + i);
  if (outcomeSelect) outcomeSelect.value = o.fr;
});
document.getElementById('edit-modal-comment').value = a.comment || '';

const batchScope = document.getElementById('edit-modal-batch-scope');
const batchToggle = document.getElementById('edit-modal-apply-batch');
const batchHint = document.getElementById('edit-modal-batch-hint');
if (a.type === 'batch' && editingBatchAssessmentCount > 1) {
  batchScope.style.display = 'block';
  batchToggle.checked = true;
  batchHint.textContent = `This will update the selected learning outcome label
across ${editingBatchAssessmentCount} student records from the same saved batch
activity. Statuses and comments stay individual.`;
} else {
  batchScope.style.display = 'none';
  batchToggle.checked = false;
  batchHint.textContent = '';
}

document.getElementById('edit-modal').classList.add('open');
}

function saveAssessmentEdit() {
  const st = DB.getStudent(editingStudent);
  if (!st) return;
  const a = st.assessments.find(x => x.id === editingAssessmentId);
  if (!a) return;

  const originalOutcomeLabels = (a.outcomes || []).map(outcome => outcome.fr ||
  '');
  const updatedOutcomes = a.outcomes.map((o, i) => {
    const statusSel = document.getElementById('em-status-' + i);
    const outcomeSel = document.getElementById('em-outcome-' + i);
    const chosenOutcome = (outcomeSel?.value || o.fr || '').trim();
    const canonical = canonicalizeOutcomeChoice(a.domain, chosenOutcome);
    const storedOutcome = canonical ? canonical.stored : chosenOutcome;
    return {
      ...o,
      fr: storedOutcome,
      en: storedOutcome,
      status: statusSel ? statusSel.value : o.status
    };
  });

  const outcomeLabelsChanged = updatedOutcomes.some((outcome, index) => outcome.fr
  !== (originalOutcomeLabels[index] || ''));
  const nextComment = document.getElementById('edit-modal-comment').value.trim();
  DB.updateAssessment(editingStudent, editingAssessmentId, { outcomes:

```

```

updatedOutcomes, comment: nextComment });

let batchUpdatedCount = 0;
const shouldApplyBatch = editingBatchSessionId &&
document.getElementById('edit-modal-apply-batch')?.checked;
if (shouldApplyBatch && outcomeLabelsChanged) {
  batchUpdatedCount = applyBatchOutcomeLabelsToSession(editingBatchSessionId,
a.domain, updatedOutcomes.map(outcome => outcome.fr));
}

const profileStudentName = editingStudent;
closeEditModal();
if (profileStudentName) renderProfile(profileStudentName);
if (batchUpdatedCount > 1) flash(`Updated the learning outcome across
${batchUpdatedCount} batch records.`, 'success');
else flash('Assessment updated.', 'success');
}

function closeEditModal() {
  document.getElementById('edit-modal').classList.remove('open');
  editingStudent = null;
  editingAssessmentId = null;
  editingBatchSessionId = null;
  editingBatchAssessmentCount = 0;
  const batchScope = document.getElementById('edit-modal-batch-scope');
  const batchToggle = document.getElementById('edit-modal-apply-batch');
  const batchHint = document.getElementById('edit-modal-batch-hint');
  if (batchScope) batchScope.style.display = 'none';
  if (batchToggle) batchToggle.checked = false;
  if (batchHint) batchHint.textContent = '';
}

function printProfile(name) {
  const st = studentForActiveCycle(DB.getStudent(name));
  if (!st) return;
  const today = new Date().toLocaleDateString('en-GB',
{day:'numeric',month:'long',year:'numeric'});
  const cards = st.assessments.map(a => {
    const outs = a.outcomes.map(o => `
      <div class="pc-outcome">
        <div class="pc-dot ${o.status}"></div>
        <div class="pc-status ${o.status}">${STATUS_LABEL[o.status]}</div>
      </div><div
class="pc-fr">${esc(getOutcomeDisplayPrimary(o))}</div>${getOutcomeDisplaySecondary
(o) ? `<div class="pc-en">${esc(getOutcomeDisplaySecondary(o))}</div>` : ''}</div>
      </div>`).join('');
    const cmt = a.comment ? `<div class="pc-comment"><strong>Next step:</strong>
${esc(a.comment)}</div>` : '';
    return `<div class="pc">
      <div class="pc-name">${esc(name)}</div>

```

```

        <div class="pc-domain">${esc(a.domain)} · ${formatDate(a.date)}</div>
        ${outs}${cmt}
    </div>`;
    }).join('');
    document.getElementById('print-area').innerHTML = `<div
class="print-head"><h1>Portfolio – ${esc(name)}</h1><p>Printed ${today} ·
${st.assessments.length} assessments · French curriculum</p></div>${cards}`;
    window.print();
}

/* =====
HEATMAP TAB
===== */

let hmView = 'grid';
let hmCardDetailKey = null;
let hmSupportShowAll = false;

function setHmView(v) {
    hmView = v;
    document.getElementById('hm-view-list').classList.toggle('active', v === 'list');
    document.getElementById('hm-view-grid').classList.toggle('active', v === 'grid');
    renderHeatmap();
}

function renderHeatmap() {
    const students = DB.getAllStudents().map(student => ({
        ...student,
        assessments: filterAssessmentsForActiveCycle(student.assessments || [])
    }));
    if (!students.length) {
        document.getElementById('heatmap-list').innerHTML = `<div
class="empty-state"><div class="empty-icon"><img alt="Empty state icon" data-bbox="528 594 548 608"/></div><p>No data yet. Assess some
students first.</p></div>`;
        document.getElementById('hm-summary').textContent = '';
        renderHeatmapDashboard(null);
        return;
    }

    const classFilterEl = document.getElementById('hm-class-filter');
    const currentClass = classFilterEl?.value || '';
    const allClasses = DB.getClasses();
    if (classFilterEl) {
        classFilterEl.innerHTML = '<option value="">All classes</option>' +
            allClasses.map(c => `<option value="${esc(c)}"
${c===currentClass?'selected':''}>${esc(c)}</option>`).join('');
    }
    const filteredStudents = (currentClass ? students.filter(s => s.class ===
currentClass) : students)
        .filter(student => (student.assessments || []).length);

```

```

    const allDomains = [...new Set(filteredStudents.flatMap(s => s.assessments.map(a
=> a.domain)))].sort();
    const domainFilter = document.getElementById('hm-domain-filter');
    const currentDomain = domainFilter.value;
    domainFilter.innerHTML = '<option value="">All domains</option>' +
        allDomains.map(d => `<option value="${esc(d)}"
${d===currentDomain?'selected':''}>${esc(d)}</option>`).join('');

    const dayFilter = parseInt(document.getElementById('hm-date-filter')?.value ||
'0');
    const cutoff = dayFilter ? new Date(Date.now() - dayFilter *
86400000).toISOString().split('T')[0] : null;

    const dataset = buildHeatmapDataset(filteredStudents, currentDomain, cutoff);

    if (!dataset.allOutcomeKeys.length) {
        document.getElementById('heatmap-list').innerHTML = `<div
class="empty-state"><p>No outcomes recorded yet.</p></div>`;
        document.getElementById('hm-summary').textContent = '';
        renderHeatmapDashboard(dataset);
        return;
    }

    document.getElementById('hm-summary').textContent =
` ${dataset.allOutcomeKeys.length}
outcome${dataset.allOutcomeKeys.length!==1?'s':''} .
${dataset.studentsWithData.length}
student${dataset.studentsWithData.length!==1?'s':''}`;

    renderHeatmapDashboard(dataset);

    if (hmView === 'grid') {
        renderHeatmapGrid(dataset.studentsWithData, dataset.domainOutcomes,
dataset.outcomeInfo, dataset.studentOutcomeMap);
    } else {
        renderHeatmapList(dataset.allOutcomeKeys, dataset.outcomeInfo,
dataset.studentOutcomeMap, dataset.studentsWithData);
    }
}

function buildHeatmapDataset(filteredStudents, currentDomain, cutoff) {
    const studentOutcomeMap = {};
    const outcomeInfo = {};
    const domainOutcomes = {};
    const domainStats = {};
    const timelineAssessments = [];
    const assessmentSessionMap = buildAssessmentSessionMap();

    filteredStudents.forEach(st => {
        if (!studentOutcomeMap[st.name]) studentOutcomeMap[st.name] = {};
    });

```



```

st.assessments.forEach(a => {
  if (currentDomain && a.domain !== currentDomain) return;
  if (cutoff && a.date < cutoff) return;
  const assessmentDate = a.date || '';
  a.outcomes.forEach(o => {
    const key = o.fr;
    if (!key) return;
    if (!outcomeInfo[key]) {
      outcomeInfo[key] = { fr: o.fr, en: o.en, domain: a.domain };
      if (!domainOutcomes[a.domain]) domainOutcomes[a.domain] = [];
      if (!domainOutcomes[a.domain].includes(key))
domainOutcomes[a.domain].push(key);
    }
    if (!studentOutcomeMap[st.name][key]) studentOutcomeMap[st.name][key] =
o.status;
    timelineAssessments.push({
      student: st.name,
      domain: a.domain,
      date: assessmentDate,
      status: o.status,
      outcomeKey: key,
      assessmentId: a.id || '',
      eventId: assessmentSessionMap[a.id] || `assessment:${a.id ||
`${st.name}:${assessmentDate}:${a.domain}:${key}`}`
    });
  });
});
});
});
});

```

```

const coverageRows = buildHeatmapCoverageRows(timelineAssessments,
currentDomain);
const allOutcomeKeys = Object.keys(outcomeInfo);
const studentsWithData = filteredStudents.filter(s =>
Object.keys(studentOutcomeMap[s.name]||{}).length > 0);

```

```

const overall = { met: 0, wt: 0, nm: 0, total: 0 };
const supportStudents = new Set();
const assessedStudents = new Set();
const outcomeRows = [];

```

```

allOutcomeKeys.forEach(key => {
  const info = outcomeInfo[key];
  let met = 0, wt = 0, nm = 0;
  const stuSet = new Set();
  studentsWithData.forEach(st => {
    const s = studentOutcomeMap[st.name]?.[key];
    if (!s || s === 'none') return;
    stuSet.add(st.name);
    assessedStudents.add(st.name);
    if (s === 'met') met++;
  });
});

```

```

        else if (s === 'wt') { wt++; supportStudents.add(st.name); }
        else if (s === 'nm') { nm++; supportStudents.add(st.name); }
    });
    const total = met + wt + nm;
    if (!total) return;
    overall.met += met;
    overall.wt += wt;
    overall.nm += nm;
    overall.total += total;

    if (!domainStats[info.domain]) {
        domainStats[info.domain] = { domain: info.domain, met: 0, wt: 0, nm: 0,
total: 0, outcomeCount: 0, studentSet: new Set() };
    }
    domainStats[info.domain].met += met;
    domainStats[info.domain].wt += wt;
    domainStats[info.domain].nm += nm;
    domainStats[info.domain].total += total;
    domainStats[info.domain].outcomeCount += 1;
    stuSet.forEach(name => domainStats[info.domain].studentSet.add(name));

    outcomeRows.push({
        key,
        info,
        met,
        wt,
        nm,
        total,
        supportRate: total ? (wt + nm) / total : 0,
        metRate: total ? met / total : 0,
        studentCount: stuSet.size
    });
});

const domainRows = Object.values(domainStats).map(d => ({
    ...d,
    studentCount: d.studentSet.size,
    supportRate: d.total ? (d.wt + d.nm) / d.total : 0,
    metRate: d.total ? d.met / d.total : 0
}))).sort((a,b) => (b.supportRate - a.supportRate) || (a.domain ||
'').localeCompare(b.domain || ''));

const studentRows = studentsWithData.map(st => {
    const entries = Object.entries(studentOutcomeMap[st.name] || {});
    let met = 0, wt = 0, nm = 0;
    const domainMap = {};
    entries.forEach(([key, status]) => {
        const domain = outcomeInfo[key]?.domain || 'Unknown';
        if (!domainMap[domain]) domainMap[domain] = { met: 0, wt: 0, nm: 0, total: 0
};
    });

```

```

        if (status === 'met') met++;
        else if (status === 'wt') wt++;
        else if (status === 'nm') nm++;
        else return;
        domainMap[domain].total++;
        domainMap[domain][status]++;
    });
    const weakestDomainRow = Object.entries(domainMap)
        .map(([domain, stats]) => ({
            domain,
            supportCount: (stats.wt || 0) + (stats.nm || 0),
            supportRate: stats.total ? (((stats.wt || 0) + (stats.nm || 0)) /
stats.total) : 0
        })))
        .sort((a,b) => (b.supportCount - a.supportCount) || (b.supportRate -
a.supportRate) || a.domain.localeCompare(b.domain))[0] || null;
    const latestConcern = timelineAssessments
        .filter(entry => entry.student === st.name && (entry.status === 'wt' ||
entry.status === 'nm'))
        .sort((a,b) => (b.date || '').localeCompare(a.date || ''))[0] || null;
    return {
        name: st.name,
        assessedCount: met + wt + nm,
        met, wt, nm,
        supportCount: wt + nm,
        weakestDomain: weakestDomainRow ? weakestDomainRow.domain : '',
        lastConcernDate: latestConcern?.date || ''
    };
    }).sort((a,b) => (b.supportCount - a.supportCount) || (b.nm - a.nm) || (b.wt -
a.wt) || a.name.localeCompare(b.name));

    const trendRows = buildHeatmapTrendBuckets(timelineAssessments, cutoff);
    const toughestOutcome = outcomeRows.slice().sort((a,b) => (b.supportRate -
a.supportRate) || (b.total - a.total))[0] || null;
    const strongestDomain = domainRows.slice().sort((a,b) => (b.metRate - a.metRate)
|| (b.total - a.total))[0] || null;
    const needsSupportCount = supportStudents.size;
    const metRate = overall.total ? Math.round((overall.met / overall.total) * 100) :
0;

    return {
        filteredStudents,
        studentOutcomeMap,
        outcomeInfo,
        domainOutcomes,
        allOutcomeKeys,
        studentsWithData,
        overall,
        outcomeRows,
        studentRows,

```

```

    domainRows,
    trendRows,
    coverageRows,
    toughestOutcome,
    strongestDomain,
    needsSupportCount,
    assessedStudentCount: assessedStudents.size,
    metRate
  };
}

function buildAssessmentSessionMap() {
  const map = {};
  DB.getSessions().forEach(session => {
    const eventId = `session:${session.id}`;
    (session.students || []).forEach(entry => {
      if (entry.assessmentId) map[entry.assessmentId] = eventId;
    });
  });
  return map;
}

function buildHeatmapCoverageRows(timelineAssessments, currentDomain) {
  const useCounts = {};
  timelineAssessments.forEach(entry => {
    const domain = entry.domain || '';
    const match = canonicalizeOutcomeChoice(domain, entry.outcomeKey);
    const stored = match?.stored || entry.outcomeKey;
    if (!domain || !stored) return;
    const key = `${domain}\n${stored}`;
    if (!useCounts[key]) useCounts[key] = { events: new Set(), students: new Set(),
lastSeen: '' };
    if (entry.eventId) useCounts[key].events.add(entry.eventId);
    if (entry.student) useCounts[key].students.add(entry.student);
    if (entry.date && entry.date > useCounts[key].lastSeen) useCounts[key].lastSeen
= entry.date;
  });

  const domains = currentDomain ? [currentDomain] : APPROVED_DOMAIN_LIST.slice();
  return domains.flatMap(domain => getAllowedOutcomeEntries(domain).map(entry => {
    const stats = useCounts[`${domain}\n${entry.stored}`] || { count: 0, students:
new Set(), lastSeen: '' };
    return {
      domain,
      subdomain: entry.subdomain || 'General',
      outcome: entry.outcome,
      stored: entry.stored,
      count: stats.events ? stats.events.size : 0,
      studentCount: stats.students ? stats.students.size : 0,
      lastSeen: stats.lastSeen || ''
    };
  }));
}

```

```

    });
  })).sort((a, b) =>
    (a.count - b.count)
    || a.domain.localeCompare(b.domain)
    || a.subdomain.localeCompare(b.subdomain)
    || a.outcome.localeCompare(b.outcome)
  );
}

function buildHeatmapTrendBuckets(entries, cutoff) {
  const dated = entries.filter(e => e.date);
  if (!dated.length) return [];
  const dayWindow = cutoff ? Math.max(1, Math.round((Date.now() - new
Date(cutoff).getTime()) / 86400000)) : 365;
  let mode = 'month';
  if (dayWindow <= 45) mode = 'day';
  else if (dayWindow <= 140) mode = 'week';

  const buckets = {};
  dated.forEach(entry => {
    const key = heatmapBucketKey(entry.date, mode);
    if (!buckets[key]) buckets[key] = { key, label: heatmapBucketLabel(key, mode),
met: 0, wt: 0, nm: 0, total: 0 };
    if (entry.status === 'met') buckets[key].met++;
    else if (entry.status === 'wt') buckets[key].wt++;
    else if (entry.status === 'nm') buckets[key].nm++;
    else return;
    buckets[key].total++;
  });

  return Object.values(buckets)
    .filter(b => b.total > 0)
    .sort((a,b) => a.key.localeCompare(b.key))
    .slice(-10);
}

function heatmapBucketKey(dateStr, mode) {
  const d = new Date(dateStr + 'T00:00:00');
  if (Number.isNaN(d.getTime())) return dateStr;
  if (mode === 'day') return dateStr;
  if (mode === 'week') {
    const day = d.getDay();
    const diff = (day + 6) % 7;
    d.setDate(d.getDate() - diff);
    return d.toISOString().split('T')[0];
  }
  return `${d.getFullYear()}-${String(d.getMonth()+1).padStart(2, '0')}`;
}

function heatmapBucketLabel(key, mode) {

```

```

    if (mode === 'day') {
      const [y, m, d] = key.split('-');
      return `${d}/${m}`;
    }
    if (mode === 'week') {
      const d = new Date(key + 'T00:00:00');
      const end = new Date(d);
      end.setDate(end.getDate() + 6);
      return
`${String(d.getDate()).padStart(2, '0')}/${String(d.getMonth()+1).padStart(2, '0')}`;
    }
    const [y, m] = key.split('-');
    const names =
['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'];
    return `${names[(parseInt(m, 10) || 1) - 1]} ${String(y).slice(-2)}`;
  }
}

function renderHeatmapDashboard(dataset) {
  const cardsEl = document.getElementById('hm-cards');
  const detailEl = document.getElementById('hm-card-detail');
  const coverageEl = document.getElementById('hm-coverage-watch');
  const domainEl = document.getElementById('hm-domain-summary');
  const trendEl = document.getElementById('hm-trend-summary');
  if (!cardsEl || !detailEl || !coverageEl || !domainEl || !trendEl) return;

  if (!dataset) {
    cardsEl.innerHTML = '';
    detailEl.innerHTML = '';
    detailEl.classList.add('hidden');
    coverageEl.innerHTML = '';
    coverageEl.classList.add('hidden');
    domainEl.innerHTML = '';
    trendEl.innerHTML = '';
    domainEl.classList.add('hidden');
    trendEl.classList.add('hidden');
    hmCardDetailKey = null;
    return;
  }

  if (hmCardDetailKey &&
!['met-rate', 'support', 'outcomes'].includes(hmCardDetailKey)) {
    hmCardDetailKey = null;
  }

  cardsEl.innerHTML = renderHeatmapStatCards(dataset);
  cardsEl.querySelectorAll('[data-hm-card]').forEach(btn => {
    btn.addEventListener('click', () => {
      const nextKey = btn.getAttribute('data-hm-card') || null;
      hmCardDetailKey = hmCardDetailKey === nextKey ? null : nextKey;
      renderHeatmapDashboard(dataset);
    });
  });
}

```

```

    });
  });
  renderHeatmapCardDetail(dataset, detailEl);
  renderHeatmapCoverageWatch(dataset.coverageRows || [], coverageEl);
  renderHeatmapDomainSummary(dataset.domainRows, domainEl);
  renderHeatmapTrendSummary(dataset.trendRows, trendEl);
}

function renderHeatmapStatCards(dataset) {
  const cards = [
    {
      key: 'met-rate',
      label: 'Overall met rate',
      value: `${dataset.metRate}%`,
      sub: `${dataset.overall.met} of ${dataset.overall.total} assessed outcomes
are met`,
    },
    {
      key: 'support',
      label: 'Students needing support',
      value: `${dataset.needsSupportCount}`,
      sub: `${dataset.assessedStudentCount} students with assessed data in this
view`,
    },
    {
      key: 'outcomes',
      label: 'Assessed outcomes',
      value: `${dataset.allOutcomeKeys.length}`,
      sub: `${dataset.domainRows.length}
domain${dataset.domainRows.length !== 1 ? 's' : ''} represented`
    }
  ];

  return cards.map(card => `
    <button type="button" class="hm-stat-card hm-stat-card-btn is-clickable
${hmCardDetailKey===card.key?'active':''}" data-hm-card="${card.key}">
      <div class="hm-stat-card-top">
        <div class="hm-stat-label">${card.label}</div>
        <div class="hm-stat-action">${hmCardDetailKey===card.key?'Hide
detail':'View detail'}</div>
      </div>
      <div class="hm-stat-value">${card.value}</div>
      <div class="hm-stat-sub">${card.sub}</div>
    </button>`).join('');
}

function renderHeatmapCardDetail(dataset, mountEl) {
  if (!hmCardDetailKey) {
    mountEl.innerHTML = '';
    mountEl.classList.add('hidden');
  }
}

```

```

    return;
  }

  mountEl.classList.remove('hidden');
  if (hmCardDetailKey === 'met-rate') {
    mountEl.innerHTML = renderHeatmapMetRateDetail(dataset);
  } else if (hmCardDetailKey === 'support') {
    mountEl.innerHTML = renderHeatmapSupportDetail(dataset);
  } else if (hmCardDetailKey === 'outcomes') {
    mountEl.innerHTML = renderHeatmapOutcomesDetail(dataset);
  } else {

    mountEl.innerHTML = '';
    mountEl.classList.add('hidden');
    return;
  }

  mountEl.querySelectorAll('[data-hm-filter-domain]').forEach(btn => {
    btn.addEventListener('click', () =>
      applyHeatmapDomainFilter(btn.getAttribute('data-hm-filter-domain') || ''));
  });
  mountEl.querySelectorAll('[data-hm-open-profile]').forEach(btn => {
    btn.addEventListener('click', () =>
      openProfile(btn.getAttribute('data-hm-open-profile') || ''));
  });
  mountEl.querySelectorAll('[data-hm-support-toggle]').forEach(btn => {
    btn.addEventListener('click', () => {
      hmSupportShowAll = (btn.getAttribute('data-hm-support-toggle') || '') ===
        'all';
      renderHeatmap();
    });
  });
}

function renderHeatmapMetRateDetail(dataset) {
  const weakestDomain = dataset.domainRows[0] || null;
  const domainRows = dataset.domainRows.slice().sort((a,b) => (b.metRate -
    a.metRate) || (b.total - a.total)).slice(0, 5);
  const weakestOutcomes = dataset.outcomeRows.slice().sort((a,b) => (b.supportRate
    - a.supportRate) || (b.total - a.total)).slice(0, 5);
  return `
    <div class="hm-card-detail-head">
      <div>
        <div class="hm-card-detail-title">Overall met rate breakdown</div>
        <div class="hm-card-detail-sub">A quick snapshot of what is going well and
where the wobble lives.</div>
      </div>
      <div class="hm-detail-actions">
        ${weakestDomain ? `<button type="button" class="hm-detail-btn"
data-hm-filter-domain="${escAttr(weakestDomain.domain)}">Filter to weakest

```



```

domain</button>` : ''}
  </div>
</div>
<div class="hm-chip-row">
  <span class="hm-chip met">${dataset.overall.met} Met</span>
  <span class="hm-chip wt">${dataset.overall.wt} Working Towards</span>
  <span class="hm-chip nm">${dataset.overall.nm} Not Met</span>
  <span class="hm-chip">${dataset.assessedStudentCount} students
assessed</span>
</div>
<div class="hm-detail-grid">
  <div class="hm-detail-section">
    <h4>Top domain performance</h4>
    <div class="hm-detail-list">
      ${domainRows.map(row => renderHeatmapDetailRow(row.domain,
`${Math.round(row.metRate*100)}% met · ${row.total} outcomes`, row.met, row.wt,
row.nm, row.total)).join('')} || '<div class="hm-empty-note">No domain data
yet.</div>'}
    </div>
  </div>
  <div class="hm-detail-section">
    <h4>Weakest outcomes right now</h4>
    <div class="hm-detail-list">
      ${weakestOutcomes.map(row => renderHeatmapDetailRow(row.info.fr,
row.info.domain, row.met, row.wt, row.nm, row.total)).join('')} || '<div
class="hm-empty-note">No outcome data yet.</div>'}
    </div>
  </div>
</div>`;
}

function renderHeatmapSupportDetail(dataset) {
  const flaggedRows = dataset.studentRows.filter(row => row.supportCount > 0);
  const visibleRows = hmSupportShowAll ? flaggedRows : flaggedRows.slice(0, 10);
  const hiddenCount = Math.max(flaggedRows.length - visibleRows.length, 0);
  const helperText = flaggedRows.length > 10
    ? (hmSupportShowAll
      ? `Showing all ${flaggedRows.length} flagged students in this filtered
view.`
      : `Showing the top ${visibleRows.length} of ${flaggedRows.length} flagged
students in this filtered view.`)
    : `${flaggedRows.length} flagged student${flaggedRows.length!==1?'s':''} in
this filtered view.`;
  return `
  <div class="hm-card-detail-head">
    <div>
      <div class="hm-card-detail-title">Students needing support</div>
      <div class="hm-card-detail-sub">Sorted by how many Working Towards / Not
Met outcomes they have in this filtered view.</div>
    </div>

```

```

    ${flaggedRows.length > 10 ? `<div class="hm-detail-actions">
      <button type="button" class="hm-detail-btn"
data-hm-support-toggle="${hmSupportShowAll ? 'top' : 'all'}">${hmSupportShowAll ?
'Show top 10' : 'Show all flagged students'}</button>
    </div>` : ''}
  </div>
  <div class="hm-chip-row">
    <span class="hm-chip wt">${dataset.needsSupportCount} students flagged</span>
    <span class="hm-chip">${dataset.studentRows.length} students with data</span>
    <span class="hm-chip">${dataset.allOutcomeKeys.length} outcomes in
play</span>
  </div>
  <div class="hm-detail-section">
    <h4>Support priority list</h4>
    <div class="hm-panel-sub" style="margin-bottom:10px">${helperText}</div>
    <div class="hm-detail-list">
      ${visibleRows.map(row => {
        const sub = [
          row.weakestDomain ? `Most affected domain: ${row.weakestDomain}` : '',
          row.lastConcernDate ? `Latest concern: ${row.lastConcernDate}` : ''
        ].filter(Boolean).join(' · ');
        return `
          <button type="button" class="hm-student-support is-clickable"
data-hm-open-profile="${escAttr(row.name)}" title="Open ${escAttr(row.name)}'s
profile">
            <div class="hm-detail-row-main">
              <div class="hm-detail-row-title">${esc(row.name)}</div>
              <div class="hm-detail-row-sub">${esc(sub || `${row.assessedCount}
assessed outcomes`)}</div>
            </div>
            <div class="hm-support-score">${row.supportCount} support
flag${row.supportCount!==1?'s':''}</div>
          </button>;
        }).join('') || '<div class="hm-empty-note">No students currently need
support in this filtered view.</div>'
      }
    </div>
    ${hiddenCount > 0 && !hmSupportShowAll ? `<div class="hm-panel-sub"
style="margin-top:10px">${hiddenCount} more flagged
student${hiddenCount!==1?'s':''} hidden. Use “Show all flagged students” to see the
rest.</div>` : ''}
  </div>`;
}

```

```

function renderHeatmapOutcomesDetail(dataset) {
  const mostUsed = dataset.outcomeRows.slice().sort((a,b) => (b.total - a.total) ||
(b.supportRate - a.supportRate)).slice(0, 8);
  return `
    <div class="hm-card-detail-head">
      <div>
        <div class="hm-card-detail-title">Assessed outcomes detail</div>

```

```

        <div class="hm-card-detail-sub">A tighter look at which outcomes are
        showing up most and how students are doing on them.</div>
    </div>
</div>
<div class="hm-chip-row">
    <span class="hm-chip">${dataset.allOutcomeKeys.length} distinct
    outcomes</span>
    <span class="hm-chip">${dataset.domainRows.length} domains represented</span>
    <span class="hm-chip">${dataset.overall.total} total outcomes</span>
</div>
<div class="hm-detail-section">
    <h4>Most assessed outcomes</h4>
    <div class="hm-detail-list">
        ${mostUsed.map(row => renderHeatmapDetailRow(row.info.fr,
        `${row.info.domain} · ${row.studentCount} student${row.studentCount!==1?'s':''}`,
        row.met, row.wt, row.nm, row.total)).join('') || '<div class="hm-empty-note">No
        outcomes recorded yet.</div>'}
    </div>
</div>`;
}

```

```

function renderHeatmapStrongestDomainDetail(dataset) {
    const strongest = dataset.strongestDomain;
    if (!strongest) {
        return `
        <div class="hm-card-detail-head">
            <div>
                <div class="hm-card-detail-title">Strongest domain</div>
                <div class="hm-card-detail-sub">Not enough data yet.</div>
            </div>
        </div>`;
    }
    const domainOutcomes = dataset.outcomeRows.filter(row => row.info.domain ===
    strongest.domain)
        .sort((a,b) => (b.metRate - a.metRate) || (b.total - a.total));
    const bestOutcome = domainOutcomes[0] || null;
    const toughestOutcome = domainOutcomes.slice().sort((a,b) => (b.supportRate -
    a.supportRate) || (b.total - a.total))[0] || null;
    return `
    <div class="hm-card-detail-head">
        <div>
            <div class="hm-card-detail-title">${esc(strongest.domain)} at a
            glance</div>
            <div class="hm-card-detail-sub">This is currently the strongest-performing
            domain in the filtered view.</div>
        </div>
        <div class="hm-detail-actions">
            <button type="button" class="hm-detail-btn"
            data-hm-filter-domain="${escAttr(strongest.domain)}">Filter heatmap to this
            domain</button>

```

```

    </div>
  </div>
  <div class="hm-chip-row">
    <span class="hm-chip met">${Math.round(strongest.metRate * 100)}% met</span>
    <span class="hm-chip">${strongest.outcomeCount} outcomes</span>
    <span class="hm-chip">${strongest.studentCount} students</span>
    <span class="hm-chip">${strongest.total} outcomes</span>
  </div>
  <div class="hm-detail-grid">
    <div class="hm-detail-section">
      <h4>Best outcome</h4>
      ${bestOutcome ? renderHeatmapDetailRow(bestOutcome.info.fr,
` ${bestOutcome.info.domain} · ${Math.round(bestOutcome.metRate*100)}% met`,
bestOutcome.met, bestOutcome.wt, bestOutcome.nm, bestOutcome.total) : '<div
class="hm-empty-note">No outcome data yet.</div>'}
    </div>
    <div class="hm-detail-section">
      <h4>Still worth watching</h4>
      ${toughestOutcome ? renderHeatmapDetailRow(toughestOutcome.info.fr,
` ${toughestOutcome.info.domain} ·
${Math.round((toughestOutcome.supportRate||0)*100)}% support flags`,
toughestOutcome.met, toughestOutcome.wt, toughestOutcome.nm, toughestOutcome.total)
: '<div class="hm-empty-note">No outcome data yet.</div>'}
    </div>
  </div>`;
}

```

```

function renderHeatmapDetailRow(title, sub, met, wt, nm, total) {
  const pMet = total ? (met / total) * 100 : 0;
  const pWt = total ? (wt / total) * 100 : 0;
  const pNm = total ? (nm / total) * 100 : 0;
  return `
    <div class="hm-detail-row">
      <div class="hm-detail-row-main">
        <div class="hm-detail-row-title">${esc(title)}</div>
        <div class="hm-detail-row-sub">${esc(sub || '')}</div>
      </div>
      <div class="hm-status-mini">
        <div class="hm-status-mini-track">
          ${pMet > 0 ? `<div class="hm-status-mini-seg met"
style="width:${pMet}%"></div>` : ''}
          ${pWt > 0 ? `<div class="hm-status-mini-seg wt"
style="width:${pWt}%"></div>` : ''}
          ${pNm > 0 ? `<div class="hm-status-mini-seg nm"
style="width:${pNm}%"></div>` : ''}
        </div>
        <div class="hm-status-mini-meta">${total} outcome${total!==1?'s':''}</div>
      </div>
    </div>`;
}

```

```

function renderHeatmapCoverageWatch(coverageRows, mountEl) {
  if (!coverageRows.length) {
    mountEl.classList.add('hidden');
    mountEl.innerHTML = '';
    return;
  }

  mountEl.classList.remove('hidden');
  const unusedCount = coverageRows.filter(row => row.count === 0).length;
  const mostUsedCount = Math.max(...coverageRows.map(row => row.count), 0);
  const domainGroups = coverageRows.reduce((domains, row) => {
    if (!domains[row.domain]) domains[row.domain] = { domain: row.domain, rows: [],
subdomains: {} };
    domains[row.domain].rows.push(row);
    if (!domains[row.domain].subdomains[row.subdomain]) {
      domains[row.domain].subdomains[row.subdomain] = { subdomain: row.subdomain,
rows: [] };
    }
    domains[row.domain].subdomains[row.subdomain].rows.push(row);
    return domains;
  }, {});
  const groups = Object.values(domainGroups);

  mountEl.innerHTML = `
    <details class="hm-coverage-panel" open>
      <summary>
        <div>
          <div class="hm-panel-title">Coverage watch</div>
          <div class="hm-panel-sub">Least-used learning outcomes rise to the top,
so ignored areas are easy to spot before favourites take over.</div>
        </div>
        <div class="hm-coverage-summary">
          <span class="hm-chip">${unusedCount} unused</span>
          <span class="hm-chip">${coverageRows.length} outcomes tracked</span>
          <span class="hm-chip">${mostUsedCount} max assessments</span>
          <span class="hm-coverage-arrow">v</span>
        </div>
      </summary>
      <div class="hm-coverage-groups">
        ${groups.map((group, groupIndex) => {
          const groupUnused = group.rows.filter(row => row.count === 0).length;
          const groupMax = Math.max(...group.rows.map(row => row.count), 0);
          const subgroups = Object.values(group.subdomains);
          return `
            <details class="hm-coverage-group" ${groupIndex === 0 ? 'open' : ''}>
              <summary>
                <div class="hm-coverage-group-title">
                  <span>${esc(group.domain)}</span>
                </div>

```

```

        <div class="hm-coverage-group-meta">
          <span>${groupUnused} unused</span>
          <span>${group.rows.length} outcomes</span>
          <span>${subgroups.length} subdomain${subgroups.length !== 1 ? 's' :
''}</span>
          <span>${groupMax} max assessments</span>
          <span class="hm-coverage-arrow">v</span>
        </div>
      </summary>
      <div class="hm-coverage-subgroups">
        ${subgroups.map((subgroup, subgroupIndex) => {
          const subgroupUnused = subgroup.rows.filter(row => row.count ===
0).length;
          const subgroupMax = Math.max(...subgroup.rows.map(row =>
row.count), 0);
          return `
            <details class="hm-coverage-subgroup" ${groupIndex === 0 &&
subgroupIndex === 0 ? 'open' : ''}>
              <summary>
                <div
class="hm-coverage-subgroup-title">${esc(subgroup.subdomain)}</div>
                <div class="hm-coverage-group-meta">
                  <span>${subgroupUnused} unused</span>
                  <span>${subgroup.rows.length} outcomes</span>
                  <span>${subgroupMax} max assessments</span>
                  <span class="hm-coverage-arrow">v</span>
                </div>
              </summary>
              <div class="hm-coverage-list">
                ${subgroup.rows.map(row => renderHeatmapCoverageRow(row,
mostUsedCount)).join('')}
              </div>
            </details>`;
          }).join('')}
        </div>
      </details>`;
    }).join('')}
  </div>
</details>`;
}

```

```

function renderHeatmapCoverageRow(row, maxCount) {
  const width = maxCount ? Math.max(4, Math.round((row.count / maxCount) * 100)) :
0;
  const status = row.count === 0 ? 'unused' : row.count >= Math.max(3, maxCount) ?
'heavy' : 'used';
  const label = row.count === 0 ? 'Not assessed yet' : `Assessed ${row.count}
time${row.count !== 1 ? 's' : ''}`;
  const meta = row.count === 0
    ? 'Top priority for coverage'

```

```

      : `${row.studentCount} student${row.studentCount !== 1 ? 's' : ''}
included${row.lastSeen ? ` · Last ${formatDate(row.lastSeen)}` : ''}`;
return `
  <div class="hm-coverage-row ${status}">
    <div class="hm-coverage-main">
      <div class="hm-coverage-outcome">${esc(row.outcome)}</div>
      <div class="hm-coverage-meta">${esc(meta)}</div>
    </div>
    <div class="hm-coverage-score">
      <div class="hm-coverage-count">${esc(label)}</div>
      <div class="hm-coverage-track">
        ${width > 0 ? `<div class="hm-coverage-fill"
style="width:${width}%"></div>` : ''}
      </div>
    </div>
  </div>`;
}

function renderHeatmapDomainSummary(domainRows, mountEl) {
  if (!domainRows.length) {
    mountEl.classList.add('hidden');
    mountEl.innerHTML = '';
    return;
  }
  mountEl.classList.remove('hidden');
  mountEl.innerHTML = `
    <div class="hm-panel-head">
      <div>
        <div class="hm-panel-title">Domain summary</div>
        <div class="hm-panel-sub">Stacked bars show the balance of Met, Working
Towards and Not Met across each domain.</div>
      </div>
    </div>
    <div class="hm-domain-rows">
      ${domainRows.map(row => {
        const pMet = row.total ? (row.met / row.total) * 100 : 0;
        const pWt = row.total ? (row.wt / row.total) * 100 : 0;
        const pNm = row.total ? (row.nm / row.total) * 100 : 0;
        return `
          <div class="hm-domain-row">
            <div>
              <div class="hm-domain-name">${esc(row.domain)}</div>
              <div class="hm-domain-meta">${row.outcomeCount}
outcome${row.outcomeCount!==1?'s':''} · ${row.studentCount}
student${row.studentCount!==1?'s':''}</div>
            </div>
            <div class="hm-domain-bar-track">
              ${pMet > 0 ? `<div class="hm-domain-bar-seg met"
style="width:${pMet}%"></div>` : ''}
              ${pWt > 0 ? `<div class="hm-domain-bar-seg wt"

```

```

style="width:${pWt}%"></div>` : ''}
    ${pNm > 0 ? `<div class="hm-domain-bar-seg nm"
style="width:${pNm}%"></div>` : ''}
    </div>
    <div class="hm-domain-total">${row.total} outcomes</div>
  </div>`;
  }).join('')}
</div>
<div class="hm-legend">
  <span class="hm-legend-chip"><span class="hm-legend-dot
met"></span>Met</span>
  <span class="hm-legend-chip"><span class="hm-legend-dot wt"></span>Working
Towards</span>
  <span class="hm-legend-chip"><span class="hm-legend-dot nm"></span>Not
Met</span>
</div>`;
}

function renderHeatmapTrendSummary(trendRows, mountEl) {
  if (!trendRows.length) {
    mountEl.classList.add('hidden');
    mountEl.innerHTML = '';
    return;
  }
  mountEl.classList.remove('hidden');
  const maxTotal = Math.max(...trendRows.map(row => row.total), 1);
  mountEl.innerHTML = `
    <div class="hm-panel-head">
      <div>
        <div class="hm-panel-title">Time trends</div>
        <div class="hm-panel-sub">Each column shows outcomes over time, with colour
showing the mix of results.</div>
      </div>
    </div>
    <div class="hm-trend-wrap">
      <div class="hm-trend-chart">
        <div class="hm-trend-bars">
          ${trendRows.map(row => {
            const heightPct = Math.max(8, Math.round((row.total / maxTotal) *
100));
            const segMet = row.total ? (row.met / row.total) * 100 : 0;
            const segWt = row.total ? (row.wt / row.total) * 100 : 0;
            const segNm = row.total ? (row.nm / row.total) * 100 : 0;
            return `
              <div class="hm-trend-col" title="${esc(row.label)}: ${row.total}
outcomes">
                <div class="hm-trend-bar-area">
                  <div class="hm-trend-bar" style="height:${heightPct}%">
                    ${segMet > 0 ? `<div class="hm-trend-seg met"
style="height:${segMet}%"></div>` : ''}

```



```

        ${segWt > 0 ? `<div class="hm-trend-seg wt"
style="height:${segWt}%"></div>` : ''}
        ${segNm > 0 ? `<div class="hm-trend-seg nm"
style="height:${segNm}%"></div>` : ''}
        </div>
    </div>
    <div class="hm-trend-total">${row.total}</div>
    <div class="hm-trend-label">${esc(row.label)}</div>
    </div>`;
    }).join('')}
</div>
</div>
</div>
<div class="hm-legend">
    <span class="hm-legend-chip"><span class="hm-legend-dot
met"></span>Met</span>
    <span class="hm-legend-chip"><span class="hm-legend-dot wt"></span>Working
Towards</span>
    <span class="hm-legend-chip"><span class="hm-legend-dot nm"></span>Not
Met</span>
</div>`;
}

```

```

function applyHeatmapDomainFilter(domain) {
    const filterEl = document.getElementById('hm-domain-filter');
    if (!filterEl) return;
    filterEl.value = domain || '';
    renderHeatmap();
}

```

```

function renderHeatmapGrid(students, domainOutcomes, outcomeInfo,
studentOutcomeMap) {

```

```

    const domains = Object.keys(domainOutcomes).sort();

```

```

    const studentHeaders = students.map(st => {
        const init = st.name.trim()[0]?.toUpperCase() || '?';
        const shortName = st.name.split(' ')[0];
        return `<th class="col-student">
            <div class="st-init">${init}</div>
            <div class="st-name" title="${esc(st.name)}">${esc(shortName)}</div>
        </th>`;
    }).join('');

```

```

    let bodyRows = '';
    domains.forEach(domain => {
        const keys = domainOutcomes[domain];
        bodyRows += `<tr class="domain-group-row"><td colspan="${students.length +
1}">${esc(domain)}</td></tr>`;
        keys.forEach(key => {
            const info = outcomeInfo[key];
            const cells = students.map(st => {

```

```

        const status = studentOutcomeMap[st.name]?.[key] || 'none';
        const cls = status === 'met' ? 'c-met' : status === 'wt' ? 'c-wt' : status
=== 'nm' ? 'c-nm' : 'c-none';
        const tip = status === 'met' ? 'Met' : status === 'wt' ? 'Working Towards'
: status === 'nm' ? 'Not Met' : '-';
        return `  |
```

```

function renderHeatmapList(keys, outcomeInfo, studentOutcomeMap, students) {
    const rows = keys.map(key => {
        const info = outcomeInfo[key];
        let met = 0, wt = 0, nm = 0;
        const stuSet = new Set();
        students.forEach(st => {
            const s = studentOutcomeMap[st.name]?.[key];
            if (s === 'met') { met++; stuSet.add(st.name); }
            else if (s === 'wt') { wt++; stuSet.add(st.name); }
            else if (s === 'nm') { nm++; stuSet.add(st.name); }
        });
    });
}

```

```

    });
    const total = met + wt + nm;
    if (!total) return null;
    const pMet = Math.round(met/total*100), pWt = Math.round(wt/total*100), pNm =
Math.round(nm/total*100);
    const challScore = (wt + nm) / total;
    return { info, met, wt, nm, total, pMet, pWt, pNm, challScore, studentCount:
stuSet.size };
  }).filter(Boolean).sort((a,b) => b.challScore - a.challScore);

```

```

document.getElementById('heatmap-list').innerHTML = rows.map(o => `
  <div class="outcome-hm-row">
    <div class="hm-outcome-fr">${esc(getOutcomeDisplayPrimary(o.info))}</div>
    ${getOutcomeDisplaySecondary(o.info) ? `<div
class="hm-outcome-en">${esc(getOutcomeDisplaySecondary(o.info))}</div>` : ''}
    <div class="hm-bar-wrap">
      ${o.pMet>0?`<div class="hm-bar-seg hm-bar-met"
style="width:${o.pMet}%"></div>`:''}
      ${o.pWt >0?`<div class="hm-bar-seg hm-bar-wt"
style="width:${o.pWt}%"></div>`:''}
      ${o.pNm >0?`<div class="hm-bar-seg hm-bar-nm"
style="width:${o.pNm}%"></div>`:''}
    </div>
    <div class="hm-counts">
      ${o.met>0?`<span class="hm-count met">${o.met} met</span>`:''}
      ${o.wt >0?`<span class="hm-count wt">${o.wt} working towards</span>`:''}
      ${o.nm >0?`<span class="hm-count nm">${o.nm} not met</span>`:''}
      <span class="hm-students">${o.studentCount}
student${o.studentCount!==1?'s':''}</span>
    </div>
  </div>`).join('');
}

```

```

/* =====
  SETTINGS
===== */

```

```

/* =====
  BATCH RECORD MODE
===== */

```

```

let batchInputType = 'text';
let rosterClassFilter = 'all'; // 'all' | class name
let batchPhotoData = null;    // {base64, mimeType}
let batchPDFData    = null;    // {base64}
let batchContext     = null;    // {domain, outcomes:[{fr,en}], description}
let batchStudentStatuses = {}; // name -> { outcomeStatuses:[], comment, absent }

```

```

function switchMode(mode) {
  // Tabs are now separate – delegate to goTab
  if (mode === 'assess') goTab('assess');
}

```

```

    else goTab('record');
}

function setInputType(type) {
    batchInputType = type;
    ['text', 'photo', 'pdf'].forEach(t => {
        document.getElementById('itype-' + t).classList.toggle('active', t === type);
        document.getElementById('binput-' + t).style.display = t === type ? 'block' :
        'none';
    });
}

function handleBatchPhoto(e) {
    const file = e.target.files[0]; if (!file) return;
    const reader = new FileReader();
    reader.onload = ev => {
        const img = new Image();
        img.onload = () => {
            const MAX = 1600;
            let {width: w, height: h} = img;
            if (w > MAX || h > MAX) {
                if (w > h) { h = Math.round(h * MAX / w); w = MAX; }
                else { w = Math.round(w * MAX / h); h = MAX; }
            }
            const c = document.createElement('canvas');
            c.width = w; c.height = h;
            c.getContext('2d').drawImage(img, 0, 0, w, h);
            const dataUrl = c.toDataURL('image/jpeg', 0.82);
            batchPhotoData = { base64: dataUrl.split(',')[1], mimeType: 'image/jpeg' };
            const prev = document.getElementById('batch-photo-preview');
            prev.src = dataUrl; prev.style.display = 'block';
        };
        img.src = ev.target.result;
    };
    reader.readAsDataURL(file);
}

function handleBatchPDF(e) {
    const file = e.target.files[0]; if (!file) return;
    const reader = new FileReader();
    reader.onload = ev => {
        batchPDFData = { base64: ev.target.result.split(',')[1] };
        document.getElementById('batch-pdf-name').textContent = '✓ ' + file.name;
    };
    reader.readAsDataURL(file);
}

function onBatchDomainChange() {
    const domain = document.getElementById('batch-domain-select').value;
    const picker = document.getElementById('batch-outcome-picker');

```

```

    if (!domain) { picker.classList.remove('visible'); return; }
    picker.classList.add('visible');
    renderOutcomeOptions(domain, 'batch-outcome-opts', 'bopt',
'toggleBatchOutcomeOpt');
    document.getElementById('batch-outcome-ai').checked = true;
    onBatchOutcomeModeChange();
}

function toggleBatchOutcomeOpt(optionId) {
    document.getElementById('batch-outcome-ai').checked = false;
    document.getElementById('batch-outcome-custom').checked = false;
    document.getElementById('batch-custom-outcome-wrap').style.display = 'none';
    const el = document.getElementById(optionId);
    if (!el) return;
    el.classList.toggle('selected');
    el.querySelector('.outcome-opt-check').textContent =
el.classList.contains('selected') ? '✓' : '';
}

function onBatchOutcomeModeChange() {
    const mode =
document.querySelector('input[name="batch-outcome-mode"]:checked').value;
    document.getElementById('batch-custom-outcome-wrap').style.display = mode ===
'custom' ? 'block' : 'none';
    if (mode === 'ai' || mode === 'custom') {
        document.querySelectorAll('#batch-outcome-opts .outcome-opt').forEach(el => {
            el.classList.remove('selected');
            el.querySelector('.outcome-opt-check').textContent = '';
        });
    }
}

function getBatchChosenOutcomes() {
    const domain = document.getElementById('batch-domain-select').value;
    if (!domain) return { domain: '', outcomes: [], mode: 'ai' };
    const mode =
document.querySelector('input[name="batch-outcome-mode"]:checked').value || 'ai';
    if (mode === 'custom') {
        const txt = document.getElementById('batch-custom-outcome-text').value.trim();
        return { domain, outcomes: txt ? [txt] : [], mode: 'custom' };
    }
    const selected = [...document.querySelectorAll('#batch-outcome-opts
.outcome-opt.selected')]
        .map(el => el.dataset.value ||
el.querySelector('.outcome-opt-text').textContent || '');
    if (selected.length > 0) return { domain, outcomes: selected, mode: 'selected' };
    return { domain, outcomes: [], mode: 'ai' };
}

async function runBatchIdentify() {

```

```

const { domain: chosenDomain, outcomes: chosenOutcomes, mode: outcomeMode } =
getBatchChosenOutcomes();

if (chosenDomain && outcomeMode === 'selected' && chosenOutcomes.length > 0) {
  batchContext = {
    domain: chosenDomain,
    outcomes: chosenOutcomes.map(o => ({ fr: o, en: o })),
    description: getActivityDescription()
  };
  showBatchGrid();
  return;
}

if (chosenDomain && outcomeMode === 'custom' && chosenOutcomes.length > 0) {
  batchContext = {
    domain: chosenDomain,
    outcomes: [{ fr: chosenOutcomes[0], en: chosenOutcomes[0] }],
    description: getActivityDescription()
  };
  showBatchGrid();
  return;
}

const desc = getActivityDescription();
if (!desc && batchInputType === 'text') { flash('Please describe the activity
first.');
```

first.');

```

  return; }
if (batchInputType === 'photo' && !batchPhotoData) { flash('Please upload a photo
first.');
```

first.');

```

  return; }
if (batchInputType === 'pdf' && !batchPDFData) { flash('Please upload a PDF
first.');
```

first.');

```

  return; }

document.getElementById('batch-identify-btn').textContent = 'Identifying...';
document.getElementById('batch-identify-btn').disabled = true;

try {
  let sourceContent = [];
  if (batchInputType === 'photo' && batchPhotoData) {
    sourceContent.push({ type: 'image', source: { type: 'base64', media_type:
batchPhotoData.mimeType, data: batchPhotoData.base64 } });
  } else if (batchInputType === 'pdf' && batchPDFData) {
    sourceContent.push({ type: 'document', source: { type: 'base64', media_type:
'application/pdf', data: batchPDFData.base64 } });
  }
  if (batchInputType === 'text' && desc) {
    sourceContent.push({ type: 'text', text: `Teacher description of the
activity: ${desc}` });
  }

  const includePdfBeta = batchInputType === 'pdf';
  const resolvedDomain = chosenDomain || await identifyApprovedDomain({

```

```

    messageContent: sourceContent,
    timeoutMs: 45000,
    includePdfBeta,
    activityDescription: desc
  });

```

const outcomeInstruction = `Use ONLY approved learning outcomes from the app's curriculum library for "\${resolvedDomain}". Choose 1-3 outcomes that best fit this activity. Prefer 1-2 outcomes. Only choose 3 if the activity clearly and substantially addresses three distinct approved outcomes. Copy the wording EXACTLY from the library. Do NOT invent, paraphrase, merge, translate or adapt outcomes. If the fit is imperfect, choose the nearest approved outcome rather than making one up.

Approved library for \${resolvedDomain}:
 \${buildOutcomeLibraryPrompt(resolvedDomain)}`;

const prompt = `You are an experienced teacher at an international school following the French curriculum for \${getAssessCycleContext()}.

\${batchInputType === 'text' ? 'The teacher has described an activity.' : 'A photo or document of the activity/worksheet is attached.'}
 The domain is fixed as: \${resolvedDomain}. Use this exact domain. The approved in-app curriculum library is the source of truth.
 \${outcomeInstruction}

Return between 1 and 3 outcomes only. Prefer 1-2 unless 3 are clearly justified by the activity.

Respond with ONLY valid JSON – no markdown, no backticks:

```

{
  "domain": "${resolvedDomain}",
  "outcomes": [
    { "fr": "<exact approved outcome wording>", "en": "<same wording>", "status":
"wt" }
  ]
}
`;

```

```

const result = await callAnthropicJSON({
  messageContent: [...sourceContent, { type: 'text', text: prompt }],
  maxTokens: 700,
  timeoutMs: 60000,
  includePdfBeta
});

```

```

let approvedOutcomes = enforceApprovedOutcomes(resolvedDomain, result.outcomes
|| []);
if (outcomeMode === 'ai') approvedOutcomes = approvedOutcomes.slice(0, 3);
if (!approvedOutcomes.length) {
  throw new Error('AI did not return outcomes from the approved curriculum

```

```

library.');
```

```

    batchContext = { domain: resolvedDomain, outcomes: approvedOutcomes,
description: desc };
    showBatchGrid();
  } catch(err) {
    flash('Could not identify outcomes: ' + err.message);
  } finally {
    document.getElementById('batch-identify-btn').textContent = 'Next: set up class
grid →';
    document.getElementById('batch-identify-btn').disabled = false;
  }
}

function getActivityDescription() {
  const name = document.getElementById('batch-name')?.value.trim();
  if (name) return name;
  if (batchInputType === 'text') return
document.getElementById('batch-desc').value.trim().slice(0, 60) || 'Activity';
  if (batchInputType === 'photo') return 'Activity photo';
  if (batchInputType === 'pdf') return
document.getElementById('batch-pdf-name').textContent.replace('✓ ', '') || 'PDF
worksheet';
  return 'Activity';
}

function renderBatchOutcomes() {
  document.getElementById('batch-outcomes-display').innerHTML =
batchContext.outcomes.map((o, i) =>
  `<div
style="display:flex;align-items:flex-start;gap:8px;font-size:12.5px;padding:7px
10px;background:var(--accent-lt);border:1px solid
var(--accent-bd);border-radius:7px">
    <div style="flex:1">

<strong>${esc(getOutcomeDisplayPrimary(o))}</strong>${getOutcomeDisplaySecondary(o)
? `<br><span
style="font-size:11.5px;color:var(--ink-soft);font-style:italic">${esc(getOutcomeDi
splaySecondary(o))}</span>` : ''}
    </div>
    <button onclick="removeBatchOutcome(${i})" title="Remove this outcome"
style="background:none;border:none;cursor:pointer;font-size:14px;color:var(--ink-so
ft);padding:0;flex-shrink:0;line-height:1"
onmouseover="this.style.color='var(--nm)'"
onmouseout="this.style.color='var(--ink-soft)'">X</button>
    </div>`
  ).join('');
}

```



```

function removeBatchOutcome(i) {
  batchContext.outcomes.splice(i, 1);
  if (!batchContext.outcomes.length) {
    flash('You need at least one outcome. Add one manually or go back to
re-identify.');
```

```

    batchContext.outcomes = [{ fr: '', en: '' }];
  }
  Object.keys(batchStudentStatuses).forEach(name => {
    const st = batchStudentStatuses[name];
    if (Array.isArray(st?.outcomeStatuses)) st.outcomeStatuses.splice(i, 1);
    rerenderBatchSplitBox(name);
    syncBatchRowUI(name);
  });
  renderBatchOutcomes();
}

function batchKey(name) {
  return encodeURIComponent(String(name || ''));
}

function jsq(value) {
  return JSON.stringify(String(value ?? ''));
}

function createEmptyBatchOutcomeStatuses() {
  return (batchContext?.outcomes || []).map(() => null);
}

function createBatchStudentState() {
  return { outcomeStatuses: createEmptyBatchOutcomeStatuses(), comment: '', absent:
false, assessmentId: '' };
}

function getBatchFilledOutcomeCount(state) {
  return (state?.outcomeStatuses || []).filter(Boolean).length;
}

function getBatchAggregateStatus(state) {
  const total = (batchContext?.outcomes || []).length;
  const values = (state?.outcomeStatuses || []).filter(Boolean);
  if (!values.length) return null;
  if (values.length < total) return '__partial__';
  return values.every(v => v === values[0]) ? values[0] : '__mixed__';
}

function renderBatchSplitRowsHTML(name) {
  const state = batchStudentStatuses[name] || createBatchStudentState();
  return (batchContext?.outcomes || []).map((o, i) => `
    <div class="cg-split-row">
```

```

    <div class="cg-split-title">

    <strong>${esc(getOutcomeDisplayPrimary(o))}</strong>${getOutcomeDisplaySecondary(o)
    ? `<span>${esc(getOutcomeDisplaySecondary(o))}</span>` : ''}
    </div>
    <div class="cg-split-actions">
        <button id="cgmini-met-${batchKey(name)}-${i}"
class="cg-mini-status${state.outcomeStatuses?.[i] === 'met' ? ' active-met' : ''}"
onclick='setBatchOutcomeStatus(${jsq(name)}, ${i}, "met")' title="Met">✓</button>
        <button id="cgmini-wt-${batchKey(name)}-${i}"
class="cg-mini-status${state.outcomeStatuses?.[i] === 'wt' ? ' active-wt' : ''}"
onclick='setBatchOutcomeStatus(${jsq(name)}, ${i}, "wt")' title="Working
Towards">~</button>
        <button id="cgmini-nm-${batchKey(name)}-${i}"
class="cg-mini-status${state.outcomeStatuses?.[i] === 'nm' ? ' active-nm' : ''}"
onclick='setBatchOutcomeStatus(${jsq(name)}, ${i}, "nm")' title="Not
Met">X</button>
    </div>
    </div>`).join('');
}

```

```

function renderBatchStudentRow(name, opts = {}) {
    const { isNew = false } = opts;
    const key = batchKey(name);
    const state = batchStudentStatuses[name] || createBatchStudentState();
    const total = (batchContext?.outcomes || []).length;
    const filled = getBatchFilledOutcomeCount(state);
    const metaLabel = total ? `${filled}/${total} outcomes set${state.assessmentId ?
    ' · saved record' : ''}` : 'No outcomes';
    return `<div class="cg-row" id="cgrow-${key}">
        <div class="cg-main">
            <div class="cg-name">${esc(name)}${isNew ? ` <span
style="font-size:10px;color:var(--accent);font-weight:600">new</span>` : ''}<span
class="cg-name-meta">${metaLabel}</span></div>
            <button class="cg-status-btn" id="cgbtn-met-${key}"
onclick='setCGStatus(${jsq(name)}, "met")' title="Set all selected outcomes to
Met">✓</button>
            <button class="cg-status-btn" id="cgbtn-wt-${key}"
onclick='setCGStatus(${jsq(name)}, "wt")' title="Set all selected outcomes to
Working Towards">~</button>
            <button class="cg-status-btn" id="cgbtn-nm-${key}"
onclick='setCGStatus(${jsq(name)}, "nm")' title="Set all selected outcomes to Not
Met">X</button>
            <div class="cg-extras">
                <button class="cg-split-toggle" id="cgsplitbtn-${key}"
onclick='toggleCGSplit(${jsq(name)})' title="Edit each selected
outcome">Split</button>
                <button class="cg-comment-toggle" onclick='toggleCGComment(${jsq(name)})'
title="Add comment">💬</button>
                <button class="cg-absent-toggle" id="cgabs-${key}"

```

```

onclick='toggleAbsent(${jsq(name)})' >Absent</button>
    </div>
</div>
<div class="cg-split-box"
id="cgsplit-${key}">${renderBatchSplitRowsHTML(name)}</div>
    <div class="cg-comment-box" id="cgcmnt-${key}">
        <textarea class="cg-comment-input" placeholder="Optional comment for
${esc(name)}..." onchange='batchStudentStatuses[${jsq(name)}].comment = this.value'
></textarea>
    </div>
</div>`;
}

function rerenderBatchSplitBox(name) {
    const key = batchKey(name);
    const box = document.getElementById('cgsplit-' + key);
    if (box) box.innerHTML = renderBatchSplitRowsHTML(name);
}

function syncBatchRowUI(name) {
    const state = batchStudentStatuses[name];
    if (!state) return;
    const key = batchKey(name);
    const total = (batchContext?.outcomes || []).length;
    const filled = getBatchFilledOutcomeCount(state);
    const aggregate = getBatchAggregateStatus(state);

    ['met', 'wt', 'nm'].forEach(s => {
        const btn = document.getElementById(`cgbtn-${s}-${key}`);
        if (btn) btn.className = 'cg-status-btn' + (aggregate === s ? ' active-' + s :
''');
    });

    (batchContext?.outcomes || []).forEach((_, i) => {
        ['met', 'wt', 'nm'].forEach(s => {
            const btn = document.getElementById(`cgmini-${s}-${key}-${i}`);
            if (btn) btn.className = 'cg-mini-status' + (state.outcomeStatuses?.[i] === s
? ' active-' + s : '');
        });
    });

    const row = document.getElementById('cgrow-' + key);
    if (row) {
        row.classList.toggle('mixed', aggregate === '__mixed__');
        row.classList.toggle('partial', aggregate === '__partial__');
        row.classList.toggle('absent', !!state.absent);
    }

    const meta = row?.querySelector('.cg-name-meta');
    if (meta) meta.textContent = total ? `${filled}/${total} outcomes set` : 'No

```

```
outcomes';
```

```
const splitBtn = document.getElementById('cgsplitbtn-' + key);
if (splitBtn) {
  const splitBox = document.getElementById('cgsplit-' + key);
  const splitOpen = isElementOpen(splitBox);
  splitBtn.textContent = splitOpen ? 'Hide split' : (filled && filled < total ?
`Split ${filled}/${total}` : `Split ${total}`);
  splitBtn.classList.toggle('active', splitOpen || aggregate === '__mixed__' ||
aggregate === '__partial__');
}
```

```
const absentBtn = document.getElementById('cgabs-' + key);
if (absentBtn) {
  absentBtn.textContent = state.absent ? 'Present' : 'Absent';
  absentBtn.style.color = state.absent ? 'var(--met)' : '';
  absentBtn.style.borderColor = state.absent ? 'var(--met-bd)' : '';
}
}
```

```
function applyStudentPreset(name, status) {
  const state = batchStudentStatuses[name];
  if (!state || state.absent) return;
  state.outcomeStatuses = createEmptyBatchOutcomeStatuses().map(() => status);
  rerenderBatchSplitBox(name);
  syncBatchRowUI(name);
}
```

```
function setBatchOutcomeStatus(name, index, status) {
  const state = batchStudentStatuses[name];
  if (!state || state.absent) return;
  state.outcomeStatuses[index] = status;
  syncBatchRowUI(name);
}
```

```
function showBatchGrid(options = {}) {
  const { preserveStatuses = false, forcedStudents = null, disableClassFilter =
false } = options;
  const namedStudent = document.getElementById('batch-student-name')?.value.trim();
  const bannerEl = document.getElementById('batch-edit-banner');
  const saveBtn = document.getElementById('batch-save-btn');

  document.getElementById('batch-activity-summary').textContent =
batchContext.description || '';
  document.getElementById('batch-domain-chip').textContent = '📖 ' +
batchContext.domain;
  const warnEl = document.getElementById('batch-domain-warn');
  warnEl.style.display = batchContext.domain === 'Modern Languages' ? 'flex' :
'none';
  renderBatchOutcomes();
}
```

```

if (!preserveStatuses) batchStudentStatuses = {};
const allStudents = DB.getAllStudents();
const classes = DB.getClasses();
let students = forcedStudents ? forcedStudents.slice() : allStudents;
if (!forcedStudents && classes.length && rosterClassFilter !== 'all' &&
rosterClassFilter !== '__none__') {
  students = allStudents.filter(s => s.class === rosterClassFilter);
  if (!students.length) students = allStudents;
}

const rows = document.getElementById('class-grid-rows');
const noStu = document.getElementById('batch-no-students');
if (!students.length) {
  noStu.style.display = 'block';
  rows.innerHTML = '';
  showBatchStep('grid');
  return;
}
noStu.style.display = 'none';

if (!forcedStudents && namedStudent && !students.find(s => s.name ===
namedStudent)) {
  students = [...students, { name: namedStudent, assessments: [], _isNew: true
}];
}

const existingFilter = document.getElementById('batch-class-filter-row');
if (existingFilter) existingFilter.remove();
if (!forcedStudents && !disableClassFilter && classes.length) {
  const cgFilter = document.createElement('div');
  cgFilter.id = 'batch-class-filter-row';
  cgFilter.style.cssText =
'display:flex;gap:6px;flex-wrap:wrap;margin-bottom:10px';
  cgFilter.innerHTML = ['all', ...classes].map(c => {
    const label = c === 'all' ? `All (${allStudents.length})` : `${c}
(${allStudents.filter(s => s.class === c).length})`;
    const active = (c === 'all' && rosterClassFilter === 'all') || c ===
rosterClassFilter;
    return `<button class="class-tab ${active ? 'active' : ''}"
onclick='setBatchClassFilter(${jsq(c)})' >${label}</button>`;
  }).join('');
  rows.parentNode.insertBefore(cgFilter, rows);
}

rows.innerHTML = students.map(st => {
  if (!batchStudentStatuses[st.name]) batchStudentStatuses[st.name] =
createBatchStudentState();
  return renderBatchStudentRow(st.name, { isNew: !!st._isNew });
}).join('');

```

```

Object.keys(batchStudentStatuses).forEach(name => {
  const commentBox = document.getElementById('cgcmt-' + batchKey(name));
  const commentInput = commentBox ? commentBox.querySelector('textarea') : null;
  if (commentInput) commentInput.value = batchStudentStatuses[name]?.comment ||
'';
  syncBatchRowUI(name);
});

if (batchEditingSessionId) {
  if (bannerEl) {
    const savedCount = Object.values(batchStudentStatuses).filter(state =>
state.assessmentId).length;
    bannerEl.textContent = `Editing saved batch grid · ${savedCount} student
record${savedCount !== 1 ? 's' : ''}. Save changes here to update the whole class
grid faster.`;
    bannerEl.style.display = 'block';
  }
  if (saveBtn) saveBtn.textContent = '✓ Save batch changes';
} else {
  if (bannerEl) {
    bannerEl.textContent = '';
    bannerEl.style.display = 'none';
  }
  if (saveBtn) saveBtn.textContent = '✓ Save activity record';
}

showBatchStep('grid');
}

function setCGStatus(name, status) {
  applyStudentPreset(name, status);
}

function setElementOpenState(el, open, displayValue = 'block') {
  if (!el) return false;
  el.classList.toggle('open', !!open);
  el.style.display = open ? displayValue : 'none';
  return !!open;
}

function isElementOpen(el) {
  if (!el) return false;
  return el.classList.contains('open') || el.style.display === 'block';
}

function toggleCGSplit(name) {
  const key = batchKey(name);
  const box = document.getElementById('cgsplit-' + key);
  if (!box) return;

```

```

    if (!box.innerHTML.trim()) rerenderBatchSplitBox(name);
    const nowOpen = !isElementOpen(box);
    setElementOpenState(box, nowOpen, 'block');
    const btn = document.getElementById('cgsplitbtn-' + key);
    if (btn) {
        btn.classList.toggle('active', nowOpen ||
    ['__mixed__', '__partial__'].includes(getBatchAggregateStatus(batchStudentStatuses[name])));
        btn.textContent = nowOpen ? 'Hide split' : (() => {
            const state = batchStudentStatuses[name] || createBatchStudentState();
            const total = (batchContext?.outcomes || []).length;
            const filled = getBatchFilledOutcomeCount(state);
            return filled && filled < total ? `Split ${filled}/${total}` : `Split
    ${total}`;
        })();
    }
}

```

```

function toggleCGComment(name) {
    const key = batchKey(name);
    const box = document.getElementById('cgcmt-' + key);
    if (!box) return;
    const nowOpen = !isElementOpen(box);
    setElementOpenState(box, nowOpen, 'block');
}

```

```

function toggleAbsent(name) {
    const state = batchStudentStatuses[name];
    if (!state) return;
    state.absent = !state.absent;
    if (state.absent) state.outcomeStatuses = createEmptyBatchOutcomeStatuses();
    rerenderBatchSplitBox(name);
    syncBatchRowUI(name);
}

```

```

function quickFillAll(status) {
    Object.keys(batchStudentStatuses).forEach(name => {
        if (!batchStudentStatuses[name].absent) applyStudentPreset(name, status);
    });
}

```

```

function setBatchClassFilter(cls) {
    rosterClassFilter = cls;
    // Rebuild the grid with new filter
    if (batchContext) showBatchGrid();
}

```

```

function addStudentToBatchGrid() {
    const input = document.getElementById('batch-new-student');
}

```

```

const name = input.value.trim();
if (!name) return;
if (batchStudentStatuses[name]) { flash(name + ' is already in the grid.');
```

```
return; }
```

```

batchStudentStatuses[name] = createBatchStudentState();
const rows = document.getElementById('class-grid-rows');
rows.insertAdjacentHTML('beforeend', renderBatchStudentRow(name, { isNew: true
}));
rerenderBatchSplitBox(name);
syncBatchRowUI(name);
document.getElementById('batch-no-students').style.display = 'none';
input.value = '';
input.focus();
}

```

```

function saveBatchRecord() {
  const today = new Date().toISOString().split('T')[0];
  const cycleSnapshot = getAssessCycleSnapshot();
  const totalOutcomes = (batchContext?.outcomes || []).length;
  const activeRows = Object.entries(batchStudentStatuses).filter(([ , v]) =>
!v.absent);
  const partial = [];
  const toSave = [];

  activeRows.forEach(([name, v]) => {
    const filled = getBatchFilledOutcomeCount(v);
    if (!filled) return;
    if (filled < totalOutcomes) {
      partial.push(name);
      return;
    }
    toSave.push([name, v]);
  });

  if (partial.length) {
    flash('Complete the outcome split for: ' + partial.slice(0, 4).join(', ') +
(partial.length > 4 ? '...' : ''), 'warn');
    return;
  }

  if (!toSave.length) { flash('No students recorded – please set at least one
status preset or split outcome statuses.');
```

```
return; }
```

```

const data = DB.loadData();
if (!data.sessions) data.sessions = [];
let session = batchEditingSessionId ? (data.sessions || []).find(entry =>
entry.id === batchEditingSessionId) : null;
let createdCount = 0;
let updatedCount = 0;

```



```

    if (!session) {
      session = { id: Date.now().toString(36), date: today, type: 'batch', activity:
batchContext.description, ...cycleSnapshot, students: [] };
      data.sessions.unshift(session);
    } else {
      session.activity = batchContext.description;
      Object.assign(session, cycleSnapshot);
    }

    toSave.forEach(([name, v]) => {
      if (!data.students[name]) data.students[name] = { name, assessments: [] };
      if (rosterClassFilter !== 'all' && rosterClassFilter !== '__none__' &&
!data.students[name].class) data.students[name].class = rosterClassFilter;
      const student = data.students[name];
      const nextOutcomes = batchContext.outcomes.map((o, i) => ({ fr: o.fr, en: o.en,
status: v.outcomeStatuses[i] }));

      if (v.assessmentId) {
        const existing = (student.assessments || []).find(item => item.id ===
v.assessmentId);
        if (existing) {
          existing.type = 'batch';
          Object.assign(existing, cycleSnapshot);
          existing.domain = batchContext.domain;
          existing.activity = batchContext.description;
          existing.outcomes = nextOutcomes;
          existing.comment = v.comment || '';
          existing.thumb = '';
          updatedCount += 1;
        } else {
          const assessment = {
            id: v.assessmentId,
            date: today,
            ...cycleSnapshot,
            type: 'batch',
            domain: batchContext.domain,
            activity: batchContext.description,
            outcomes: nextOutcomes,
            comment: v.comment || '',
            thumb: ''
          };
          student.assessments.unshift(assessment);
          updatedCount += 1;
        }
      }
      const existingEntry = (session.students || []).find(entry =>
entry.assessmentId === v.assessmentId || entry.name === name);
      if (existingEntry) {
        existingEntry.name = name;
        existingEntry.domain = batchContext.domain;
        Object.assign(existingEntry, cycleSnapshot);
      }
    });
  }

```

```

        existingEntry.assessmentId = v.assessmentId;
    } else {
        session.students.push({ name, domain: batchContext.domain, assessmentId:
v.assessmentId, ...cycleSnapshot });
    }
    } else {
        const assessment = {
            id: Date.now().toString(36) + Math.random().toString(36).slice(2,5),
            date: today,
            ...cycleSnapshot,
            type: 'batch',
            domain: batchContext.domain,
            activity: batchContext.description,
            outcomes: nextOutcomes,
            comment: v.comment || '',
            thumb: ''
        };
        student.assessments.unshift(assessment);
        v.assessmentId = assessment.id;
        session.students.push({ name, domain: batchContext.domain, assessmentId:
assessment.id, ...cycleSnapshot });
        createdCount += 1;
    }
    });

    DB.saveData(data);

    if (batchEditingSessionId) {
        const totalChanged = updatedCount + createdCount;
        flash(`Saved changes to ${totalChanged} batch record${totalChanged !== 1 ? 's'
: ''}.`, 'success');
        const returnTo = batchEditReturn || { sourceTab: 'students', studentName: '' };
        resetBatchFlow();
        if (returnTo.sourceTab === 'profile' && returnTo.studentName) {
            showScreen('profile');
            document.querySelectorAll('.nav-btn').forEach(b =>
b.classList.remove('active'));
            document.getElementById('nav-students')?.classList.add('active');
            renderProfile(returnTo.studentName);
        } else {
            goTab('students');
            const panel = document.getElementById('history-panel');
            if (panel) panel.style.display = 'block';
            renderHistory();
        }
        return;
    }

    flash(`Saved for ${toSave.length} student${toSave.length !== 1 ? 's' : ''}.`,
'success');

```

```

    resetBatchFlow();
    goTab('record');
}

function resetBatchFlow() {
    batchInputType = 'text'; batchPhotoData = null; batchPDFData = null; batchContext
= null; batchStudentStatuses = {}; batchEditingSessionId = null; batchEditReturn =
null;
    document.getElementById('batch-desc').value = '';
    const batchClsEl = document.getElementById('batch-class-select'); if (batchClsEl)
batchClsEl.value = '';
    const batchNewEl = document.getElementById('batch-new-student'); if (batchNewEl)
batchNewEl.value = '';
    const batchStuEl = document.getElementById('batch-student-name'); if (batchStuEl)
batchStuEl.value = '';
    const batchStuSel = document.getElementById('batch-student-select'); if
(batchStuSel) batchStuSel.value = '';
    document.getElementById('batch-photo-preview').style.display = 'none';
    document.getElementById('batch-pdf-name').textContent = '';
    document.getElementById('batch-domain-select').value = '';
    document.getElementById('batch-outcome-picker').classList.remove('visible');
    const batchBanner = document.getElementById('batch-edit-banner'); if
(batchBanner) { batchBanner.textContent = ''; batchBanner.style.display = 'none'; }
    const batchSaveBtn = document.getElementById('batch-save-btn'); if (batchSaveBtn)
batchSaveBtn.textContent = '✓ Save activity record';
    setInputType('text');
    showBatchStep('input');
}

function showBatchStep(step) {
    document.getElementById('batch-step-input').style.display = step === 'input' ? ''
: 'none';
    document.getElementById('batch-step-grid').style.display = step === 'grid' ?
'block' : 'none';
}

function backToBatchInput() {
    if (batchEditingSessionId) {
        const returnTo = batchEditReturn || { sourceTab: 'students', studentName: '' };
        resetBatchFlow();
        if (returnTo.sourceTab === 'profile' && returnTo.studentName) {
            showScreen('profile');
            document.querySelectorAll('.nav-btn').forEach(b =>
b.classList.remove('active'));
            document.getElementById('nav-students')?.classList.add('active');
            renderProfile(returnTo.studentName);
        } else {
            goTab('students');
            const panel = document.getElementById('history-panel');
            if (panel) panel.style.display = 'block';
        }
    }
}

```

```

        renderHistory();
    }
    return;
}
showBatchStep('input');
}

```

// populateBatchStudentSelect defined above

```
let renamingStudent = null;
```

```
function openRenameStudent(name) {
    renamingStudent = name;
    document.getElementById('rename-input').value = name;
    document.getElementById('rename-modal').classList.add('open');
    setTimeout(() => document.getElementById('rename-input').select(), 100);
}

```

```
function closeRename() {
    document.getElementById('rename-modal').classList.remove('open');
    renamingStudent = null;
}

```

```

async function saveRename() {
    const newName = document.getElementById('rename-input').value.trim();
    if (!newName) { flash('Name cannot be empty.');
```

 return; }
 if (newName === renamingStudent) { closeRename(); return; }
 if (DB.getStudent(newName)) { flash(`"\${newName}" already exists – choose a
different name.`, "warn"); return; }
 const data = DB.loadData();
 if (data.students[renamingStudent]) {
 data.students[newName] = { ...data.students[renamingStudent], name: newName };
 delete data.students[renamingStudent];
 }
 if (data.sessions) {
 data.sessions.forEach(s => {
 s.students.forEach(e => { if (e.name === renamingStudent) e.name = newName;
 }));
 });
 DB.saveData(data);
 const old = renamingStudent;
 closeRename();
 renderRoster();
 renderProfile(newName);
}

```
function openSettings() {
```

```

    const s = DB.loadSettings();
    applyAssessCycle(s.preferredCycle || currentCycle || 'cycle3', s.yearLevel ||
currentYearLevel || '');
    document.getElementById('settings-key').value      = s.apiKey || '';
    document.getElementById('settings-context').value = s.defaultContext || '';
    document.getElementById('settings-modal').classList.add('open');
}
function saveSettings() {
    const key = document.getElementById('settings-key').value.trim();
    const ctx = document.getElementById('settings-context').value.trim();
    if (!key) { flash('API key cannot be empty.');
```

return; }
 const cycle = document.getElementById('settings-cycle')?.value || currentCycle;
 const year = document.getElementById('settings-year')?.value || currentYearLevel;
 applyAssessCycle(cycle, year);
 DB.saveSettings({ apiKey: key, defaultContext: ctx, preferredCycle: currentCycle,
yearLevel: currentYearLevel });
 window.__key = key;
 closeSettings();
}
function closeSettings() {
document.getElementById('settings-modal').classList.remove('open'); }

function exportData() {
 const data = DB.loadData();
 const blob = new Blob([JSON.stringify(data, null, 2)], { type: 'application/json'
});
 const url = URL.createObjectURL(blob);
 const today = new Date().toISOString().split('T')[0];
 const a = Object.assign(document.createElement('a'), { href: url, download:
'CycleAssess_backup_' + today + '.json' });
 a.click();
 URL.revokeObjectURL(url);
}

async function importData(e) {
 const file = e.target.files[0];
 if (!file) return;
 // Confirm before reading – await must be outside the FileReader callback
 const ok = await appConfirm('Import backup?', 'This will merge with your existing
data. Students with the same name will have assessments combined.', 'Import',
'btn-primary');
 if (!ok) { e.target.value = ''; return; }
 const reader = new FileReader();
 reader.onload = ev => {
 try {
 const data = JSON.parse(ev.target.result);
 if (!data.students) throw new Error('Invalid backup file – missing student
data.');

```

        if (!existing.students[name]) {
            existing.students[name] = st;
        } else {
            const existingIds = new Set(existing.students[name].assessments.map(a =>
a.id));
            st.assessments.forEach(a => { if (!existingIds.has(a.id))
existing.students[name].assessments.push(a); });
            existing.students[name].assessments.sort((a,b) =>
b.date.localeCompare(a.date));
            if (st.notes && !existing.students[name].notes)
existing.students[name].notes = st.notes;
        }
    });
    if (data.sessions) {
        if (!existing.sessions) existing.sessions = [];
        const existingSessionIds = new Set(existing.sessions.map(s => s.id));
        data.sessions.forEach(s => { if (!existingSessionIds.has(s.id))
existing.sessions.push(s); });
        existing.sessions.sort((a,b) => b.date.localeCompare(a.date));
    }
    DB.saveData(existing);
    closeSettings();
    flash('Import successful – data merged.', 'success');
    renderRoster();
} catch(err) {
    flash('Import failed: ' + err.message, 'error');
}
};
reader.readAsText(file);
}

```

/* — Excel colours — */

```

const XL_COLORS = {
    met: { fill: 'FFD6F0E0', font: 'FF1e8c5a', label: 'Met' },
    wt: { fill: 'FFFFFF3CD', font: 'FFc47a10', label: 'Working Towards' },
    nm: { fill: 'FFFDE8E8', font: 'FFc0272d', label: 'Not Met' },
    hdr: { fill: 'FF2D3561', font: 'FFFFFFFF' },
    sub: { fill: 'FFE8ECF8', font: 'FF1c1f2e' },
};

```

```

function xlFill(hex) { return { type: 'pattern', pattern: 'solid', fgColor: {
argb: hex } }; }
function xlFont(hex, bold=false, size=11) { return { color: { argb: hex }, bold,
size, name: 'Calibri' }; }
function xlBorder() {
    const s = { style: 'thin', color: { argb: 'FFD0D5E8' } };
    return { top: s, left: s, bottom: s, right: s };
}

```

```

function buildWorkbook(students) {
  const wb = new ExcelJS.Workbook();
  wb.creator = 'CycleAssess';
  wb.created = new Date();

  /* — OVERVIEW sheet — */
  const ws = wb.addWorksheet('All Students', { views: [{ state: 'frozen', ySplit: 2
}] });
  ws.columns = [
    { key: 'name', width: 18 },
    { key: 'date', width: 14 },
    { key: 'domain', width: 22 },
    { key: 'fr', width: 44 },
    { key: 'en', width: 44 },
    { key: 'status', width: 18 },
    { key: 'comment', width: 36 },
  ];

  // Title row
  ws.mergeCells('A1:G1');
  const titleCell = ws.getCell('A1');
  titleCell.value = 'CycleAssess — Class Overview · ' + new
Date().toLocaleDateString('en-GB', {day:'numeric',month:'long',year:'numeric'});
  titleCell.fill = xlFill(XL_COLORS.hdr.fill);
  titleCell.font = xlFont(XL_COLORS.hdr.font, true, 13);
  titleCell.alignment = { horizontal: 'center', vertical: 'middle' };
  ws.getRow(1).height = 26;

  // Header row
  const hdrs = ['Student', 'Date', 'Domain', 'Outcome (FR)', 'Outcome
(EN)', 'Status', 'Comment'];
  const hdrRow = ws.addRow(hdrs);
  hdrRow.eachCell(cell => {
    cell.fill = xlFill(XL_COLORS.sub.fill);
    cell.font = xlFont(XL_COLORS.sub.font, true, 11);
    cell.border = xlBorder();
    cell.alignment = { vertical: 'middle', wrapText: true };
  });
  hdrRow.height = 20;

  // Data rows
  let lastStudent = '';
  students.forEach(st => {
    st.assessments.forEach(a => {
      a.outcomes.forEach((o, oi) => {
        const isFirst = oi === 0;
        const row = ws.addRow({
          name: isFirst ? st.name : '',
          date: isFirst ? formatDate(a.date) : '',
          domain: isFirst ? a.domain : '',

```

```

        fr:      o.fr,
        en:      o.en,
        status:  XL_COLORS[o.status]?.label || o.status,
        comment: isFirst && a.comment ? a.comment : '',
    });

    const colors = XL_COLORS[o.status] || XL_COLORS.wt;
    row.eachCell({ includeEmpty: true }, (cell, colNum) => {
        cell.border = xlBorder();
        cell.alignment = { vertical: 'middle', wrapText: true };
        cell.font = xlFont('FF1c1f2e', false, 11);
    });

    // Colour the status cell
    const statusCell = row.getCell(6);
    statusCell.fill = xlFill(colors.fill);
    statusCell.font = xlFont(colors.font, true, 11);
    statusCell.alignment = { horizontal: 'center', vertical: 'middle' };

    // Subtle stripe when student changes
    if (st.name !== lastStudent) {
        row.getCell(1).font = xlFont('FF1c1f2e', true, 11);
    }
    row.height = 18;
    });
    lastStudent = st.name;
});
});

/* — Per-student sheets — */
students.forEach(st => {
    const wsS = wb.addWorksheet(st.name.slice(0, 31));
    wsS.columns = [
        { key: 'date',    width: 14 },
        { key: 'domain',  width: 22 },
        { key: 'fr',      width: 40 },
        { key: 'en',      width: 40 },
        { key: 'status',  width: 18 },
        { key: 'comment', width: 36 },
    ];

    // Title
    wsS.mergeCells('A1:F1');
    const t = wsS.getCell('A1');
    t.value = st.name + (st.notes ? ' | ' + st.notes : '');
    t.fill = xlFill(XL_COLORS.hdr.fill);
    t.font = xlFont(XL_COLORS.hdr.font, true, 13);
    t.alignment = { horizontal: 'left', vertical: 'middle', indent: 1 };
    wsS.getRow(1).height = 26;

```



```

    // Headers
    const h = wsS.addRow(['Date', 'Domain', 'Outcome (FR)', 'Outcome
(EN)', 'Status', 'Comment']);
    h.eachCell(cell => {
        cell.fill = xlFill(XL_COLORS.sub.fill);
        cell.font = xlFont(XL_COLORS.sub.font, true, 11);
        cell.border = xlBorder();
        cell.alignment = { vertical: 'middle', wrapText: true };
    });
    h.height = 20;

    st.assessments.forEach(a => {
        a.outcomes.forEach((o, oi) => {
            const colors = XL_COLORS[o.status] || XL_COLORS.wt;
            const row = wsS.addRow({
                date: oi === 0 ? formatDate(a.date) : '',
                domain: oi === 0 ? a.domain : '',
                fr: o.fr,
                en: o.en,
                status: colors.label,
                comment: oi === 0 && a.comment ? a.comment : '',
            });
            row.eachCell({ includeEmpty: true }, cell => {
                cell.border = xlBorder();
                cell.alignment = { vertical: 'middle', wrapText: true };
                cell.font = xlFont('FF1c1f2e', false, 11);
            });
            const sc = row.getCell(5);
            sc.fill = xlFill(colors.fill);
            sc.font = xlFont(colors.font, true, 11);
            sc.alignment = { horizontal: 'center', vertical: 'middle' };
            row.height = 18;
        });
    });
});
return wb;
}

```

```

async function exportAllExcel() {
    const students = studentsForActiveCycle(DB.getAllStudents());
    if (!students.length) { flash('No student data to export for the active cycle.',
'warn'); return; }
    const wb = buildWorkbook(students);
    const today = new Date().toISOString().split('T')[0];
    const buf = await wb.xlsx.writeBuffer();
    const blob = new Blob([buf], { type:
'application/vnd.openxmlformats-officedocument.spreadsheetml.sheet' });
    const url = URL.createObjectURL(blob);
    const a = Object.assign(document.createElement('a'), { href: url, download:

```

```

'CycleAssess_All_' + today + '.xlsx' });
  a.click();
  URL.revokeObjectURL(url);
}

async function exportStudentExcel(name) {
  const st = studentForActiveCycle(DB.getStudent(name));
  if (!st) return;
  if (!st.assessments.length) { flash('No assessments to export for the active
cycle.', 'warn'); return; }
  const wb = buildWorkbook([st]);
  // Remove the overview sheet for individual export – just keep the student sheet
  // Actually keep both – overview has 1 student which is fine
  const today = new Date().toISOString().split('T')[0];
  const buf = await wb.xlsx.writeBuffer();
  const blob = new Blob([buf], { type:
'application/vnd.openxmlformats-officedocument.spreadsheetml.sheet' });
  const url = URL.createObjectURL(blob);
  const a = Object.assign(document.createElement('a'), { href: url, download:
'CycleAssess_' + name.replace(/\s+/g, '_') + '_' + today + '.xlsx' });
  a.click();
  URL.revokeObjectURL(url);
}

async function clearAssessmentsOnly() {
  if (!await appConfirm('Clear all assessments across all cycles?', 'Students stay
in the roster, but assessment records, history and session logs from every cycle
will be deleted. Cannot be undone.', 'Clear all cycles', 'btn-ghost')) return;
  const data = DB.loadData();
  Object.values(data.students).forEach(st => { st.assessments = []; });
  data.sessions = [];
  DB.saveData(data);
  closeSettings();
  renderRoster();
  flash('All assessments cleared. Students kept.', 'success');
}

async function clearAllData() {
  if (!await appConfirm('Delete all data across all cycles?', 'All students,
assessments and history from every cycle will be permanently deleted. Cannot be
undone.', 'Delete everything', 'btn-ghost')) return;
  localStorage.removeItem(DB.KEY);
  closeSettings();
  renderRoster();
  renderHeatmap();
  flash('All student data cleared.', 'success');
}

/* =====
  UTILS

```

```

===== */
function esc(s) {
  if (s === undefined || s === null) return '';
  return
String(s).replace(/&/g, '&').replace(/</g, '<').replace(/>/g, '>').replace(/
"/g, '"');
}
function escAttr(s) { return esc(s).replace(/'/g, '&#39;'); }
function getOutcomeDisplayPrimary(outcome) {
  const en = String(outcome?.en || '').trim();
  const fr = String(outcome?.fr || '').trim();
  return en || fr;
}
function getOutcomeDisplaySecondary(outcome) {
  const en = String(outcome?.en || '').trim();
  const fr = String(outcome?.fr || '').trim();
  if (!fr || fr === en) return '';
  return fr;
}
function formatDate(iso) {
  if (!iso) return '-';
  const [y,m,d] = iso.split('-');
  const months =
['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'];
  return `${parseInt(d)} ${months[parseInt(m)-1]} ${y}`;
}
let _toastTimer = null;
function flash(msg, type='') {
  const t = document.getElementById('app-toast');
  t.textContent = msg;
  t.className = 'toast' + (type ? ' toast-' + type : '');
  t.classList.add('show');
  if (_toastTimer) clearTimeout(_toastTimer);
  _toastTimer = setTimeout(() => t.classList.remove('show'), 3000);
}

function appConfirm(msg, sub, okLabel, okClass) {
  return new Promise(resolve => {
    document.getElementById('confirm-msg').textContent = msg;
    document.getElementById('confirm-sub').textContent = sub || '';
    const okBtn = document.getElementById('confirm-ok-btn');
    const cancelBtn = document.getElementById('confirm-cancel-btn');
    okBtn.textContent = okLabel || 'Confirm';
    okBtn.className = 'btn ' + (okClass || 'btn-primary');
    const overlay = document.getElementById('confirm-overlay');
    overlay.classList.add('open');
    const close = (result) => {
      overlay.classList.remove('open');
      okBtn.replaceWith(okBtn.cloneNode(true));
      cancelBtn.replaceWith(cancelBtn.cloneNode(true));
    }
  });
}

```

```

        resolve(result);
    };
    document.getElementById('confirm-ok-btn').onclick = () => close(true);
    document.getElementById('confirm-cancel-btn').onclick = () => close(false);
});
}

// Close modals on overlay click
document.getElementById('settings-modal').addEventListener('click', function(e) {
    if(e.target===this) closeSettings(); });
document.getElementById('edit-modal').addEventListener('click', function(e) {
    if(e.target===this) closeEditModal(); });
document.getElementById('add-student-modal').addEventListener('click', function(e)
{ if(e.target===this) closeAddStudent(); });
const _rm = document.getElementById('rename-modal'); if(_rm)
_rm.addEventListener('click', function(e) { if(e.target===this) closeRename(); });

/* — Boot — */
if (window.SylCycle && SylCycle.onReady) SylCycle.onReady(initApp);
else initApp();
</script>
</body>
</html>

```